Ministère de l'Enseignement Supérieur,
de la Recherche Scientifique et de la
Technologie
****
Université de Carthage
****
**Institut National des Sciences
Appliquées et de Technologie**

Projet de Fin d'Etudes

Pour l'obtention du

Diplôme National d'Ingénieur

Filière : Génie Logiciel

Sujet :

Refonte architecturale d'une plateforme monolithique de pilotage en utilisant des microservices

Réalisé par : **Salma DALDOUL**

Entreprise d'accueil :

Sofrecom
The Know-How Network

Soutenu le 17/06/2016

Président: **Mme Faten CHAIEB**
Examinateur: **Mme Ghada GASMI**
Responsable INSAT: **Mme Sonia BOUZIDI**
Responsable Entreprise: **M. Mohamed Aymen HOUISSA**
Responsable Entreprise: **M. Sabri MTIBAA**

Année Universitaire : 2015/2016

Ministère de l'Enseignement Supérieur,
de la Recherche Scientifique et de la
Technologie
****
Université du Carthage
****
**Institut National des Sciences
Appliquées et de Technologie**

Projet de Fin d'Etudes

Pour l'obtention du

Diplôme National d'Ingénieur

Filière : Génie Logiciel

Sujet :

Refonte architecturale d'une plateforme monolithique de pilotage en utilisant des microservices

Réalisé par : **Salma DALDOUL**

Entreprise d'accueil :

Sofrecom
The Know-How Network

Soutenu le 17/06/2014

Les Responsables à l'Entreprise	*Le Responsable à l'INSAT*
Monsieur **Mohamed Aymen HOUISSA** Monsieur **Sabri MTIBAA**	Madame **Sonia BOUZIDI**
Cachet & Signature	Signature

Année Universitaire : 2015/2016

Dédicaces

A ceux qui m'ont mis sur la bonne route, et à qui je dois la femme que je suis… A mes chers parents, Jaafer et Monia

A celui qui m'a fait aimer l'informatique dès mon enfance… A mon grand frère et mon ami, Eslim

A celui qui m'a accompagné durant ces cinq ans… A mon frère et binôme, Sami

A celles qui m'apportent la joie et le bonheur… A mes nièces, Sirine et Lyne

A celui qui me considère comme sa fille… A mon cher oncle, Montasar

A celle qui attend le jour où je serai ingénieur… A ma grand-mère, Emna

A ceux qui ont toujours été ma source de fierté, d'aide et d'encouragement… A ma tendre famille

A celui que je considère mon troisième frère et avec qui j'ai partagé les moments de douleur et du bonheur durant ces cinq années… A mon binôme, Hamza

A celle qui est toujours près de moi et avec qui je partage les repas, les chocolats et les moments de folie… A mon amie et soeur, Aroua

A celle qui est gravée dans mon cœur et qui me comprend et me fait confiance… A mon amie, Mouna

A ceux qui je considère comme ma deuxième famille..... A mes adorables amis.

A ceux qui ont toujours été mes sources d'inspiration… A mes professeurs pendant mon parcourt éducatif de l'école vers l'INSAT

A toutes les personnes qui me sont chères, et à toutes celles qui veulent partager mon bonheur,

Je dédie ce modeste travail, symbole de ma profonde gratitude et couronnement de leur assistance

Remerciements

J'exprime ses sincères reconnaissances à l'égard de tous ceux qui ont contribué à ma formation.

Je tiens à remercier tout particulièrement Monsieur Mohamed Aymen HOUISSA mon tuteur de stage qui a dirigé mon projet de fin d'études, pour son engagement permanent, son soutien constant et la confiance totale qu'il m'a accordée.

J'adresse aussi mes profonds remerciements à Madame Sonia BOUZIDI, mon encadrante au sein de l'INSAT, pour son encadrement, son soutien, sa disponibilité et ses conseils qui m'ont guidé tout au long de mon stage.

Je voudrais témoigner par la suite ma reconnaissance à Monsieur Sabri MTIBAA, qui malgré ses responsabilités et ses occupations, a toujours eu le temps pour m'écouter et me faire bénéficier de ses conseils et de son expérience.

Je tiens à remercier également Monsieur Ammar MISSAOUI, pour l'agréable intégration au sein de Sofrecom.

Je remercie également toute l'équipe Portail UPR chez Sofrecom et Orange pour leur accueil et suggestions

Je tiens à remercier vivement Madame Lilia SFAXI, pour la bonne préparation théorique à ce projet ainsi que ses conseils en or.

Je tiens, par ailleurs, à exprimer ma profonde gratitude et mon immense reconnaissance à :

Madame Faten CHAIB, pour l'honneur qu'elle a bien voulu me faire en acceptant de présider le jury chargé d'évaluer ce travail.

Et Madame Ghada GASMI, d'avoir accepté d'examiner ce travail.

J'espère que le présent projet soit à la hauteur de vos attentes.

TABLE DES MATIERES

Liste des figures vi

Liste des tableaux ix

INTRODUCTION GENERALE 1

CADRE GENERAL DU TRAVAIL 3

Introduction 3
1. Présentation de l'organisme d'accueil 3
   1.1. Le groupe Sofrecom 3
   1.2. Sofrecom Tunisie 4
2. Cadre du projet 4
   2.1. Contexte général de projet: Sur la route vers DevOps 5
   2.2. Présentation du projet Portail de pilotage UPR 5
   2.3. Diagnostic de l'existant : Architecture Monolithique 5
3. Méthodologie et approche de développement 8
   3.1. La méthodologie SCRUM 9
   3.2. L'approche DevOps 12

Conclusion 14
LES CONCEPTS DE L'ARCHITECTURE MICROSERVICES 15

Introduction 15
1. Les concepts liés à l'architecture microservices 15
   1.1. Domain Driven Design 15
   1.2. Développement à base de composants 16
   1.3. Polyglot persistence 17
2. L'architecture Microservice 18
   2.1. Définition de l'architecture microservices 18
   2.2. Caractéristiques de l'architecture microservices 18
   2.3. Bilan sur l'architecture microservices 21
3. Microservices en pratique: isolation des microservices dans des conteneurs 22

Conclusion 23
SPECIFICATION DES BESOINS ET URBANISATION FONCTIONNELLE EN MICROSERVICE 24

Introduction 24
1. Recueil des besoins Fonctionnels globaux 24
1.1. Les besoins fonctionnels 24
1.2. Identification des acteurs 27
1.3. Backlog produit 27
1.4. Architecture fonctionnelle générale 29
1.5. Modélisation des cas d'utilisation 30
2. Urbanisation fonctionnelle en microservices 31
3. Contraintes non fonctionnelles 33
Conclusion 34
SPRINT 1 : MICROSERVICE ADMINISTRATION 35
Introduction 35
1. Orientation technologique 35
2. Étude fonctionnelle de l'administration 36
3. Conception du microservice administration 40
3.1. Architecture logicielle de microservice administration 40
3.2. Conception générale de microservice administration : Diagrammes de package 40
3.3. Modèle relationnel de microservice administration 41
3.4. Structure globale de microservice administration 42
4. Architecture technique de microservice administration 44
5. Réalisation de microservice administration 44
6. Phase de test de microservice administration 45
Conclusion 46
SPRINT 2 : MICROSERVICE PORTEFEUILLE 48
Introduction 48
1. Étude fonctionnelle du microservice portefeuille 48
2. Conception de microservice portefeuille 50
2.1. Architecture logicielle de microservice portefeuille 50
2.2. Modèle relationnel de microservice portefeuille 50
2.3. Structure globale de microservice portefeuille 51
2.4. Modélisation dynamique de cas d'utilisation modifier portefeuille 53
2.5. Diagramme de déploiement de microservice portefeuille 54
3. Configuration logicielle de microservice portefeuille 54
4. Réalisation de microservice portefeuille 55
5. Test de microservice portefeuille 56
Conclusion 57
SPRINT 3 : MICROSERVICE REPORTING 58

Introduction 58
1. Étude fonctionnelle de microservice reporting 58
2. Solution pour l'amélioration de temps de réponse 60
3. Conception de microservice reporting 62
3.1. Architecture logicielle de microservice reporting 62
3.2. Modèle de données de la base NOSQL orienté document 62
3.3. Structure globale de microservice reporting 65
4. Choix technologiques de reporting 65
4.1. Base de données : MongoDB 65
4.2. Framework d'accès aux données 66
5. Réalisation de microservice reporting 67
6. Test de microservice reporting 68
Conclusion 68
INTEGRATION DU PORTAIL UPR 69
Introduction 69
1. Intégration des microservices 69
1.1. Service de découverte 70
1.2. Navigation entre les microservices 72
1.3. Communication entre les microservices 77
2. Monitoring applicatif et des temps de réponses 83
2.1. Traçage distribué : implémentation avec Spring Cloud Sleuth 84
2.2. Supervision statique avec Twitter Zipkin 85
2.3. Supervision dynamique : Indexation des logs avec ELK 89
Conclusion 92
DEPLOIEMENT DU PORATIL UPR 93
Introduction 94
1. Déploiement sur le Platform-As-A-Service (PaaS) Cloud Foundry 94
2. Préparation des conteneurs de déploiement (Docker) 96
3. Monitoring des conteneurs Docker 101
Conclusion 105
CONCLUSION ET PRESPECTIVES 106
Bibliographie 108
Webographie 109
Glossaire 110
Glossaire métier 114
Annexes ..... - 117 -

LISTE DES FIGURES

Figure 1 Carte des filiales et clients de Sofrecom 3
Figure 2 Evolution des effectifs de Sofrecom 4
Figure 3 Architecture monolithique Portail Pilotage UPR 7
Figure 4 Agilité du bout en bout 8
Figure 5 Pratique SCRUM: vue d'ensemble 9
Figure 6 Définition DevOps 13
Figure 7 Automatisation DevOps 14
Figure 8 Division du domaine en Bounded Contexts 16
Figure 9 Carte de contexte 16
Figure 10 Architecture à base des composants 17
Figure 11 Décomposition selon le style d'architecture 19
Figure 12 Démarche d'urbanisation du Portail UPR 25
Figure 13 Architecture fonctionnelle générale du Portail UPR 30
Figure 14 Diagramme de cas d'utilisation global 31
Figure 15 Modèle métier du portail UPR 32
Figure 16 Carte de contexte du pilotage UPR 32
Figure 17 Diagramme de cas d'utilisation de l'administration 37
Figure 18 Architecture logicielle de microservice administration 40
Figure 19 Diagramme de package coté client 41
Figure 20 Diagramme de package coté serveur 41
Figure 21 Modèle de données de microservice administration 42
Figure 22 Structure globale de microservice administration 43
Figure 23 Architecture technique de microservice administration 44
Figure 24 Interface d'administration 44
Figure 25 Aperçu sur l'exécution des tests unitaires de microservice administration 45
Figure 26 Diagramme de cas d'utilisation de gestion de portefeuille 48
Figure 27 Architecture logicielle de microservice portefeuille 50
Figure 28 Modèle de domaine de gestion de portefeuille 51
Figure 29 Structure globale de microservice portefeuille 52
Figure 30 Diagramme de séquence : Modifier un portefeuille 53
Figure 31 Diagramme de déploiement de microservice portefeuille 54
Figure 32 Interface de gestion de portefeuille 56

Figure 33 Test unitaire de microservice portefeuille 57
Figure 34 Diagramme de cas d'utilisation de reporting 58
Figure 35 Propriétés des bases relationnelles et NOSQL 61
Figure 36 Architecture logicielle de microservice reporting 62
Figure 37 Structure des documents de reporting sans dénormalisation 63
Figure 38 Modèle qui illustre la dénormalisation de la base de reporting 64
Figure 39 Structure globale de microservice reporting 65
Figure 40 Capture avec Robomongo sur le document OpusOperation 66
Figure 41 Table d'analyse des opérations de déploiement 67
Figure 42 Liste des opérations de déploiement 68
Figure 43 Test unitaire de microservice reporting 68
Figure 44 Architecture technique portail UPR microservices 70
Figure 45 Architecture de service de découverte Eureka 2.0 71
Figure 46 Enregistrement des microservices du Portail UPR dans l'annuaire 71
Figure 47 Interface de service de découverte Eureka 72
Figure 48 API Gateway pattern sur la portail UPR 73
Figure 49 Fonctionnement de l'API Gateway 73
Figure 50 Interface de l'administration à partir de l'API Gateway 74
Figure 51 Interface de paramétrage portefeuille à partir de l'API Gateway 74
Figure 52 Table d'analyse à partir de l'API Gateway 75
Figure 53 Données des opérations à partir de l'API Gateway 75
Figure 54 Anticipation de Circuit Breaker au niveau de l'API Gateway 76
Figure 55 Tolérance aux pannes du microservice reporting 76
Figure 56 Load balancing avec Netflix Ribbon 77
Figure 57 Communication synchrone entre les microservices administration et portefeuille . 78
Figure 58 Patron circuit breaker 79
Figure 59 Hystrix Dashboard quand le microservice administrateur est lancé 80
Figure 60 Hystrix Dashboard quand le microservice administrateur a échoué 80
Figure 61 Communication synchrone avec rabbitMq 82
Figure 62 Liste des queues 83
Figure 63 Cycle de vie de span 85
Figure 64 Architecture twitter zipkin 86
Figure 65 Supervision distribué avec twitter zipkin 86
Figure 66 Dépendance entre les microservices portefeuille et administration 87
Figure 67 Supervision des traces dans zipkin 87
Figure 68 Trace d'une communication asynchrone 88

Figure 69 Appel distant de rest api 88
Figure 70 Trace des envois de message avec rabbitmq 89
Figure 71 Détails sur une communication asynchrone entre administration et reporting 89
Figure 72 Structure du fichier logstash.conf 90
Figure 73 Indexation des log avec ELK 91
Figure 74 Exemple de graphe avec kibana 91
Figure 75 Exemple de dashbord avec kibana pour le portail UPR Microservices 92
Figure 76 Architecture Cloud Foundry 94
Figure 77 Déploiement au PaaS Cloud Foundry 95
Figure 78 Architecture docker 96
Figure 79 Les commandes docker 98
Figure 80 Conteneur du portail UPR 100
Figure 81 Monitoring des conteneurs docker 101
Figure 82 Ressources alloué à la machine ubuntu 102
Figure 83 Aperçu général sur la consommation avant le lancement des conteneurs 102
Figure 84 Aperçu général sur la consommation après le lancement des conteneurs 103
Figure 85 Aperçu général sur la consommation de conteneur de reporting seul 103
Figure 86 Usage de CPU par le conteneur reporting 104
Figure 87 Usage de mémoire par le conteneur reporting 104
Figure 88 Usage de réseau par le conteneur reporting 105

LISTE DES TABLEAUX

Tableau 1: Inconvénients de l'architecture monolithique 6
Tableau 2 SCRUM : pré-requis, difficultés et bénéfices 11
Tableau 3 Backlog de produit 28
Tableau 4 Partitionnement des user stories par sprint 33
Tableau 5 Apport de Spring cloud pour l'architecture microservice 35
Tableau 6 Description détaillée de cas d'utilisation "gérer les critères métiers" 37
Tableau 7 Description détaillée de cas d'utilisation "Créer des données locales complémentaires" 38
Tableau 8 Description détaillée de cas d'utilisation "Définir les seuils critiques" 39
Tableau 9 Les cas de test pour l'importation des jalons 45
Tableau 10 description de cas d'utilisation "Gérer son portefeuille" 48
Tableau 11 description de cas d'utilisation "Définir son équipe" 49
Tableau 12 Cas d'utilisation : Consulter la synthèse des opérations d'une famille 59
Tableau 13 Cas d'utilisation : Consulter la liste des opérations 60
Tableau 14 Règles de dénormalisation 63
Tableau 15 Synthèse RabbitMq Vs ActiveMq Vs Apache kafka 81
Tableau 16 les intérêts et les avantages de docker 97
Tableau 17 Inconvénients de Docker 98
Tableau 18 Les bonnes pratiques avec docker 100

INTRODUCTION GENERALE

La mutation des grandes entreprises au sein du monde numérique génère de profonds changements sur leur stratégie, leurs modèles d'affaires et leur gouvernance. Avec l'évolution technologique, la gouvernance de l'entreprise ne se limite plus à la gestion des ressources matérielles et humaines, mais aussi à la gestion des cycles de vie de ces projets.

La façon classique de gestion de projet se base sur un cycle linéaire qui consiste à recueillir les besoins, définir le produit, le développer et le tester avant de le livrer. Dans ce cas, les risques sont détectés tardivement et le client n'est pas satisfait. Tandis que, depuis des années, l'entreprise suivit une course folle à l'innovation. Elle tente de réagir vite et mieux, adapter son organisation, ses produits et services aux enjeux du marché et anticiper ses évolutions. Face à la course au renouvellement, les défis à relever sont donc, comment livrer un logiciel plus rapidement répondant véritablement aux besoins des utilisateurs et comment mettre à jour une application avec le moindre risque.

Maintenant, les freins ne sont plus technologiques, mais ils tiennent plus à l'organisation de la Direction des Systèmes d'Information (DSI), à la résistance au changement, et aux styles d'architectures rigides des Systèmes d' Information (SI). Par conséquent, les entreprises ont besoin de prendre la décision d'évolution et de transformer leurs (SI) pour qu'ils permettent de soutenir les tendances technologiques et s'adapter aux évolutions.

En raison de son appartenance au marché des télécommunications qui est très concurrentiel, Orange groupe cherche à avoir des systèmes d'information non seulement qui répondent à ses besoins variables mais encore qui s'adaptent rapidement aux évolutions requises par son environnement actif. Ainsi Sofrecom doit présenter des solutions anticipatives pour répondre aux besoins de son partenaire et plus important client.

C'est dans ce cadre que s'inscrit notre Projet de Fin d' Études ; Il consiste à piloter un projet de transformation d'un SI qui est un Portail de pilotage de déploiement réseau. Notre contribution aide donc à démontrer l'ensemble des méthodes, outils et bonnes pratiques à mettre en œuvre, afin d'aboutir à des SI maitrisés et évolutifs. Ceci pour arriver à offrir aux clients des

livraisons récurrentes des versions stables et une fréquence de déploiement renforcée. C'est pourquoi, nous mettons en œuvre une architecture en microservices.

Le présent rapport qui décrit nos travaux pour la refonte du Portail Pilotage UPR est structuré en huit principaux chapitres.

Le premier chapitre présente le contexte général et le cadre du projet. Il permet d'exposer les objectifs à atteindre et l'existant de son environnement.

Le deuxième chapitre présente théoriquement les concepts utiles pour l'élaboration de notre projet principalement une couverture des caractéristiques de base de l'architecture Microservices.

Le troisième chapitre est consacré à la spécification fonctionnelle générique de Portail Pilotage UPR. Il montre la subdivision en sous domaines fonctionnels des besoins de Portail UPR.

Le quatrième, cinquième et sixième chapitres décrivent les cycles de vie de constitution des différents microservices. Pour chaque microservice nous décrivons les fonctionnalités, la conception et la réalisation.

Le septième chapitre se concentre sur le chantier concerné par l'action de refonte de toute l'application. Il décrit l'application dans sa totalité, présente les contrats de communication entre les microservices et montre la phase de supervision applicative et de temps de réponse.

Le huitième et dernier chapitre recouvre la phase de déploiement de plateforme de pilotage et un monitoring des paramètres systèmes.

Et pour conclure, nous terminons par une synthèse de cette expérience et une illustration des perspectives.

CHAPITRE I

CADRE GENERAL DU TRAVAIL

Sommaire

Introduction 3
1. Présentation de l'organisme d'accueil 3
1.1. Le groupe Sofrecom 3
1.2. Sofrecom Tunisie 4
2. Cadre du projet 4
2.1. Contexte général de projet: Sur la route vers DevOps 5
2.2. Présentation du projet Portail de pilotage UPR 5
2.3. Diagnostic de l'existant : Architecture Monolithique 5
2.3.1. Impact de l'architecture monolithique sur le portail UPR 5
2.3.2. Impact de modèle de données sur les temps de réponse 7
3. Méthodologie et approche de développement 8
3.1. La méthodologie SCRUM 9
3.2. L'approche DevOps 12
Conclusion 14

INTRODUCTION

Ce chapitre introductif est consacré à la présentation de l'organisme d'accueil ainsi que le cadre général du projet. Nous exposons, dans un premier temps, le groupe Sofrecom et sa plus jeune filiale SofrecomTunisie. Nous présentons par la suite, l'existant et les enjeux de l'entreprise dans le but de dévoiler les objectifs de notre travail. Nous expliquons, enfin, la méthode adoptée pour y arriver et la démarche suivie lors de ce projet.

1. PRESENTATION DE L'ORGANISME D'ACCUEIL

1.1. **Le groupe Sofrecom**

Sofrecom est une filiale d'Orange qui a développé depuis plus de 50 ans un savoir-faire unique dans les métiers de l'opérateur, qui en fait aujourd'hui l'un des leaders mondiaux du conseil et de l'ingénierie. Ces dernières années, plus de 200 acteurs majeurs, dans plus de 100 pays, ont confié à Sofrecom la conduite de leurs projets stratégiques et opérationnels : transformation et optimisation, modernisation technologique, innovation et développement comme le montre la carte de la figure 1.

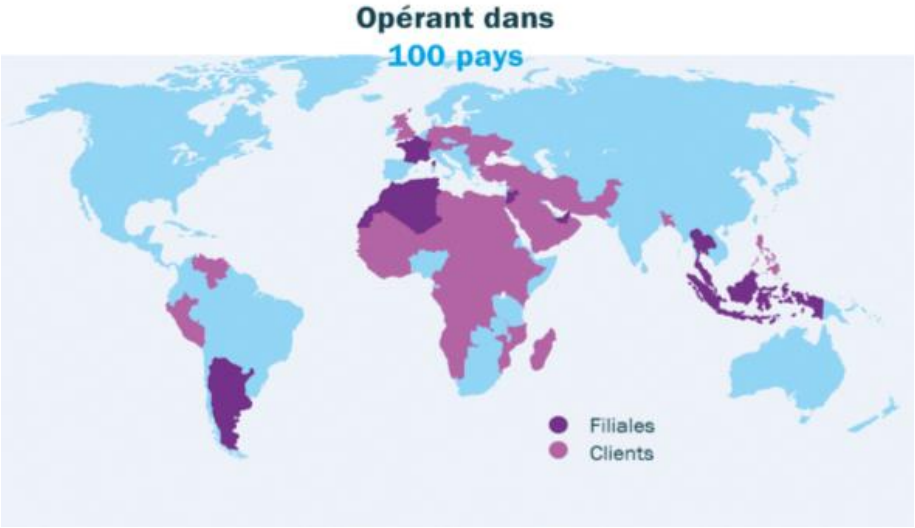


Figure 1 Carte des filiales et clients de Sofrecom

Sofrecom est une filiale du groupe Orange, un des 10 premiers opérateurs avec :

- 244 millions de clients
- 220 pays et territoires
- 15 Otange Labs et technocentres dans le monde
- 156 000 employés dans le monde

Sofrecom soutient le groupe Orange dans son développement international, en pilotant notamment des lancements d'opérateurs.

1.2. **Sofrecom Tunisie**

Lancée en novembre 2011, Sofrecom Tunisie est la plus jeune des filiales du groupe Sofrecom. Elle compte aujourd'hui 300 salariés et étend la présence de Sofrecom sur la zone Afrique du Nord et Moyen-Orient afin de répondre à une demande croissante de solutions dédiées et compétitives.

Sofrecom Tunisie est un centre de développement et d'expertise pour les plates-formes de services et du système d'information, qui s'inscrit en complémentarité des domaines d'activité des autres implantations de Sofrecom dans la région et permettra aux clients du groupe Sofrecom de bénéficier des synergies entre les filiales.

Les ambitions de Sofrecom en tunisie sont :

- développer un centre de Nearshore de référence sur le marché Tunisien,
- développer les activités de conseil en télécommunications du groupe Sofrecom et du groupe Orange et, a la volonté d'apporter son expertise dans les projets de digitalisation de la Tunisie, notamment dans les domaines de l'e-gouvernement, l'e-éducation, l'e-santé, e-banking…

Sofrecom Tunisie prévoit de recruter 200 ingénieurs supplémentaires dans ces deux ans à venir. La figure 2 présente l'évolution du nombre des effectifs depuis 2011.

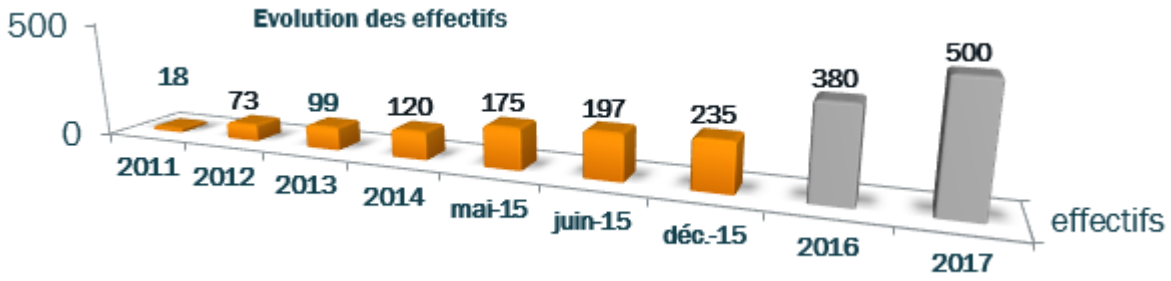


Figure 2 Evolution des effectifs de Sofrecom

2. CADRE DU PROJET

Notre projet de migration en microservice vise à trouver des solutions aux limites des applications monolithiques en favorisant les tendances DevOps.

2.1. **Contexte général de projet: Sur la route vers DevOps**

Passer vers une culture DevOps est aujourd'hui au cœur des stratégies du groupe Orange. Véritable prolongement des méthodes agiles, l'approche DevOps vise à aboutir à une agilité de bout en bout.

Le terme DevOps correspond au mélange des tâches qu'effectuent les équipes d'une entreprise chargées du développement des applications (Dev) et de l'exploitation des systèmes (Ops, pour Operations). La mise en œuvre de cette approche recourt à des outils de gestion du développement, gestion de versioning, gestion de l'intégration continue, gestion de la qualité de codes, gestion des tests, supervision, gestion de conteneur, et engendre des complexités techniques. De plus, elle est fortement dépendante de l'architecture logicielle adoptée.

2.2. **Présentation du projet Portail de pilotage UPR**

Portail de Pilotage Déploiement Réseau est un projet qui a démarré en juin 2015 chez Sofrecom Tunisie en faveur de l'Unité de Production du Réseau (UPR) chez Orange France. Un grand projet qui vise à améliorer la conduite du déploiement des réseaux fixes. Le projet est subdivisé par ordre de priorité, en fonctionnalités qui sont en production, d'autres en développement, d'autre en attente pour des itérations ultérieures. D'un autre coté les cycles de changement sont répétitifs. Dans ces situations l'architecture monolithique a prouvé des limites en termes de flexibilité et dynamisme. Ce style de structure reste un frein face à la construction et la modification du Portail UPR.

Nous citons dans le prochain paragraphe les inconvénients de l'architecture monolithique et ses impacts sur l'évolution du projet portail pilotage UPR chez Sofrecom. Cette prochaine étape nous donnera l'élan suffisant pour définir nos axes d'amélioration et nous offrira une vue plus claire sur le portail existant.

2.3. **Diagnostic de l'existant : Architecture Monolithique**

Nous citons dans cette section les limites de la situation existante en mettant l'accent sur deux aspects, l'impact de l'architecture monolithique sur le portail de pilotage et l'impact de modèle de données sur les temps de réponse.

2.3.1. **Impact de l'architecture monolithique sur le portail UPR**

L'architecture monolithique est l'une des plus répandues dans les projets informatiques. Elle consiste à développer des applications entières en une seule pièce. Ces applications produisent de bons résultats pour les petits projets voire les moyens. Cependant après des mois de

développement elle limite l'intégration des ouvertures, l'agrégation de nouveaux besoins et l'innovation technologique. Le tableau 1 récapitule les principales limites de l'architecture monolithique et leurs impacts sur le portail UPR.

Tableau 1: Inconvénients de l'architecture monolithique

Limite	Impact sur le portail UPR
Limites technologiques	Le choix des technologies est décidé avant que le développement de l'application commence.
Mise à l'échelle couteuse	La mise à l'échelle d'une partie qui a besoin de plus de ressources requiert toute l'application.
Cout des tests élevé	Durée des tests automatiques longue et durée de build longue.
Modification couteuse	Une modification dans une petite partie de l'application requiert un rebuild et redéploiement entier
Tolérance aux pannes limitée	Un dysfonctionnement sur une partie du système impacte tout le système.
Agilité limitée	Agilité réduite de l'équipe et fréquence de livraison limitée à cause du couplage fort entre les composants.
	Effort important de montée en compétences pour un nouvel arrivant sur des composants fortement couplés.
	Nécessité de l'intervention de plusieurs personnes pour chaque modification

La figure 3 représente le portail pilotage déploiement réseau qui est une application web composée principalement de quatre couches techniques :

- Couche présentation : Se base sur le framework AngularJs 1 qui permet de développer des applications basées sur le paradigme MVC coté client.
- Couche métier : Représente l'ensemble métier de l'application, elle est développée avec le framework java/JEE Spring.
- Couche persistance : La couche DAO (Data Access Objects) a pour but de transformer les objets métiers en données sérialisées et inversement. Elle est réalisée en se basant sur le framework Hibernate/JPA.

- Couche données : Toutes les données issues de différentes sources sont sauvegardées dans une base de données relationnelle Postegresql.

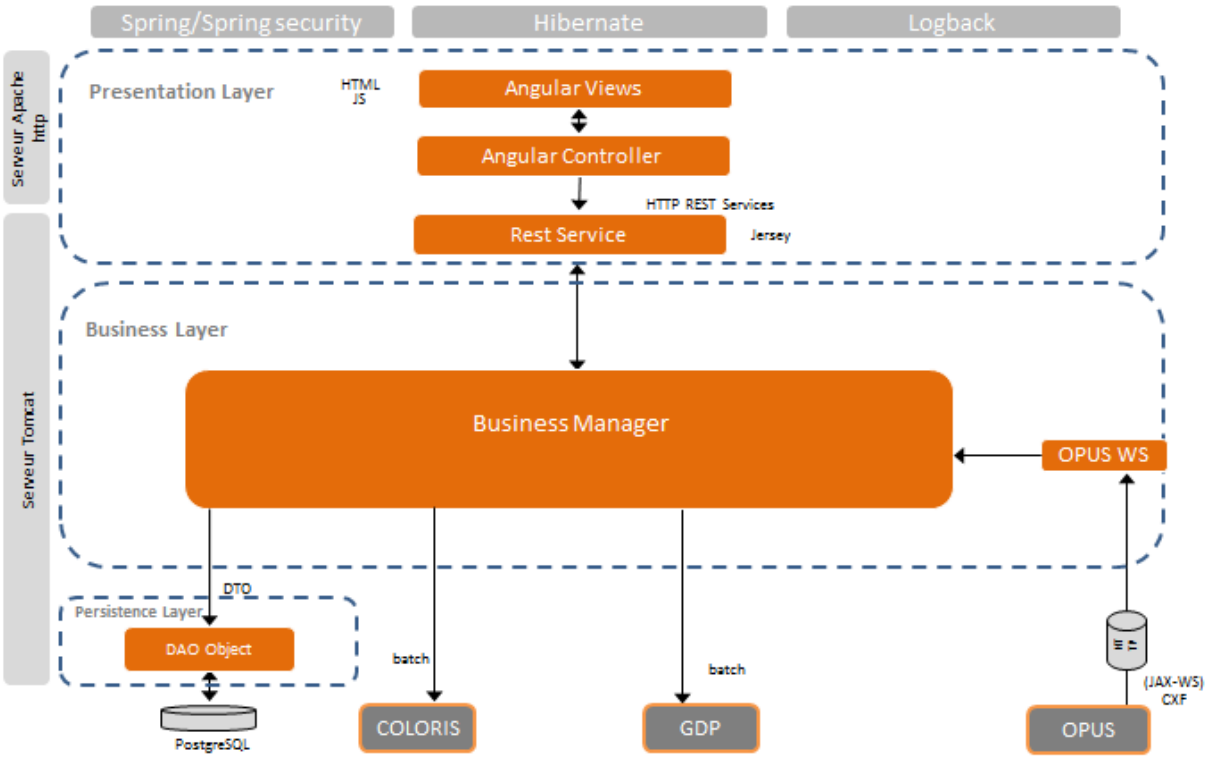


Figure 3 Architecture monolithique Portail Pilotage UPR

Malgré cette division en couche et la bonne modularité du portail de reporting, une modification ou ajout d'une fonctionnalité métier touchent toutes ses parties techniques. L'équipe grandit, les développeurs expérimentés quittent et de nouveaux rejoignent l'équipe. Néanmoins, l'effort de montée en compétences sur des composants fortement couplés devient important pour un nouvel arrivant. De même après chaque itération un redéploiement de la totalité de l'application est nécessaire. Vue ces modifications répétitives et ce cycle de développement bien mouvementé, l'architecture actuelle souffre de manque de flexibilité exigée par ces conditions. Pour remédier à ce problème, nous proposons une migration vers l'architecture Microservices. Les caractéristiques de ce style architectural sont décrites dans le chapitre II.

2.3.2. **Impact de modèle de données sur les temps de réponse**

Mis à part que l'architecture logicielle de l'application ne répond pas convenablement aux besoins d'évolution du portail UPR, la lenteur de réponse est un problème majeur de portail. Le volume des données est très important et un nombre multiple de jointures est obligatoire pour

offrir aux pilotes UPR une vue d'ensemble de l'activité de déploiement. C'est pourquoi les développeurs de l'application font recours à des requêtes en pure SQL complexes pour accélérer le temps de réponse. En effet, Le modèle de données existant est la principale source de lenteur mis à part le volume important de données. Dans le cas où le besoin est applicatif les bases relationnelles sont adéquates et répondent parfaitement au besoin. Par contre avec un besoin de reporting la normalisation produit une complexité en lecture. En conséquence nous proposons par la suite une dénormalisation de la base du reporting.

Nous réétudions donc les choix technologiques pour qu'ils soient plus adéquats avec le besoin de reporting et la nouvelle architecture proposée.

3. METHODOLOGIE ET APPROCHE DE DEVELOPPEMENT

Pour le choix de la méthodologie de développement il faut prendre en considération le périmètre et les intervenants du projet, les contraintes à savoir le temps alloué et le nombre de personnes mobilisées et enfin les spécificités de l'architecture. En analysant ces facteurs, nous avons découpé le projet en plusieurs itérations. A chaque itération une partie du projet sera traitée. A chaque fin d'itération un microservice autonome va être livré.

Pour se faire, nous avons choisi d'adopter la méthode Agile SCRUM pour le développement et l'approche DevOps pour la mise en production. En effet, comme le montre le schéma de la figure 4, l'agilité doit toucher toutes les étapes du projet et pas seulement les équipes de développement.

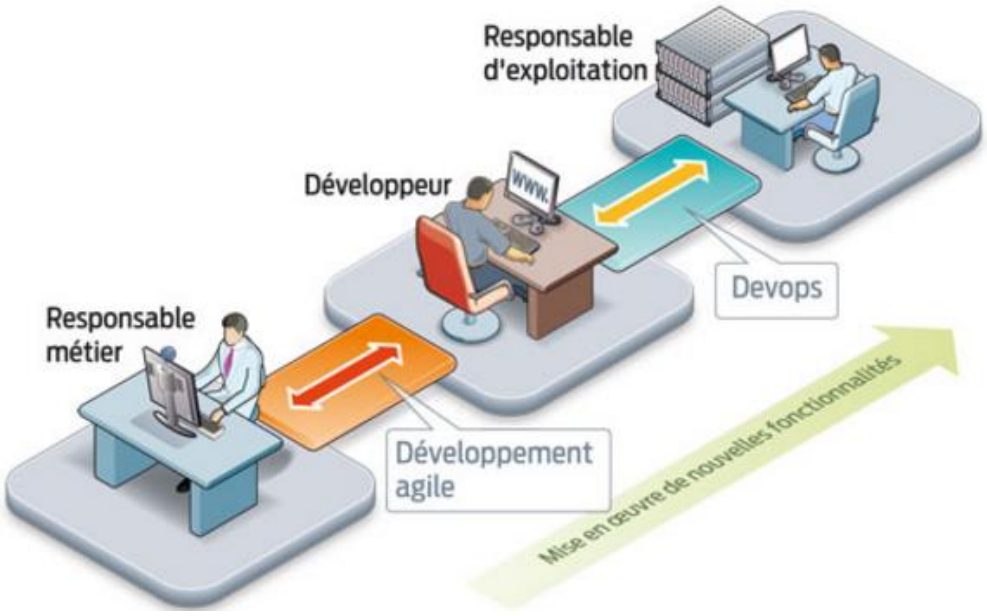


Figure 4 Agilité du bout en bout

3.1. **La méthodologie SCRUM**

La méthode SCRUM [B1] est une méthode agile créée en 2002, dont le nom est un terme emprunté au rugby qui signifie « la mêlée ». Elle s'appuie sur le découpage de projet en itérations nommées « Sprint ». Un Sprint peut avoir une durée qui varie généralement entre deux semaines et un mois. Avant chaque Sprint les taches sont estimées en temps et en complexité. Ces estimations permettent à la fois de planifier les livraisons mais aussi d'estimer le cout de ces taches auprès du client. La figure 5 présente la méthodologie SCRUM.

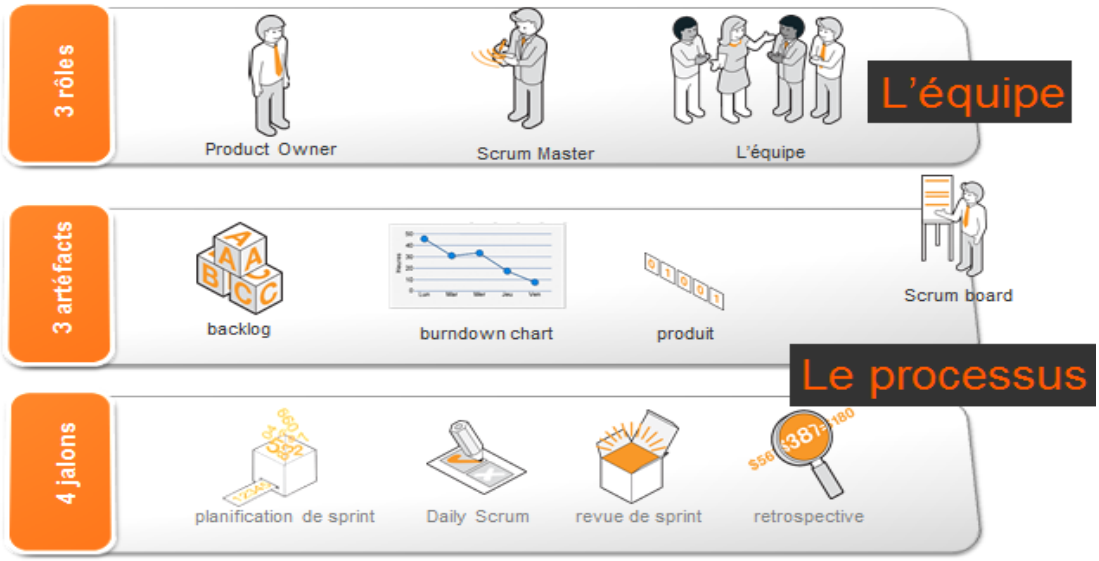


Figure 5 Pratique SCRUM: vue d'ensemble

La méthodologie SCRUM définit trois rôles [B2] pour un projet comme le montre la figure 5 :

- Le Responsable Produit (Product Owner) : c'est le représentant officiel du client au sein d'un projet SCRUM. Il définit et priorise des besoins du produit. Ce rôle est présenté dans notre projet par Monsieur Aymen HOUISSA chef de projet Portail UPR chez Sofrecom Tunisie.
- Le SCRUM Master : il s'agit de la personne qui facilite la mise en application de la méthode et veille au respect de ses objectifs. Il est responsable de la communication. Il ne joue pas le rôle d'un chef de projet, mais il se charge de lever les obstacles éventuels qui empêcheraient l'avancement de l'équipe et du projet pendant les différents Sprints. C'est l'acteur clé de la gestion des obstacles. Le SCRUM Master de notre projet est

Monsieur Sabri MTIBAA Manager technique et architecte logiciel chez Sofrecom Tunisie.

- L'équipe du projet (SCRUM Team Members) : ce sont les personnes chargées de la réalisation des Sprints et des produits livrables. Les membres doivent collaborer ensemble et être autonomes pour satisfaire les exigences du client et les engagements établis. L'équipe comporte 3 stagiaires ingénieurs. Malek GALLAS étudiante à l'École Nationale des Sciences de l'Informatique (ENSI), Yassine DERGUECHE étudiant à l'École Supérieure Privée d'Ingénieur (ESPRIT) et moi Salma DALDOUL étudiante à l'Institut National des Sciences Appliquées et de Technologie (INSAT) en tant que chef de projet.

Plusieurs réunions sont mises en œuvre pour permettre la communication entre les acteurs présentés [B2]. Nous citons parmi elles :

- Réunion pour la planification du Sprint (Sprint planning meeting) : le Sprint planning est une réunion critique et importante dans le cycle de vie du projet, et a lieu en début de Sprint. Elle permet à l'équipe projet et au responsable produit de définir l'ensemble des buts à atteindre au cours du Sprint en se basant sur le « Backlog du produit » priorisé et d'estimer la capacité de production nécessaire pour atteindre ce but.
- Mêlée quotidienne (Daily SCRUM) : Chaque jour, les membres de l'équipe projet passent en moyenne 15 minutes ensemble pour discuter. Chaque participant rapporte ce qu'il a fait la veille et les obstacles qu'il a rencontrés. Nous discutons aussi des taches à réaliser le jour de la réunion.
- Revue de Sprint (Sprint Review Meeting) : A la fin du Sprint, l'équipe projet présente les fonctionnalités réalisées au cours du Sprint et recueille les feedbacks du Product Owner et des utilisateurs finaux. C'est également le moment d'anticiper le périmètre des prochains Sprints et d'ajuster au besoin la planification de release (nombre de Sprints restants).
- Rétrospective de Sprint (Sprint Retrospective Meeting) : Une autre réunion qui tient à la fin de chaque Sprint. Le SCRUM Master anime cette réunion. L'équipe projet juge leur comportement et leur performance dans le processus de développement pour prendre des mesures afin de mieux gérer les prochains Sprints.

Les artefacts [B2] de communications font aussi partie du cycle de vie de développement d'un projet en suivant la méthodologie SCRUM comme le montre la figure 3. Parmi ces artefacts, nous citons :

- Backlog du produit (Product Backlog) : Il contient la liste des fonctionnalités désirées par le client sous forme d'User Stories. Il est visible à toutes les parties prenantes qui peuvent ajouter des éléments. Les priorités des éléments sont constamment mises à jour par le Product Owner. Le Backlog du produit de notre projet est présenté dans le chapitre spécification des besoins.
- Backlog du Sprint (Sprint Backlog) : Il comporte des exigences à réaliser, négociées entre le responsable produit et l'équipe projet lors du planning du Sprint. Il est visible à toute l'équipe dont elle se sert comme référence dans les mêlées quotidiennes.

Le tableau 2 présente les prérequis, les difficultés et les bénéfices de la méthodologie adoptée.

Tableau 2 SCRUM : pré-requis, difficultés et bénéfices

Prérequis	**Difficultés**	**Bénéfices**
Client mature sur la méthode, confiant, disponible et impliqué. Projets taille restreinte (9 personnes au plus). Forte polyvalence des développeurs. Mode contractuel en phase avec la méthode (difficulté de fortifier un périmètre variable). Locaux adéquats. Outiller pour pallier le manque de preuve de la méthode.	Concept d'équipe auto organisée assez difficile à mettre en œuvre dans la pratique. Faire adhérer l'équipe ; il n'est pas si facile de se sentir équipier et de partager de manière collective la paternité du travail (code notamment) réalisé. Builds quotidiens difficiles à mettre en œuvre sans outillage d'intégration continue et automatisation des tests de non-régression.	Eléments de garantie sur la focalisation et la productivité/qualité grâce à l'équipe auto organisée. Exigences stables en cours de réalisation d'un Sprint. Maintien du rythme et d'un résultat exécutable démontrable presque au quotidien. Egalité de communication et d'implication grâce aux SCRUM Meeting.

Avoir le bon SCRUM Master Avoir la bonne SCRUM Team	SCRUM Meeting quotidien pas totalement évident à faire en respectant le timing sans tomber dans un point projet formel.	

Les méthodes Agiles représentent un mouvement novateur qui vise à apporter plus de valeur aux clients et aux utilisateurs. Cependant, elles ne sont appliquées qu'aux développements et se trouvent freinées lors des déploiements. L'approche DevOps ambitionne de lever ce blocage.

3.2. **L'approche DevOps**

DevOps (un mot porte-manteau entre « Developpement » et « Operations ») [B3] est un ensemble de pratiques mettant en avant la communication, la collaboration et l'intégration entre les développeurs logiciels (Dev) et les exploitants (Ops) d'une même plateforme. En simplifiant les relations entre ces deux acteurs, il devient plus simple d'itérer rapidement les phases de déploiement tout en augmentant la fiabilité

L'approche DevOps se base sur quatre volets comme le montre la figure 6. À savoir la Culture, l'Automatisation, les Mesures et le Partage « CAMP ».

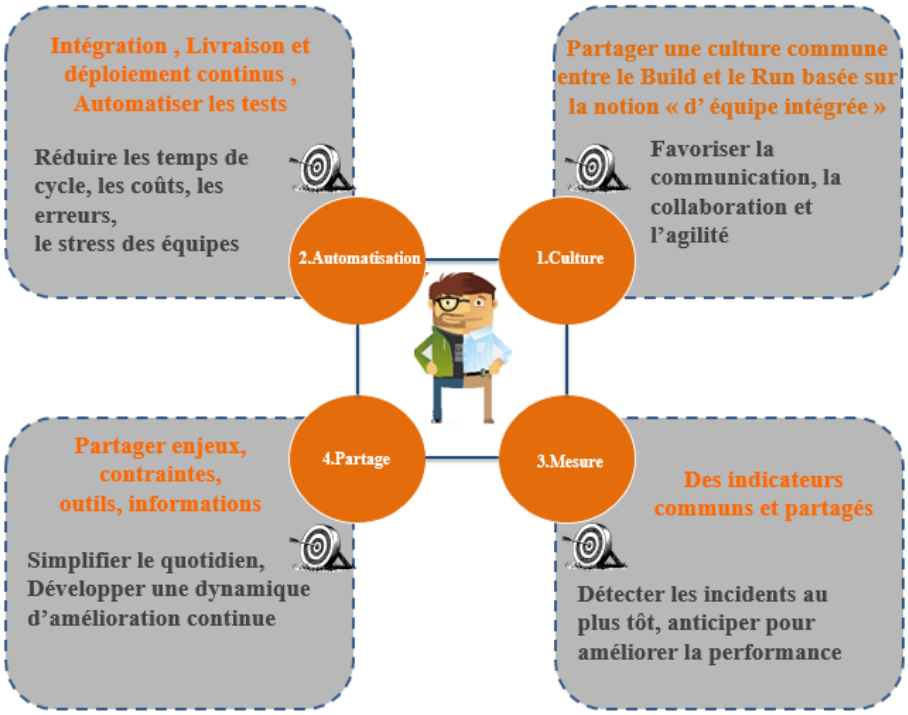


Figure 6 Définition DevOps

1- Le premier pilier culture se présente par notre intervention en tant que développeurs et opérateurs c'est-à-dire en tant que « développeurs DevOps ».
2- Le deuxième pilier automatisation schématisé dans la figure 7 se réduit en trois mots « intégration continue », «livraison continue » et « déploiement continu ».
   - L'intégration continue est un ensemble de pratiques visant à améliorer la qualité d'une application. Elle fait habituellement référence aux activités d'intégration, d'assemblage « build » et de test du code dans l'environnement de développement.
   - La livraison continue est une discipline de développement dans laquelle nous construisons un logiciel qui puisse être déployé dans l'environnement de production à tout moment.
   - Le déploiement continu [B4] veut dire que tout changement est automatiquement passé en production.

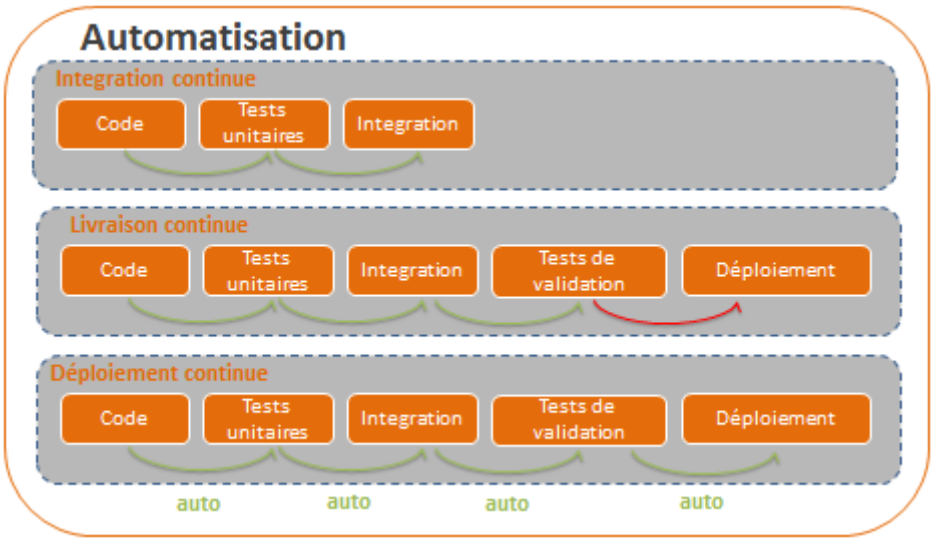


Figure 7 Automatisation DevOps

3- Pour répondre au troisième pilier mesure nous utilisons des outils du monitoring applicatif, des temps de réponse et de disponibilité.
4- Nous tirons profit des outils de partage afin de répondre au quatrième pilier. À savoir l'outil de versionning Git inscrit dans l'environnement Orange Forge pour le développement et la Platform as a Service (PaaS) Cloud Foundry pour la liaison entre le développement et l'exploitation.

CONCLUSION

Ce premier chapitre constitue une étape primordiale pour fixer les repères de notre projet. Après avoir présenté l'entreprise d'accueil et avoir reconnu ses attentes du projet, nous avons déterminé le cadre de notre travail ainsi que la méthodologie à emprunter lors de ce stage. Le chapitre suivant nous permettra alors de décrire les caractéristiques de l'architecture Microservices et les concepts en liaison

CHAPITRE II LES CONCEPTS DE L'ARCHITECTURE MICROSERVICES

Sommaire

Introduction 15
1. Les concepts liés à l'architecture microservices 15
1.1. Domain Driven Design 15
1.2. Développement à base de composants 16
1.3. Polyglot persistence 17
2. L'architecture Microservice 18
2.1. Définition de l'architecture microservices 18
2.2. Caractéristiques de l'architecture microservices 18
2.2.1. La division en composants via les services 18
2.2.2. L'organisation autours des capacités métiers 18
2.2.3. Un Produit, pas un Projet 19
2.2.4. Les extrémités Intelligentes et les Canaux Stupides 19
2.2.5. Une Gouvernance Décentralisée 19
2.2.6. Gestion de Données Décentralisée 20
2.2.7. Automatisation de l'Infrastructure 20
2.2.8. Design for Failure 20
2.2.9. Une conception évolutive 21
2.3. Bilan sur l'architecture microservices 21
2.3.1. Bénéfices clés des architectures microservices 21
2.3.1. Défis soulevés par les microservices 21
2.3.2. Inconvénients des architectures microservices 22
2.3.3. Motivations d'utilisation des microservices 22
3. Microservices en pratique: isolation des microservices dans des conteneurs 22
Conclusion 23

INTRODUCTION

Dans ce chapitre, nous nous intéressons aux concepts de base liés à notre travail. Nous projetons, dans un premier temps les concepts liées à l'architecture microservices. A savoir, l'approche Domain Driven Design (DDD), le développement à base de composants et la Polyglot persistence. Puis nous décrivons dans un deuxième temps, l'architecture Microservices. Enfin nous expliquons le déploiement des microservices avec le principe des conteneurs de déploiement.

1. LES CONCEPTS LIES A L'ARCHITECTURE MICROSERVICES

1.1. **Domain Driven Design**

Le développement logiciel est le plus souvent employé pour automatiser des processus qui existent déjà dans le monde réel, ou pour fournir des solutions à des problèmes métiers réels.

DDD est une approche de développement qui favorise la création des systèmes informatiques autour des compétences métiers pour combler l'écart entre la réalité de l'entreprise et le code. En pratique, DDD encapsule la logique métier complexe en des modèles métiers. Selon Eric Evans, [Domain Driven Design] [B5] un modèle métier n'est pas un diagramme particulier. C'est une abstraction rigoureuse des connaissances d'expert du domaine.

Le maintien d'un modèle dans un état pur est difficile quand il s'étend sur l'intégralité de l'entreprise, donc c'est préférable de tracer des limites à travers le pattern « Bounded Context » pour les définir.

D'après Martin Fowler le contexte borné [B6] est un pattern central dans la DDD. Il est au centre de la section de conception stratégique. Bounded context traite de grands modèles comme le montre la figure 8 en les divisant en différents contextes bornés.

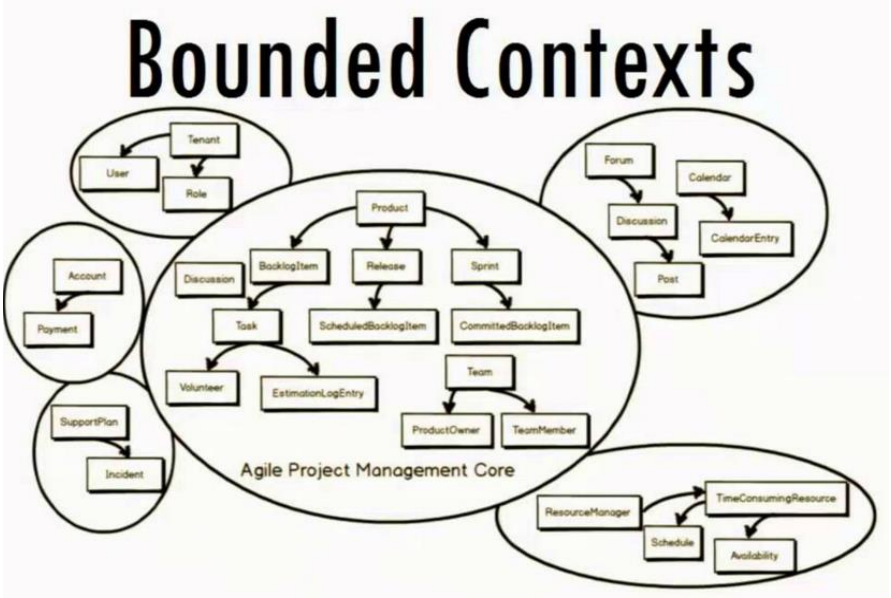


Figure 8 Division du domaine en Bounded Contexts

Il est nécessaire de définir des frontières et des relations entre les modèles multiples. Ces relations entre les contextes bornés sont généralement décrites par une carte de contexte. La Carte de Contexte est un document partagé entre ceux qui travaillent sur le projet. Elle met en évidence les différents contextes bornés et leurs liaisons. Cette carte peut être un diagramme comme illustré dans la figure 9, ou n'importe quel document écrit dans des niveaux de détail variables.

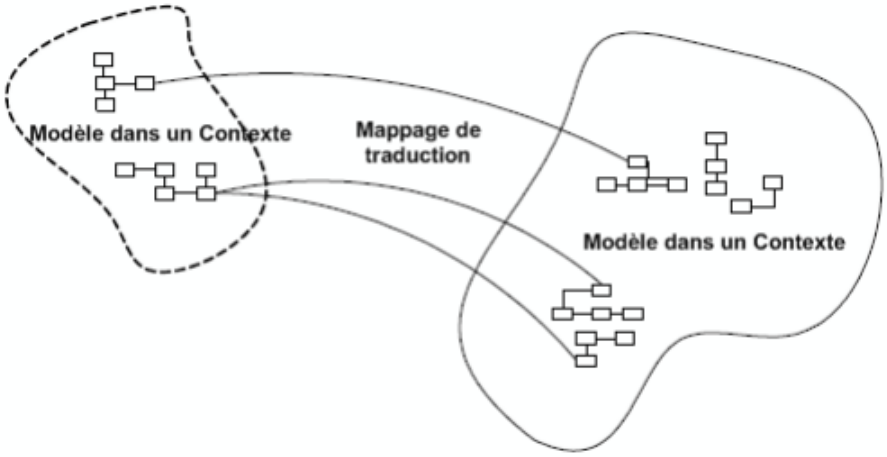


Figure 9 Carte de contexte

1.2. **Développement à base de composants**

Le développement à base de composant est une branche de l'ingénierie logicielle qui met l'accent sur la séparation des préoccupations à l'égard des vastes fonctionnalités disponibles à

travers un système de logiciel. C'est une approche basée sur la réutilisation et la redéfinition. Elle se base comme présenté par la figure 10 sur la mise en œuvre et la composition des unités indépendantes faiblement couplées dans les systèmes.

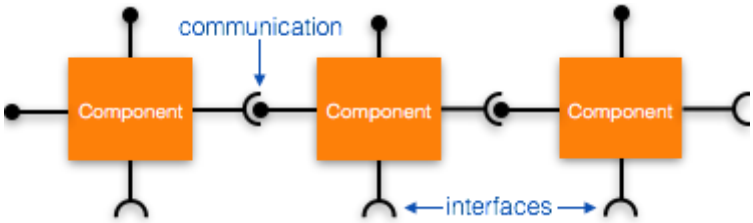


Figure 10 Architecture à base des composants

Cependant, un système est un ensemble de préoccupations fonctionnelles et extra-fonctionnelles (aspects). Les préoccupations fonctionnelles sont les fonctionnalités métiers que le système doit assurer, alors que les préoccupations extra-fonctionnelles sont des services dont le système a besoin pour effectuer ses fonctionnalités métiers. Comme exemple de préoccupations extra-fonctionnelles, on peut citer la sécurité, la synchronisation, la gestion de la persistance, etc. Dans les approches de développement de logiciels, il apparaît que les deux types de préoccupations sont enchevêtrés et le code relatif aux préoccupations extra-fonctionnelles est éparpillé dans le code fonctionnel.

Cette situation augmente la complexité, empêche la réutilisation et gêne l'évolution des systèmes. La séparation avancée des préoccupations permet de séparer les parties extra-fonctionnelles des parties fonctionnelles d'un système. L'objectif escompté étant d'offrir une meilleure réutilisation, faciliter la maintenance, réduire la complexité des systèmes et augmenter leur évolutivité.

1.3. **Polyglot persistence**

Le terme de persistance polyglotte (Polyglot Persistence) [W2] a été introduit par Scott Leberknight. Il tire son nom de la programmation polyglotte (Polyglot Programming), un principe qui favorise la coexistence de différents langages dans une même application, en tirant profit des forces de chacun d'entre eux.

Ainsi, la persistance polyglotte consiste à faire coexister un ensemble de solutions de bases de données différentes dans le cadre d'une même application. L'idée centrale de la persistence polyglotte est le fait d'être en mesure d'utiliser les meilleures technologies de stockage de données pour la tâche à accomplir.

2. L'ARCHITECTURE MICROSERVICE

2.1. **Définition de l'architecture microservices**

Le terme "Microservice" a émergé au cours des dernières années pour décrire une façon particulière de la conception de logiciels. C'est une approche pour développer une seule application comme suite de services, construits autour des compétences métiers, indépendamment déployés, une vieille idée renait pour permettre l'application modulaire. Ce qui revient à dire que l'architecture Microservice se base sur les concepts de Domain Driven Design, de Component-Based Software Engineering et l'Architectures orientées services (SOA), même si elle a une orientation différente. De plus ce style d'architecture rappelle en quelque sorte le système UNIX classique. Chaque Microservice reçoit des demandes, les traite, et génère une réponse en conséquence.

2.2. **Caractéristiques de l'architecture microservices**

D'après Martin FOWLER l'architecture Microservices possède neuf principales caractéristiques. [B7]

2.2.1. **La division en composants via les services**

Cette caractéristique est héritée du génie logiciel à base de composant présenté précédemment. Cependant, l'architecture Microservice est fondée sur l'utilisation des services en tant que composants. Les services sont indépendamment déployables. Un changement dans un service ne nécessite que le déploiement du service concerné. Certains changements dans un microservice vont modifier ces interfaces, mais l'objectif d'une bonne architecture de Microservice est de minimiser ces changements à travers des limites cohésives de service et des mécanismes d'évolution dans les contrats de service. Les services rendent plus facile d'éviter le couplage fort entre les composants en utilisant des mécanismes d'appel à distance explicites. Par contre ces appels distants sont coûteux.

2.2.2. **L'organisation autours des capacités métiers**

La décomposition classique d'une application est souvent orientée vers les couches techniques. La division des Microservices est différente. Le fractionnement est autour des capacités métier de l'entreprise. Chaque service est autonome vis à vis de la fonctionnalité qu'il réalise. Il possède son propre code, sa propre interface et gère ses propres données. La figure 11 illustre l'impact de la division en domaines métiers sur d'organisation de l'entreprise. Les équipes doivent être inter-fonctionnelles, avec tous les niveaux de compétences.

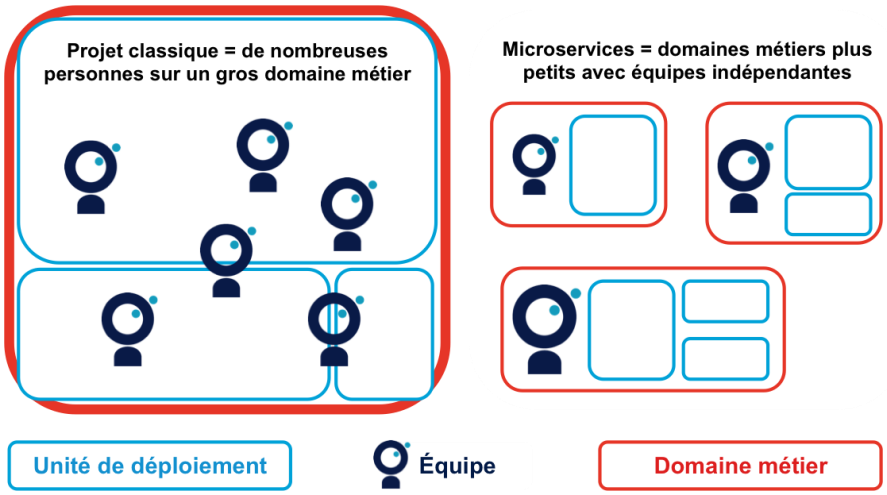


Figure 11 Décomposition selon le style d'architecture

2.2.3. **Un Produit, pas un Projet**

Ce concept se marie bien avec les méthodologies Agiles et l'approche DevOps présentées dans le premier chapitre. Le but est de livrer rapidement un morceau de logiciel qui est alors considéré comme terminé. Dans la vision Microservices, une équipe doit posséder un produit, durant tout son cycle de vie. L'équipe de développement est entièrement responsable du logiciel produit et reste au courant du comportement de son produit en production. La mentalité de produit met en avant la communication entre les développeurs, les utilisateurs et les exploitants.

2.2.4. **Les extrémités Intelligentes et les Canaux Stupides**

Plusieurs approches mettent beaucoup d'intelligence dans les canaux de communication entre les services. Prenons le cas des Enterprise Service Bus (ESB), qui comprennent souvent des équipements sophistiqués pour le routage des messages, la chorégraphie, la transformation, et l'application de règles métier. Les applications utilisant les Microservices favorisent l'utilisation de communications sans intelligence, qui ne font que transmettre les messages, alors que le service lui-même fait le reste. La communauté Microservice privilégie l'intercommunication via des protocoles ouverts. Alors que beaucoup de Microservices interagissent les uns avec les autres via des API REST ou à travers des systèmes de file d'attente.

2.2.5. **Une Gouvernance Décentralisée**

Pour bien comprendre une gouvernance décentralisée, voyons les défauts d'une gouvernance centralisée. Une des premières conséquences de la gouvernance centralisée est la

tendance à standardiser l'ensemble des développements vers des plates-formes technologiques (dans le meilleur des cas). L'expérience montre que cette approche est très limitative. Il est difficile de trouver une seule technologie qui résout tous les problèmes. Donc, il est préférable d'utiliser le bon outil au bon moment. Dans les architectures Microservices, chaque service pourra utiliser la technologie, le langage et la plateforme les plus adéquats pour ses besoins.

2.2.6. **Gestion de Données Décentralisée**

L'architecture Microservice favorise l'approche Polyglot Persistence qui admet l'utilisation de plusieurs bases de données dans le cadre d'un plus grand écosystème datastore. Donc, chaque Microservice gère sa propre base de données. Les microservices s'interrogent les uns les autres pour récupérer les données. Chaque micro-service est responsable de l'agencement de ces données. Cela amène 3 avantages majeurs :

- Il n'y pas de risque d'incohérence des données liée à l'utilisation des tables par d'autres services.
- La représentation des données dans la base est potentiellement différente de celle utilisée dans les messages.
- Les technologies des bases de données peuvent être différentes.

2.2.7. **Automatisation de l'Infrastructure**

On vient de voir que chaque service est façonné avec des outils adaptés et donc diversifiés. Cela implique dans un premier temps que la qualité des services doit être mesurée en permanence : tests automatisés et analyse du code. Par ailleurs l'aspect "système complexe" de cette architecture, implique que les tests d'intégration et les tests globaux, sont aussi importants que les tests des services isolés. Ils ont donc tout autant vocation à être automatisés. Cela implique ensuite, pour la viabilité du système en production, que les déploiements soient au maximum automatisés (déploiement continu). Les procédures doivent être reproductibles, répétables et doivent contenir le moins de sources d'erreurs possible car elles seront potentiellement exécutées un grand nombre de fois.

2.2.8. **Design for Failure**

« Si une catastrophe peut arriver, elle finira obligatoirement par arriver »

Les microservices sont conçus pour être tolérants aux pannes. Un service doit échouer sans affecter d'autres services dans la même application. Et les autres services adaptent donc leurs fonctionnements en fonction de l'état du système dans lequel ils évoluent.

Dans une application en microservices, il faut détecter les failles très rapidement par un monitoring en temps réel de l'ensemble de l'application et restaurer le service automatiquement, si possible.

2.2.9. **Une conception évolutive**

La propriété clé d'un microservice est la notion d'indépendance et d'évolutivité, ce qui implique que nous pouvons réécrire un microservice sans affecter les autres, si on reste iso fonctionnel. En effet, il faut définir la modularité suivant le patron du changement. C'est-à-dire regrouper dans le même module les éléments qui changent en même temps. En général, l'évolution d'une application implique l'ajout de nouvelles fonctionnalités, soit un nouveau périmètre pour les fonctionnalités existantes. Ceci se traduira par la création de nouveaux microservices et ou par la modification de microservices existants. En se basant sur une conception solide, la modification du code ou des données d'un service provoque uniquement la mise à jour et le redéploiement d'un service car celui-ci s'exécute dans un processus isolé.

2.3. **Bilan sur l'architecture microservices**

2.3.1. **Bénéfices clés des architectures microservices**

- La réduction du délai de mise en marché ou « time to market »
- L'agilité technologique
- L'extensibilité
- La factorisation et la réutilisation des microservices
- Evolutivité
- Résilience

2.3.1. **Défis soulevés par les microservices**

Bien que l'architecture Microservice soit la plus agile approche pour concevoir des applications, et elle présente des solutions aux limites des applications monolithiques, elle nécessite des prérequis et confronte plusieurs défis et limites. Parmi les prérequis nous citons :

- Nécessité d'une équipe produit
- Automatisation des tests obligatoires
- Besoin d'automatisation du déploiement

En conséquence pour adopter l'architecture Microservice il faut avoir un bon niveau de maturité dans l'entreprise et un bon niveau d'expertise Devops.

2.3.2. **Inconvénients des architectures microservices**

Malgré leurs avantages, les architectures de microservices font l'objet de critiques et présentent des inconvénients. Les inconvénients les plus récurrents sont :

- La complexité
- Une cohérence à terme
- La baisse de productivité sur les SI simples
- Un cout initial plus élevé
- Un déploiement par microservice
- Monitoring essentiel et complexe (problème de correlation id)

2.3.3. **Motivations d'utilisation des microservices**

Plusieurs motivations poussent à utiliser les architectures microservices :

- La frustration de ne pas obtenir le résultat souhaité avec les architectures monolithiques dans les grands projets nous encourage à affronter les défis des microservices.
- L'émergence de nouveaux outils d'automatisation de test, de monitoring et de déploiement nous permettent de voir les microservices sous un nouveau jour et suppriment une partie des problèmes que leur développement et leur déploiement avaient créés.
- L'utilisation des microservices par des grandes entreprises comme Amazon, Netflix, eBay et d'autres, donne assez de confiance pour que ce style d'architecture soit prêt à être évalué et utilisé par les développeurs d'applications professionnelles.

3. MICROSERVICES EN PRATIQUE: ISOLATION DES MICROSERVICES DANS DES CONTENEURS

Avant les années 60, pour transporter des produits de formes et de tailles variées, ce n'était pas simple. Le changement de mode de transport (passer d'un camion ou d'un train à bateau) nécessitait beaucoup de manutention. Et puis en 1956 un transporteur routier américain, Malcom MAC LEAN, a eu l'idée de transporter par bateau des remorques de camion, puis de dissocier la caisse du châssis de la remorque, ce qui a donné naissance aux conteneurs. La standardisation des conteneurs a révolutionné le monde du transport [W3].

La solution à base de conteneurs de déploiement est une analogie avec le domaine du transport martine. Les développeurs préparent des conteneurs qui sont standard pour n'importe

quel environnement. Le but est d’avoir dès le développement un environnement proche de celui de la production.

Les conteneurs de déploiement sont idéals pour la mise en œuvre des applications en microservices, du fait de la rapidité de déploiement qu'ils permettent, de l'isolation des systèmes et de la gestion de leur cycle de vie. [W4]

CONCLUSION

L'objectif de ce chapitre consistait à éclaircir les concepts de base aidant à comprendre et mener à bien notre projet. Le prochain chapitre s'étalera sur les détails des spécifications fixées dans le cadre de refonte du Portail Pilotage UPR et l’approche à suivre pour la migration.

CHAPITRE III

SPECIFICATION DES BESOINS ET URBANISATION FONCTIONNELLE EN MICROSERVICE

Sommaire

Introduction 24
1. Recueil des besoins Fonctionnels globaux 24
   1.1. Les besoins fonctionnels 24
   1.2. Identification des acteurs 27
   1.3. Backlog produit 27
   1.4. Architecture fonctionnelle générale 29
   1.5. Modélisation des cas d'utilisation 30
2. Urbanisation fonctionnelle en microservices 31
3. Contraintes non fonctionnelles 33
Conclusion 34

INTRODUCTION

Cette phase constitue l'une des plus importantes étapes dans le cycle de développement du projet car elle aide le concepteur à planifier les tâches. En effet, c'est au cours de celle-ci que les besoins sont identifiés et précisés.

Dans une architecture Microservices les services correspondent à une fonctionnalité issue d'un besoin métier. Pour cela nous allons spécifier les besoins généraux de portail. Ceci nous offrira plus d'élan afin de fournir une solution d'architecture en Microservices solide et flexible.

Nous commençons par la présentation de l'ensemble des fonctionnalités que satisfait le portail. Puis, nous identifions les acteurs en corrélation. Ensuite, nous déduisons une segmentation des fonctionnalités en sous domaines. Enfin nous présentons les différentes contraintes que respecte la solution.

1. RECUEIL DES BESOINS FONCTIONNELS GLOBAUX

1.1. **Les besoins fonctionnels**

Les UPR (Unité de Production du Réseau Orange France) sont au nombre de 6 (UPR SUD EST, UPR NORD EST, UPR SUD OUEST, UPR OUEST, UPR ILE DE France, DOM). Elles pilotent le développement, la production et optimisent la performance des réseaux fixes et mobiles d'Orange. Le pilotage de déploiement du réseau structurant s'appuie sur un outil de workflow pour la production (OPUS) et nécessite la consultation de nombreux outils complémentaires pour mener à bien l'activité de pilotage. Le projet Portail Pilotage UPR a démarré en juin 2015 en faveur du département Ingénierie Déploiement Réseau (IDR) chez l'Unité de Production Réseau (UPR). Il tient à offrir au pilote en UPR ou au manager une visualisation globale et synthétique de son portefeuille, ou celui de son équipe.

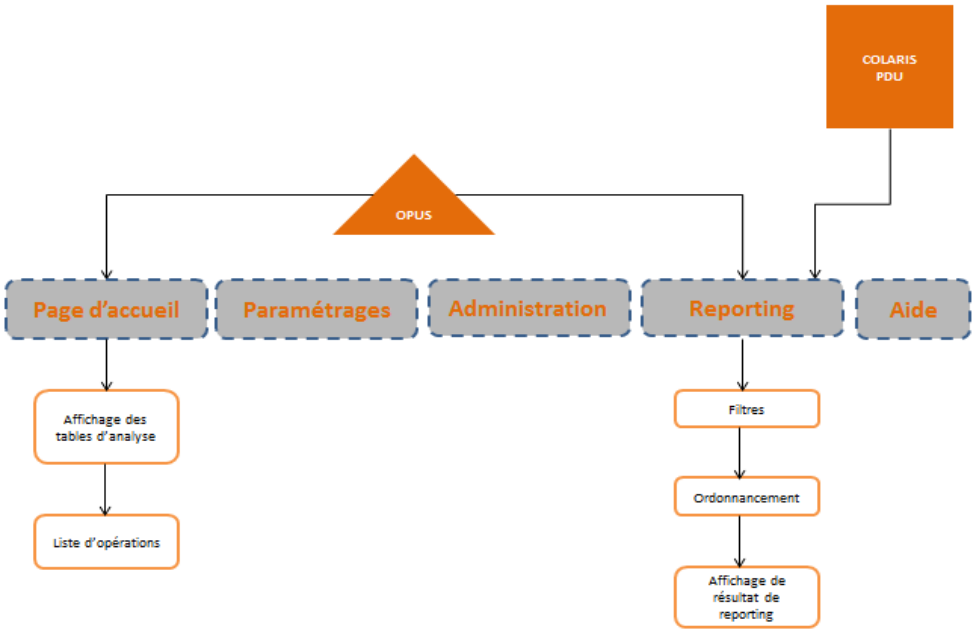


Figure 12 Démarche d'urbanisation du Portail UPR

La figure 12 présente la démarche d'urbanisation du Système d'Information de la solution. Cette démarche montre que notre portail communique avec des applications externes telles que l'Outil de Pilotage Universel (OPUS). OPUS est une application de workflow chez Orange qui gère les processus métier de déploiement réseau. Chaque opération de déploiement est représentée par un graphe OPUS. Cette communication a pour but d'alimenter les données essentielles pour le pilotage. Dans notre étude nous nous intéressons qu'à l'application externe OPUS.

Le Portail de Pilotage Déploiement Réseau (PPDR) a pour but d'automatiser l'agrégation des données, de respecter les contraintes de délais, de disposer d'une vision métier enrichie pour une prise de décision réactive et d'unifier et harmoniser le reporting. La spécification des besoins fonctionnels permet de cadrer le travail et de définir les différentes tâches à réaliser. Pour optimiser le pilotage de la mise en œuvre des structures réseau PPDR doit satisfaire les fonctionnalités suivantes :

- **Gérer le portefeuille**

Définir le périmètre d'un pilote
Définir les unités de production (administrateur local et administrateur national)

- **Consulter la synthèse d'une famille d'opérations de déploiement des réseaux**

Afficher les jalons d'une famille (structure du tableau)

Ventiler les opérations d'une famille

Accéder à la liste des opérations

Générer les tableaux de synthèse pour le périmètre

- **Consulter la liste des opérations**

Afficher la liste des opérations de la cellule du tableau de synthèse sélectionnée

Afficher les mouvements

Afficher la visualisation des jalons

Accéder au détail d'une opération (dans une nouvelle fenêtre)

- **Accéder au reporting**

Filtrer et lister des données de la recherche

Afficher les résultats de la recherche (données en cours et ou historisées)

Enregistrer une recherche

Partager une recherche

Sélectionner une recherche préenregistrée

Supprimer une recherche préenregistrée

Modifier une recherche préenregistrée

- **Accéder au détail d'une opération (formulaire)**

Afficher les données OPUS

Gérer les mouvements

Afficher les données associées des autres SI (GDP / COLORIS)

Afficher les opérations OPUS associées

- **Saisir les données d'une opération (commentaire)**

Saisir un commentaire

Modifier/Supprimer un commentaire

- **Gérer les critères métiers (admin national)**

Uploader un nouveau fichier de critères

Modifier les jalons (ajout/modif/suppression d'activités)

Modifier les seuils et les dates

- **Exporter les données dans un fichier Excel**

Export Reporting (Excel et csv)
Export critères métier (Excel et csv)

1.2. **Identification des acteurs**

Les métiers des Départements Ingénierie Déploiement Réseau (IDR) concernés par le portail de pilotage sont :

- les pilotes de déploiement réseau du fixe
- les chefs de projet déploiement
- les responsables des cellules de coordination

Dans notre solution, l'habilitation des utilisateurs est déclarée selon leurs profils métier. Nous pouvons distinguer cinq types d'utilisateurs :

- **acteur IDR (Pilote)**

C'est le pilote des déploiements, il doit pouvoir consulter la ventilation de ses opérations selon les seuils et jalons définis, consulter le détail de chaque opération, traiter ses opérations (ajout de commentaires internes au portail, confirmation des activités dans OPUS) et connaitre les mouvements des opérations de son portefeuille.

- **manager IDR (responsable de domaine)**

Il doit pouvoir avoir une vision des portefeuilles de son équipe.

- **Externe IDR (Consultant)**

C'est un acteur externe au service IDR qui doit pouvoir consulter le reporting des opérations.

- **administrateur national**

Défini les critères métier du pilotage (seuils, jalons …)

- **administrateur local**

Il définit localement (par UPR) des seuils et des formats pour répondre aux contraintes de pilotage d'une UPR.

1.3. **Backlog produit**

Un "Backlog Produit" est une liste de tâches exprimées sous forme de besoins (User Stories). Notre backlog produit est exposé dans le tableau 3:

Tableau 3 Backlog de produit

ID	User Story	Effort
1	En tant que utilisateur, je veux pouvoir me connecter au portail avec les bons droits.	8
2	En tant que pilote, je veux pouvoir définir le périmètre des opérations OPUS que je peux suivre.	13
3	En tant que manager ou administrateur, je veux pouvoir définir le périmètre de portefeuille d'un autre utilisateur.	13
4	En tant que pilote ou manager, je veux pouvoir consulter la synthèse des opérations d'une famille.	34
5	En tant que pilote ou manager, je veux pouvoir visualiser la liste des opérations correspondant à la famille, à l'article, à l'activité et au seuil qu'il a sélectionné dans le tableau de synthèse	34
6	En tant que utilisateur, je veux pouvoir sélectionner des filtres pour visualiser la liste des opérations correspondante.	34
7	En tant que pilote, manager ou consultant, je veux pouvoir voir le détail des données d'une opération ainsi que ses données complémentaires et ses associations.	21
8	En tant que pilote ou manager, je veux pouvoir ajouter/modifier/supprimer un commentaire pour compléter les données internes au portail pour une opération.	21
9	En tant que administrateur national, je veux pouvoir définir les jalons majeurs par famille, pour gérer les critères métiers.	13
10	En tant que pilote, je veux pouvoir confirmer ou mettre à jour une activité depuis le portail.	13
11	En tant que utilisateur, je veux pouvoir exporter les tableaux de synthèse et de reporting au format Excel.	21
12	En tant que administrateur national, je veux pouvoir visualiser les opérations orphelines, opération n'ayant pas de données qui permettent la ventilation dans un portefeuille.	13

13	En tant que administrateur national, je veux pouvoir créer des données complémentaires aux opérations	21
14	En tant que administrateur national, je veux pouvoir exporter les données complémentaires aux opérations au format Excel.	21
15	En tant que manager, je veux pouvoir définir la composition de mon équipe pour que je puisse avoir une vision globale des portefeuilles des membres de mon équipe.	13
16	En tant que administrateur local, je veux pouvoir définir des seuils différents de ceux établis par le métier (administrateur national)	13
17	En tant que pilote, manager ou consultant, je veux pouvoir changer l'ordre des colonnes et des autres options de personnalisation du tableau.	13

1.4. **Architecture fonctionnelle générale**

Le schéma fonctionnel présenté par la figure 13 résume les besoins fonctionnels du Portail Pilotage UPR.

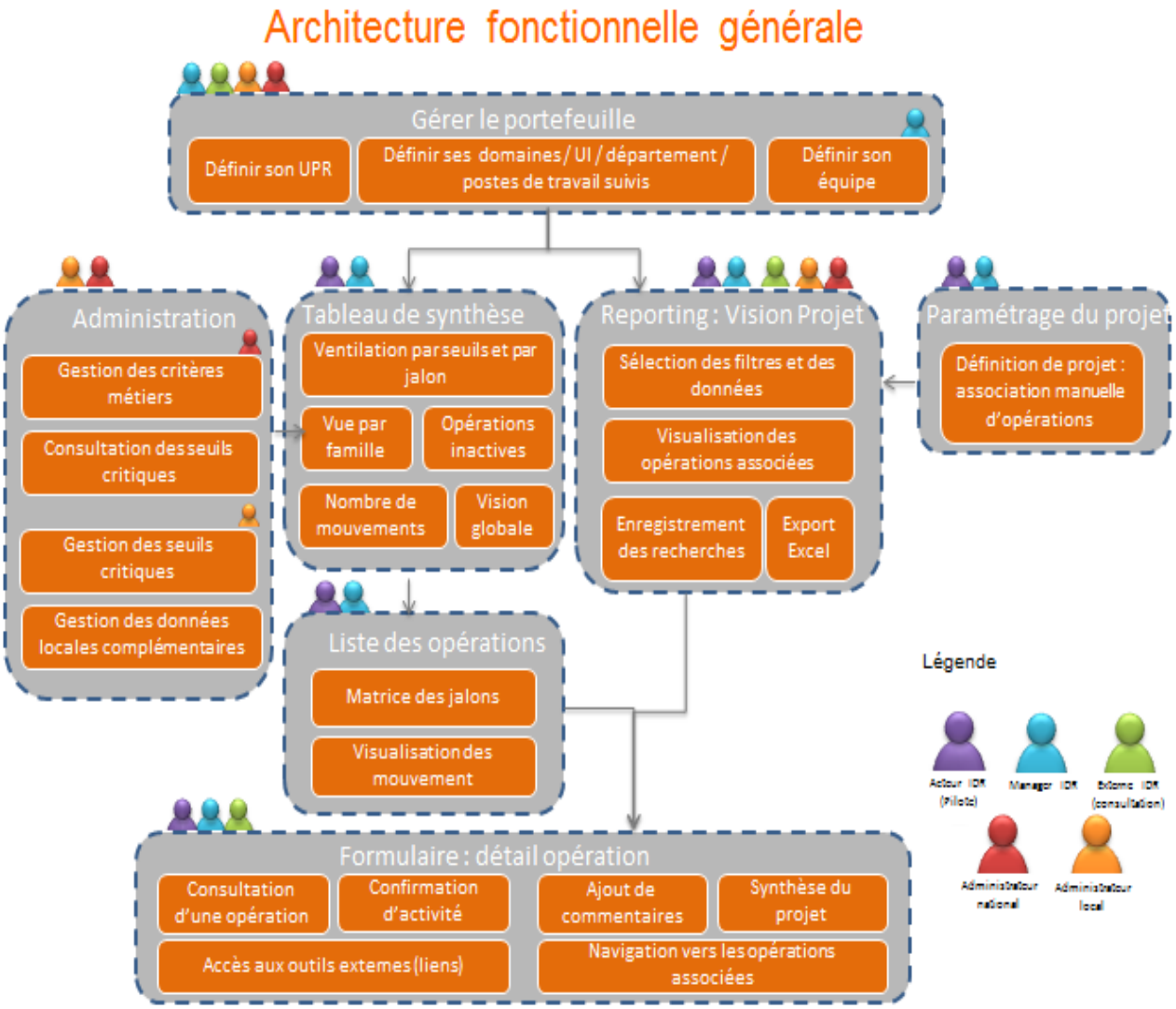


Figure 13 Architecture fonctionnelle générale du Portail UPR

1.5. **Modélisation des cas d'utilisation**

Le diagramme de cas d'utilisation de la figure 14 schématise les fonctionnalités du Portail UPR. Dans ce diagramme, nous mettons tous les cas d'utilisation. Nous détaillons dans les chapitres suivants les cas d'utilisation réalisés.

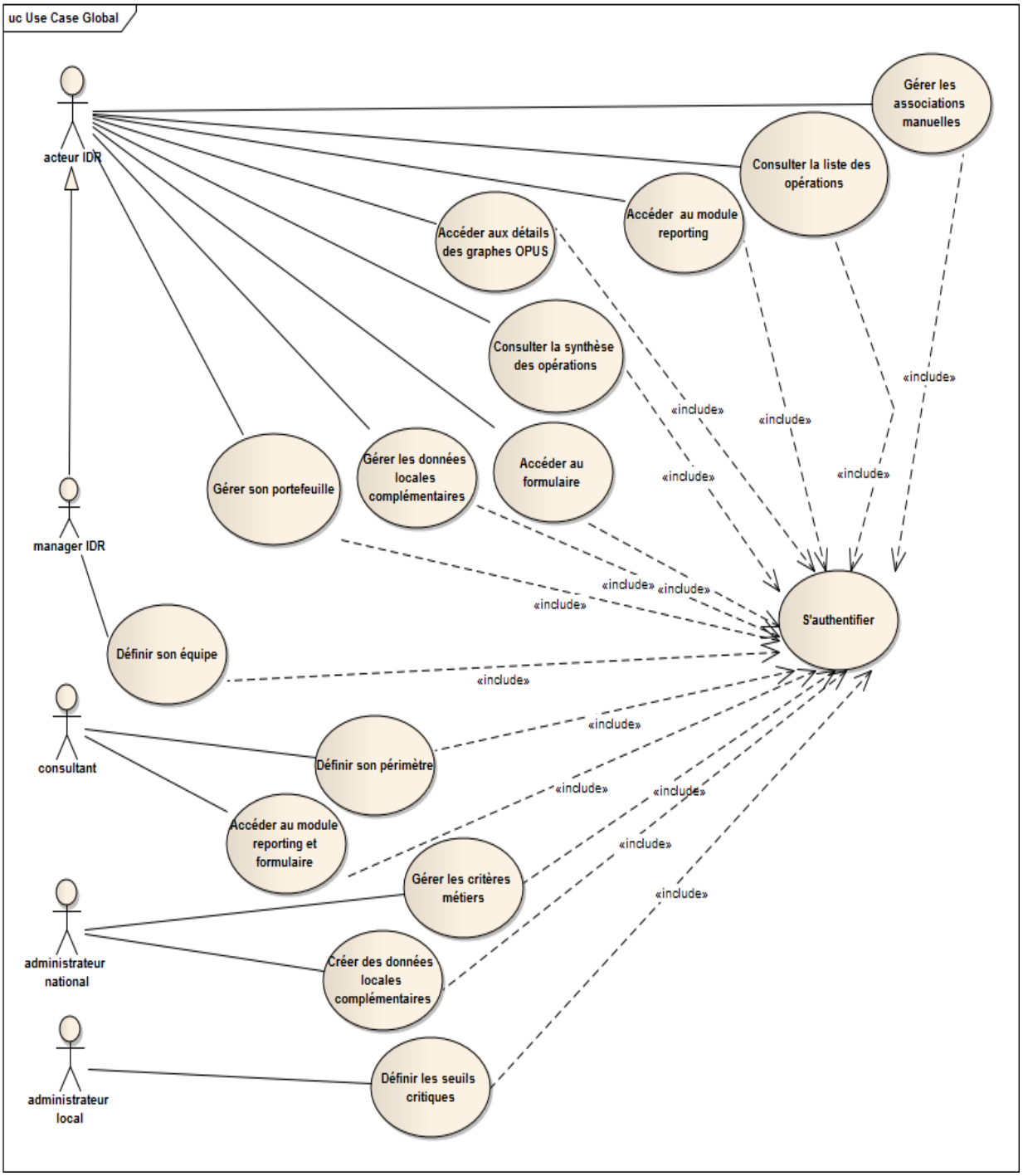


Figure 14 Diagramme de cas d'utilisation global

2. URBANISATION FONCTIONNELLE EN MICROSERVICES

La figure 15 présente le modèle métier global de Portail UPR qui met l'accent sur les différents composants métiers ainsi que les liens qui les attachent. Pour mieux comprendre ce modèle on peut se référer au glossaire métier.

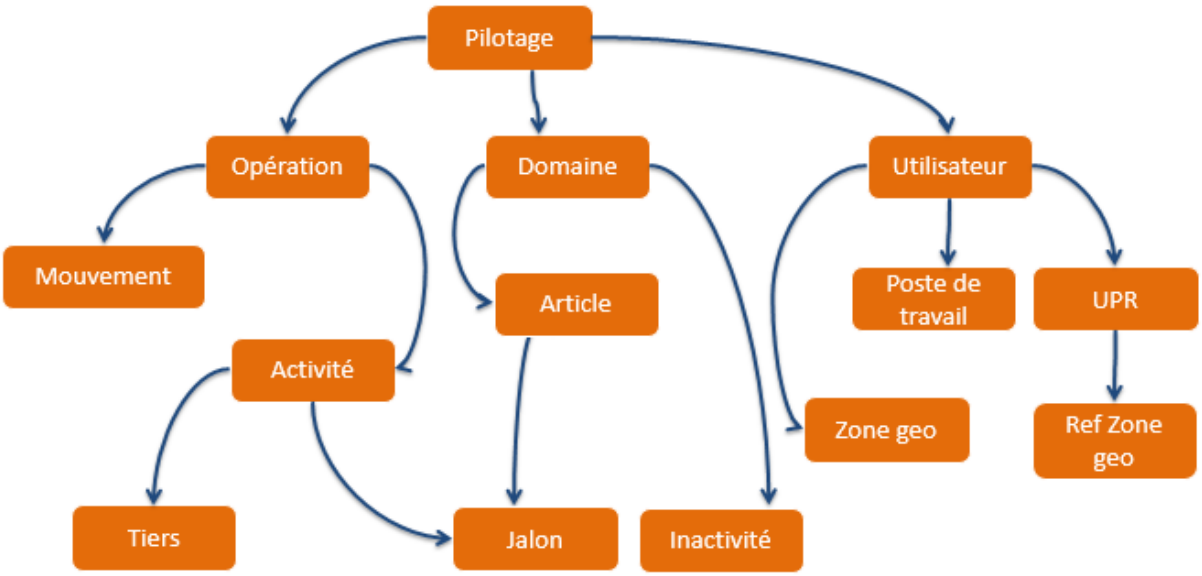


Figure 15 Modèle métier du portail UPR

En suivant le pattern Bounded Contexts du Domain Driven Design, expliqué dans le chapitre II, nous divisons notre modèle métier en trois sous modèles. La carte de contexte de la figure 16 présente les limites des contextes et deux exemples de dépendances entre eux.

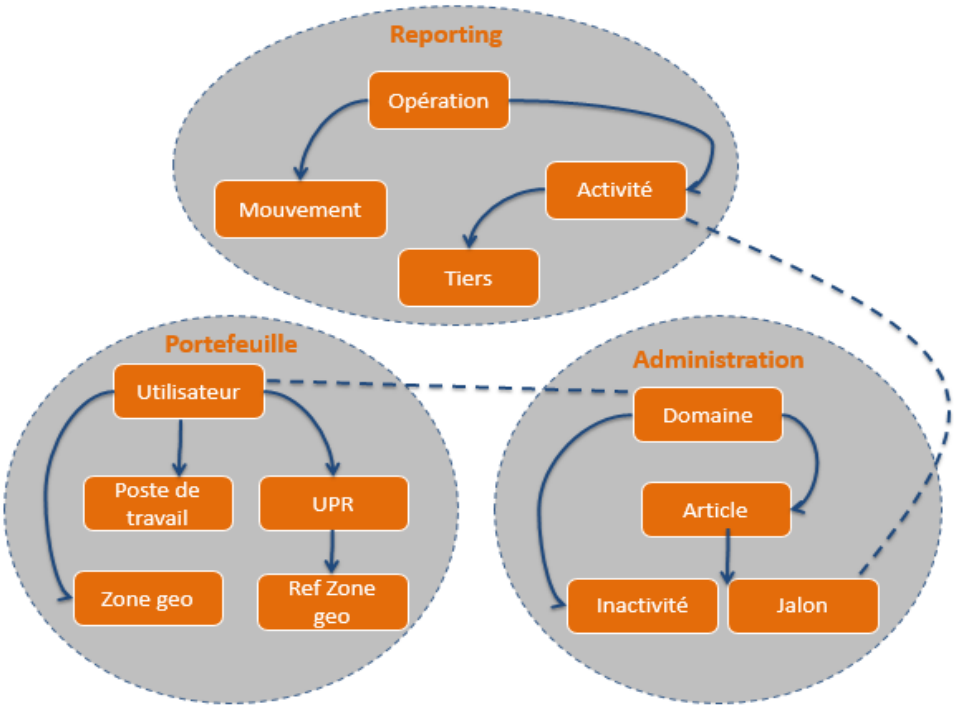


Figure 16 Carte de contexte du pilotage UPR

Chaque contexte présente un microservice autonome et le couplage entre les microservices est faible. Le tableau 4 découpe les Users stories du backlog de produit de tableau 3 en 3 parties selon les sous domaines.

Tableau 4 Partitionnement des user stories par sprint

Sprint1 : Microservice administration	**Sprint2 : Microservice portefeuille**	**Sprint3 : Microservice reporting**
8, 9,10, 13, 14, 16	1, 2, 3, 15	4, 5, 6, 7, 11, 12, 17

Il est à noter qu'un Sprint de lancement est défini au début du projet pour spécifier les choix technologiques adéquats aux spécificités de l'architecture.

3. CONTRAINTES NON FONCTIONNELLES

Dans cette partie, nous présentons les besoins non fonctionnels requis de l'application :

- **Volumétrie**

Le volume de production annuelle de l'ordre de 54.000 graphes OPUS.

- **Temps de Réponse**

Un temps de réponse qui est entre 5 à 10 secondes.

- **Nombre d'utilisateurs**

Les utilisateurs seront, dans un premier temps :
Au total un potentiel de 650 personnes max au niveau national. À cela il pourra avoir des accès en consultation hors UPR.

- **Maintenabilité**

L'architecture doit permettre l'évolution et assurer l'extensibilité de l'application.

- **Monitoring et traçabilité**

Garder et superviser l'historique de toutes les interventions des utilisateurs et le comportement de l'application.

- **Tolérance aux pannes**

Le portail UPR doit prévoir des mécanismes de reprises en cas d'éventuels pannes d'un ou plusieurs services.

- **Mise à l'échelle**

Le portail UPR doit permettre une mise à l'échelle ciblée. C'est-à-dire nous n'avons pas besoin de faire une mise à l'échelle de toute l'application mais uniquement des composants qui sont concernés.

CONCLUSION

La spécification et l'analyse des besoins procurent une vision plus claire du sujet abordé et une compréhension plus profonde des tâches à réaliser. Dans ce chapitre, nous avons établi une étude des besoins et leur analyse à travers des diagrammes des cas d'utilisation. Nous avons défini les sprints qui constitueront les livrables de notre projet. Dans la suite chaque sprint représentant un microservice sera détaillé et validé dans un chapitre. Le chapitre suivant va décrire le cycle de vie de microservice administration.

CHAPITRE VI

SPRINT 1 : MICROSERVICE ADMINISTRATION

Sommaire

Introduction 35
1. Orientation technologique 35
2. Étude fonctionnelle de l'administration 36
3. Conception du microservice administration 40
   3.1. Architecture logicielle de microservice administration 40
   3.2. Conception générale de microservice administration : Diagrammes de package 40
   3.3. Modèle relationnel de microservice administration 41
   3.4. Structure globale de microservice administration 42
4. Architecture technique de microservice administration 44
5. Réalisation de microservice administration 44
6. Phase de test de microservice administration 45
Conclusion 46

INTRODUCTION

Après avoir spécifié et analysé les besoins globaux des utilisateurs finaux, nous entamons l'analyse détaillée, la conception, la réalisation et la phase de test de microservice administration. Au début de ce chapitre nous présentons brièvement le Sprint de lancement qui porte sur les choix technologiques par une phase de « Proof Of Concept » (POC).

1. ORIENTATION TECHNOLOGIQUE

Dans le cadre d'une architecture microservices les choix technologiques ne sont pas décidés dès le début de projet. Chaque microservice peut faire appel à des technologies adéquates à son besoin. Dans le cadre de projet de refonte, le premier choix de framework s'est porté sur Spring Boot. Suite à une étude que nous avons effectuée, nous avons choisi Spring Cloud qui est plus spécifique à l'architecture microservices. Ceci est expliqué par le tableau 5 qui montre les services offerts par Spring Cloud qui ne sont pas disponible dans Spring Boot.

Tableau 5 Apport de Spring cloud pour l'architecture microservice

Besoin de l'architecture microservice		**Bénéfice avec Spring Cloud**
Développement rapide		La configuration d'un projet spring Cloud se base sur une configuration et un démarrage avec Spring Boot qui a déjà montré son efficacité.
Intégration	Service de découverte	Spring Cloud facilite l'intégration de service de découverte Eureka de Netflix ou Zookeeper d'apache
	Communication synchrone	Spring Cloud offre trois modes de communication synchrone avec l'API Rest.
	Communication asynchrone	Spring Cloud peut fournir une communication asynchrone pour l'auto configuration avec Spring Cloud Bus qui fonctionne par défautt avec RabbitMQ. Spring Cloud Stream modèle déclaratif simple d'envoi de message avec Redis, Rabbit ou Kafka

	API Gateway	Spring Cloud peut créer un Zuul proxy intégré qui fournit le routage dynamique, la surveillance, la sécurité…
	Intégration des non JVM langage	Spring Cloud Sidecar qui est inspiré par Netflix Prana permet l'intégration des services qui ne se base pas sur un (Java Virtuel Machine) JVM langage. Ces services peuvent être inscrit dans l'annuaire Eureka et intégré dans l'API Gateway.
Mise à l'échelle		Facilite l'utilisation de Ribbon Load Balancer de Netflix pour une mise à l'échelle ciblée par microservice.
Supervision		Spring Cloud Sleuth offre un traçage distribué compatible avec Zipkin, Htrace, ELK

2. ÉTUDE FONCTIONNELLE DE L'ADMINISTRATION

Le diagramme de cas d'utilisation de la figure 17 décrit l'opération de l'administration de pilotage. Les cas d'utilisation « gérer les critères métiers » et « Créer les données locales complémentaires » sont effectués par l'administrateur national et le cas d'utilisation « Administrer localement les seuils critiques » peut être effectué par l'administrateur local.

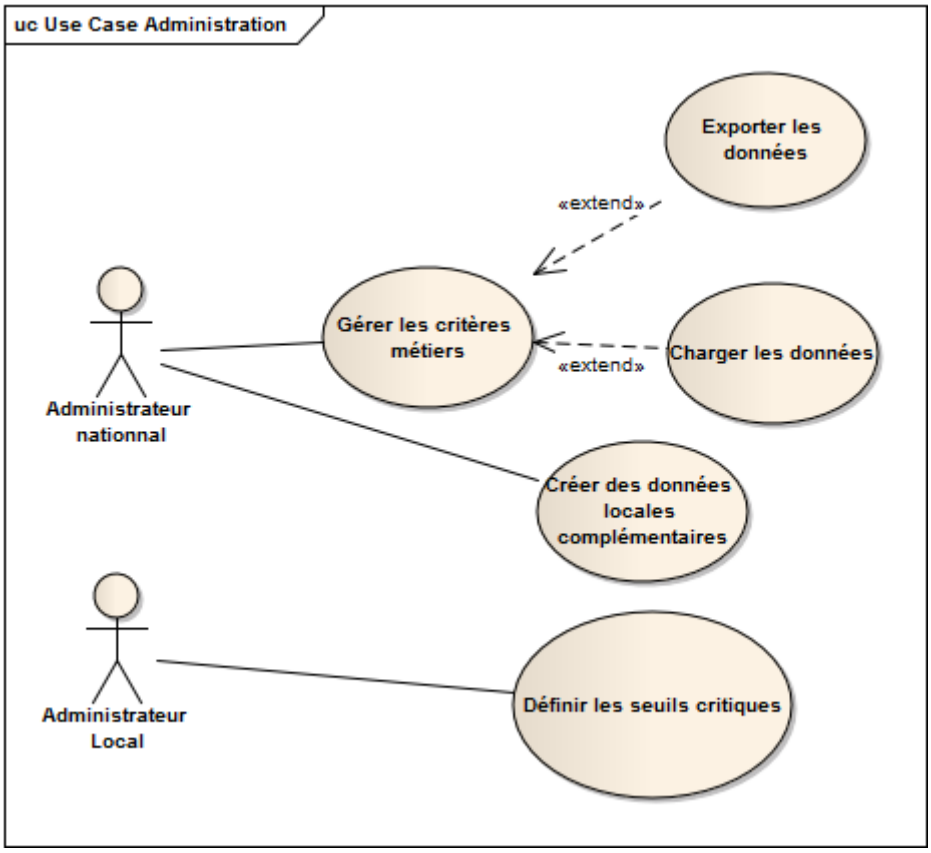


Figure 17 Diagramme de cas d'utilisation de l'administration

Nous résumons les principales étapes du cas d'utilisation « Gérer les critères métiers » dans le tableau 6.

Tableau 6 Description détaillée de cas d'utilisation "gérer les critères métiers"

	Gérer les critères métiers
Objectif	Permettre à l'administrateur national de définir les jalons majeurs par famille d'opérations
Acteurs	Administrateur national
Précondition(s)	L'utilisateur est authentifié avec le rôle administrateur national
Déclencheur(s)	Accès à l'écran d'administration
Post condition(s)	Les critères sont sauvegardés et appliqués en fonction de leur date de validité
Scénario nominal	1. L'utilisateur accède à la gestion des critères métiers 2. L'utilisateur télécharge le fichier contenant les nouveaux critères (soit composition des domaine/famille/article soit le fichier des seuils)

	3. Le portail vérifie la structure du fichier et des critères 4. Le portail intègre les nouvelles règles. 5. Le portail informe l'utilisateur du nombre de nouvelles règles ajoutées. 6. L'utilisateur peut s'il le souhaite exporter les critères existants dans un fichier Excel pour contrôler.
Scénario alternatif	Scénario nominal jusqu'à l'étape 3 1. Le portail informe l'utilisateur qu'il y a une erreur sur le fichier. 2. Aucune règle n'est ajoutée.

Les détails du cas d'utilisation « Créer des données locales complémentaires» sont listés dans le tableau 7.

Tableau 7 Description détaillée de cas d'utilisation "Créer des données locales complémentaires"

	Créer des données locales complémentaires
Objectif	Permettre à l'administrateur national de créer des données complémentaires aux opérations
Acteurs	Administrateur national
Précondition(s)	Authentifier avec le rôle administrateur national
Déclencheur(s)	Accès à la fonctionnalité de création de données complémentaires
Post condition(s)	Les données sont créées.
Scénario nominal	1. L'utilisateur accède à la création des données complémentaires 2. L'utilisateur saisit : La famille des opérations de déploiement de réseaux (les formulaires étant gérés par famille) Nom du champ (avec contrôle d'unicité avec les champs des autres sources de données) Type champ (caractères, date, numérique, lien hypertexte) Taille

	3. L'utilisateur valide la création des données 4. Le portail vérifie que les données ne sont pas déjà existantes 5. Le portail ajoute les données. 6. L'utilisateur peut maintenant les activer pour les UPR.

Le tableau 8 détaille le cas d'utilisation « Administrer localement les seuils critiques».

Tableau 8 Description détaillée de cas d'utilisation "Définir les seuils critiques"

	Définir les seuils critiques
Objectif	Permettre à l'administrateur local de définir des seuils différents de ceux établis par le métier (administrateur national)
Acteurs	Administrateur local
Précondition(s)	Etre authentifié avec le rôle administrateur local
Déclencheur(s)	Accéder à l'administration des seuils critiques
Post condition(s)	Les seuils critiques sont sauvegardés et utilisés pour les calculs en fonction de leur date de validité pour l'UPR concernée.
Scénario nominal	1. L'utilisateur affiche l'écran de gestion des seuils critiques 2. L'utilisateur charge le fichier contenant les nouveaux seuils. 3. Le portail vérifie la structure du fichier. 4. Le portail intègre les nouvelles règles, sans écraser les règles définies par l'administrateur national 5. Le portail informe l'utilisateur du nombre de nouvelles règles ajoutées 6. L'utilisateur peut s'il le souhaite exporter les critères existants dans un fichier Excel pour contrôler
Scénario d'exception	Scénario nominal jusqu'à l'étape 3 1. Le portail informe l'utilisateur qu'il y a une erreur sur le fichier. 2. Aucune règle n'est ajoutée.

3. CONCEPTION DU MICROSERVICE ADMINISTRATION

Nous détaillons dans cette partie la conception correspondante au microservice administration. Nous commençons par le diagramme de paquetages. Ensuite, nous spécifions le modèle relationnel. Puis, nous détaillons le diagramme de classe et le diagramme de séquence de quelques cas d'utilisation.

3.1. **Architecture logicielle de microservice administration**

La figure 18 présente l'architecture logicielle de microservice administration. La couche intégration est responsable des communications avec les autres microservices.

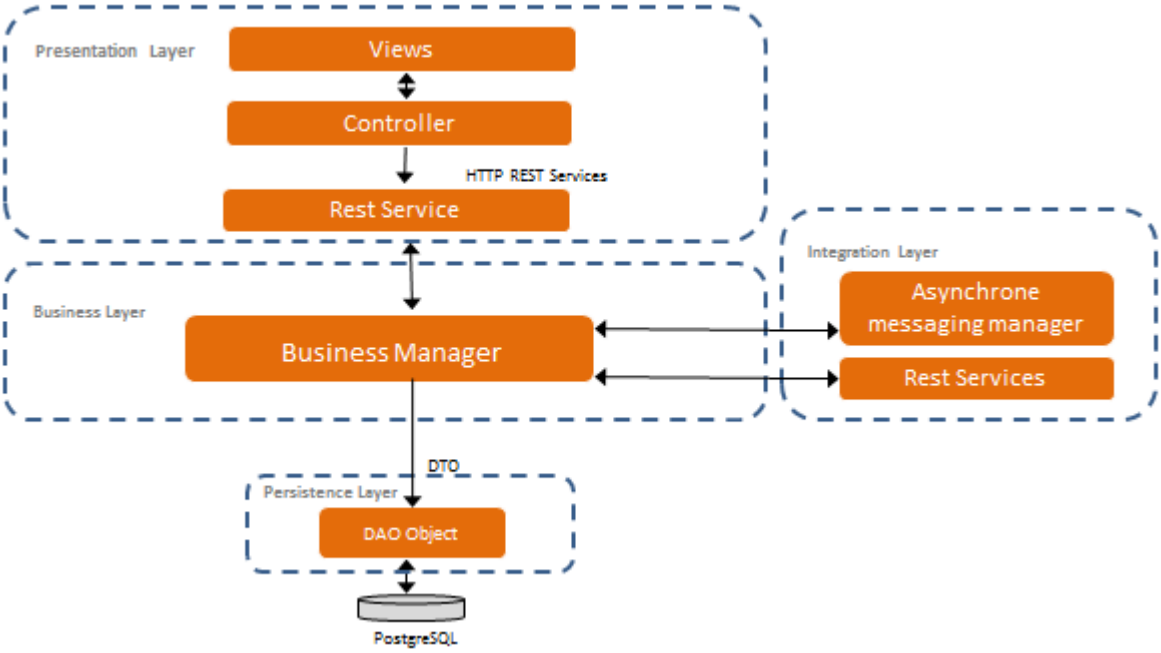


Figure 18 Architecture logicielle de microservice administration

3.2. **Conception générale de microservice administration : Diagrammes de package**

Nous illustrons les différents paquetages coté client et coté serveur afin de fournir une représentation graphique de haut niveau de l'organisation de microservice. La figure 19 illustre le diagramme de package coté client.

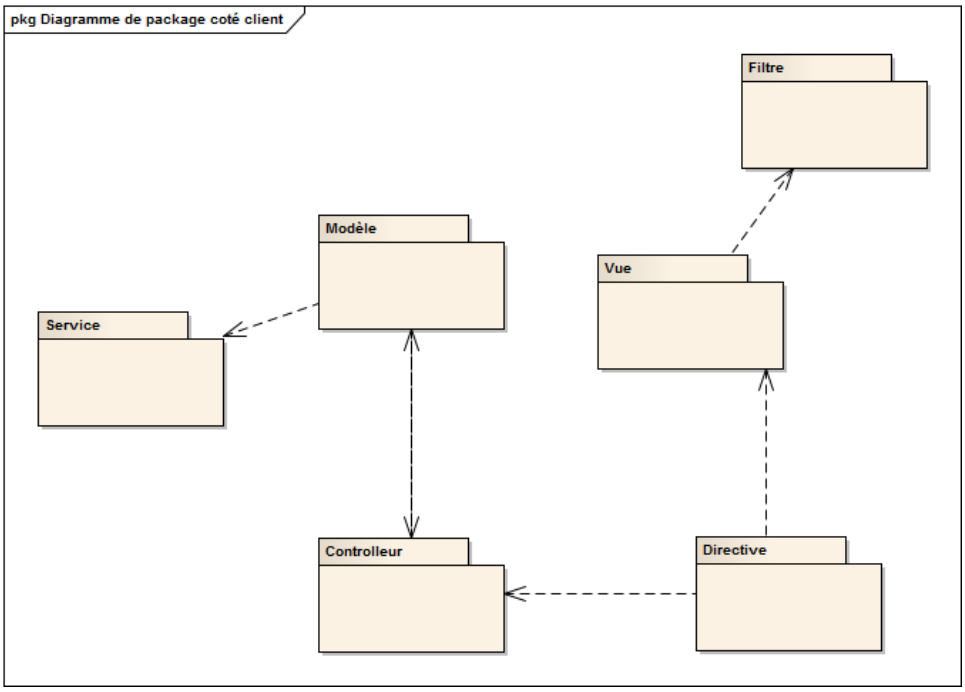


Figure 19 Diagramme de package coté client

La figure 20 montre le diagramme de package coté serveur.

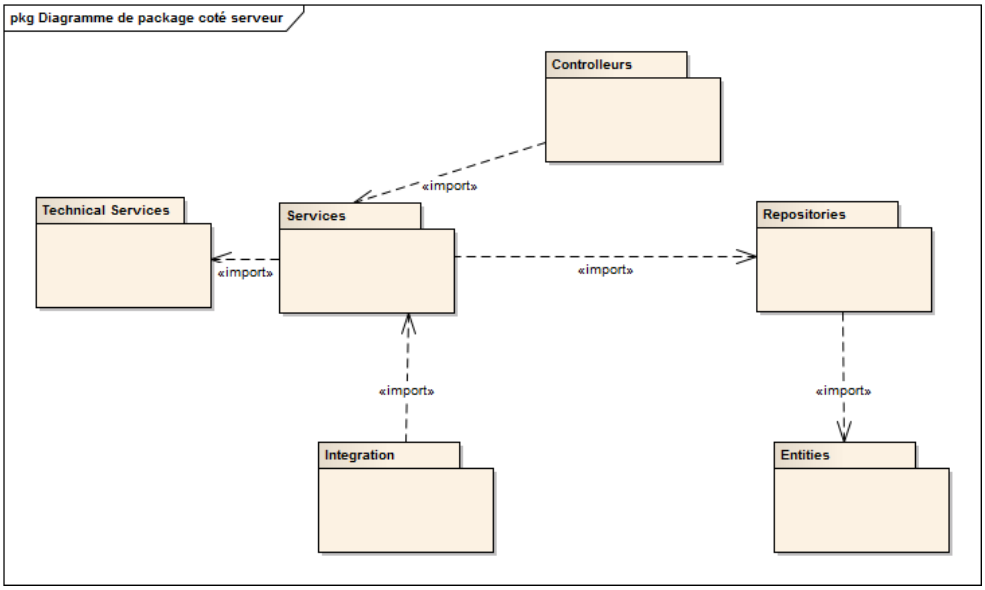


Figure 20 Diagramme de package coté serveur

3.3. **Modèle relationnel de microservice administration**

Nous détaillons dans cette partie la conception de la base de données que nous utilisons pour gérer l'administration. La base de données de ce microservice est une base relationnelle PostgreSQL. Pour décrire la base de données et les relations reliant ses différentes tables, nous réalisons le modèle de données présenté par la figure 21.

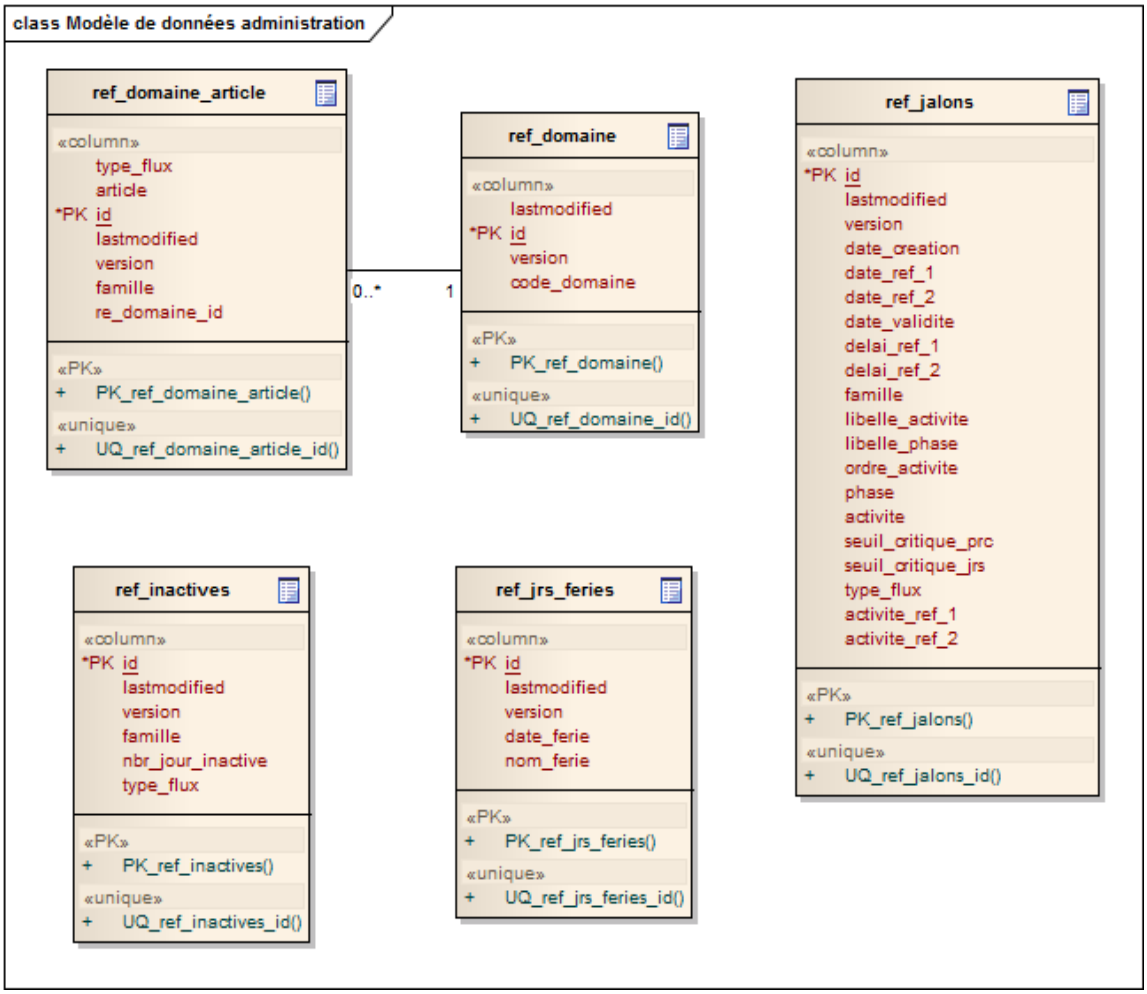


Figure 21 Modèle de données de microservice administration

3.4. **Structure globale de microservice administration**

La stucture globale de microservice administration est présenté par la figure 22. Nous avons utilisé le pattern ’Façade’ pour la couche métier. Ce pattern permet de fournir des interfaces simplifiées et uniformes pour l’exportation et l’importation des domaines, jalons, jours fériés et seuils critiques. De plus nous avons utilisé le mécanisme de l’inversion de contrôle (IOC) lors de l’instanciation des «Repositories » auprès des « Managers » et des « Managers » auprès des « Contrôleurs » en utilisant l’injection par constructeur. Ce type d’injection garantit la stabilité des instances. Le package intégration est responsable de la communication avec les autres microservices.

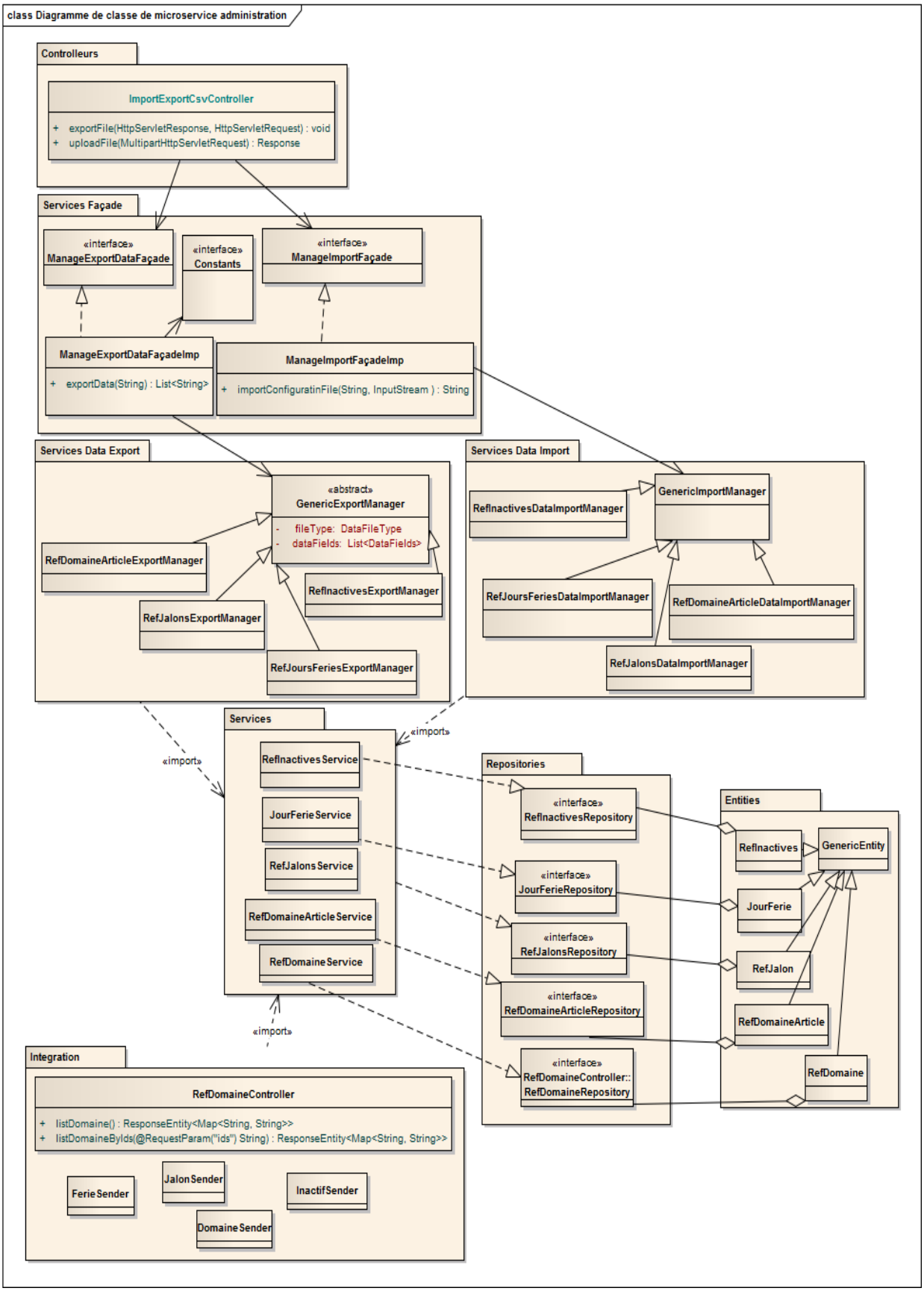


Figure 22 Structure globale de microservice administration

4. ARCHITECTURE TECHNIQUE DE MICROSERVICE ADMINISTRATION

Pour mettre en place le microservice nous avons opté pour les choix technologiques présentés dans l'architecture technique de la figure 23.

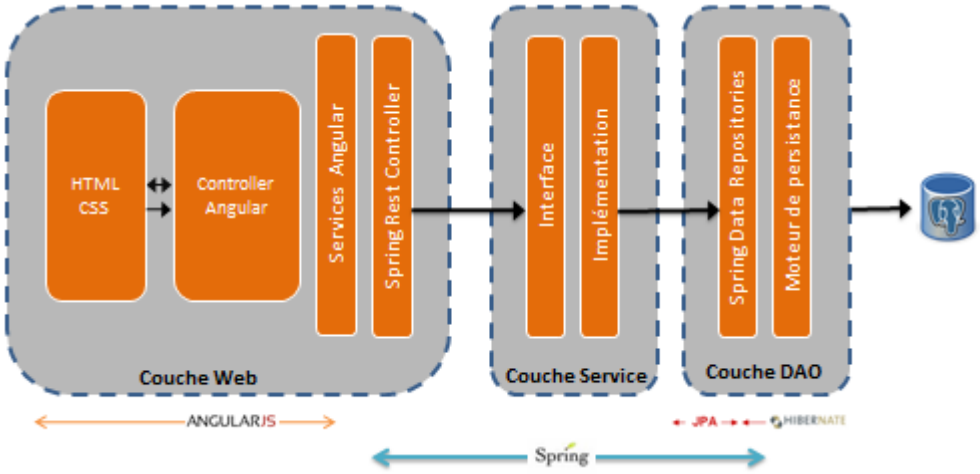


Figure 23 Architecture technique de microservice administration

5. REALISATION DE MICROSERVICE ADMINISTRATION

Les administrateurs peuvent introduire et extraire des données métiers complémentaires et des commentaires globaux des opérations pour le pilotage. A travers l'interface d'administration présentée par la figure 24, l'administrateur peut implémenter les tables de la base de données à partir des fichiers csv ou exporter les données enregistrées sous format csv.



Figure 24 Interface d'administration

6. PHASE DE TEST DE MICROSERVICE ADMINISTRATION

Les méthodes Test Driven Development (TDD) ont remis les tests unitaires, appelés « tests du programmeur », au centre de l'activité de programmation. Ces tests permettent de vérifier le bon fonctionnement d'une partie précise de l'application.

Nous avons choisi le Framework JUnit pour l'automatisation des tests unitaires de notre application. JUnit est un cadre simple pour écrire des tests reproductibles. Un aperçu sur l'exécution des tests unitaires est présenté dans la figure 25.

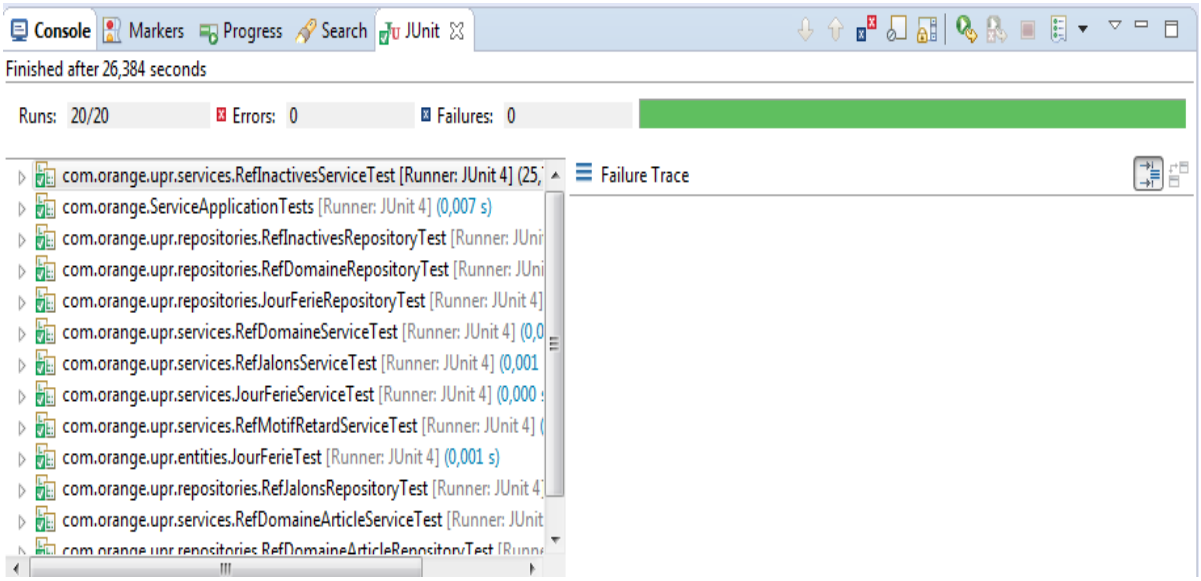


Figure 25 Aperçu sur l'exécution des tests unitaires de microservice administration

Le but derrière ces tests est la vérification des réponses des méthodes développées en fonction de l'input fourni. Principalement, le respect des contraintes d'unicité et de validation. Pour ce microservice, nous avons atteint un niveau de qualité plutôt satisfaisant. Le taux de couverture des tests unitaire est acceptable (60%).

Le tableau 9 présente les 11cas de test pour l'importation des Jalons. A savoir un cas valide et 10 doivent provoquer des erreurs de validation.

Tableau 9 Les cas de test pour l'importation des jalons

Non de fichier (input)	Message attendu (output)
REF_JALON_FILE_ACT_DUPLQ_ ERROR.csv	Ce jalon existe déjà pour une autre phase
REF_JALONS_FILE_DUPLICATIO N_ERROR.csv	Cette ligne existe déjà

REF_JALONS_FILE_FAMFLUX_ERROS.csv	Ce flux est déjà déclaré pour cette famille
REF_JALONS_FILE_WITH_ACTIV_FORMAT_ERROR.csv	Le format de la référence de l'activité n'est pas valide
REF_JALONS_FILE_WITH_DATER1_ERROR.csv	Le champ Date_REF_1 ne respecte aucun des formats suivants : T0, Date confirmation/XXXX, Date lancement/XXXX"
REF_JALONS_FILE_WITH_DATEREF2_FORMAT_ERROR.csv	Le champ DATE_REF_2 ne respecte aucun des formats suivants : DC OPUS, Date de fin au plus tard/XXXX
REF_JALONS_FILE_WITH_DELAIREF2_ERROR.csv	Délai ref 2 est vide
REF_JALONS_FILE_WITH_DREF2_ERROR.csv	Date ref 2 est vide
REF_JALONS_FILE_WITH_FLUX_FORMAT_ERROR.csv	Le flux contient des caractères spéciaux ou des espaces
REF_JALONS_FILE_WITH_NOTFOUNDFAMILLE_ERROR.csv	La famille est inconnue de la liste des domaines
REF_JALONS_FILE_WITH_SEUILPRC_FORMAT_ERROR.csv	La valeur du seuil en pourcentage est incorrecte
REF_JALONS_FILE_WITH_SEUILS_ERROR.csv	Aucun seuil n'est renseigne
REF_JALONS_VALID_FILE.csv	Les jalons ont été importés avec succès

Suite aux résultats de test, nous avons procédé aux correctifs nécessaires. Après ces correctifs les tests sont validés.

CONCLUSION

Dans ce chapitre nous avons présenté le cycle de vie de microservice administration. Nous avons commencé par une étude fonctionnelle. Puis, nous avons détaillé la conception grâce à un diagramme de classe et un diagramme de séquence. Ensuite, nous avons présenté la réalisation du microservice. Enfin, nous avons illustré les tests réalisés.

A la fin de ce Sprint nous avons un microservice autonome livrable qui gère l'administration du pilotage. Le chapitre suivant est consacré au microservice portefeuille.

CHAPITRE V

SPRINT 2 : MICROSERVICE PORTEFEUILLE

Sommaire

Introduction 48
1. Étude fonctionnelle du microservice portefeuille 48
2. Conception de microservice portefeuille 50
   2.1. Architecture logicielle de microservice portefeuille 50
   2.2. Modèle relationnel de microservice portefeuille 50
   2.3. Structure globale de microservice portefeuille 51
   2.4. Modélisation dynamique de cas d'utilisation modifier portefeuille 53
   2.5. Diagramme de déploiement de microservice portefeuille 54
3. Configuration logicielle de microservice portefeuille 54
4. Réalisation de microservice portefeuille 55
5. Test de microservice portefeuille 56
Conclusion 57

INTRODUCTION

Ce chapitre présente le cycle de vie du deuxième Sprint. A savoir une analyse fonctionnelle détaillée, la conception, les choix technologiques, la réalisation et le test de microservice portefeuille.

1. ÉTUDE FONCTIONNELLE DU MICROSERVICE PORTEFEUILLE

La figure 26 présente le diagramme de cas d'utilisation de microservice portefeuille.

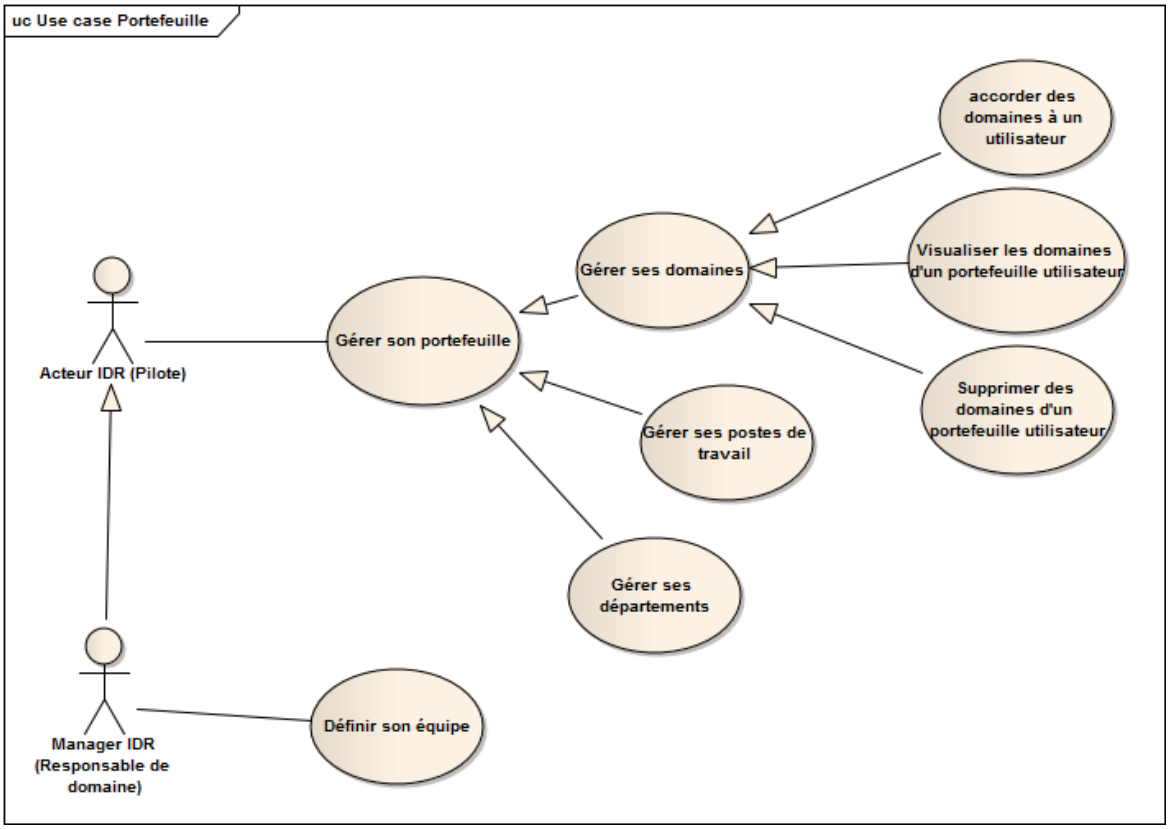


Figure 26 Diagramme de cas d'utilisation de gestion de portefeuille

Le tableau 10 détaille le cas d'utilisation « Gérer son portefeuille »

Tableau 10 description de cas d'utilisation "Gérer son portefeuille"

	Gérer son portefeuille
Objectif	Permettre à l'utilisateur de définir le périmètre de son portefeuille d'opération
Acteurs	Acteur IDR (Pilote) Manager IDR (Responsable de domaine)
Précondition(s)	L'utilisateur est authentifié sur l'application avec son rôle/droits

Déclencheur(s)	Accès à la fonctionnalité de paramétrage
Post condition(s)	Le périmètre du portefeuille de l'utilisateur est sauvegardé.
Scénario nominal	L'utilisateur est sur la page de définition de son portefeuille. Il choisit obligatoirement : son UPR au moins 1 domaine Il peut saisir (facultatif) : d'autres domaines 1 à N postes de travail de son UPR 1 à N départements de son UPR L'utilisateur valide le périmètre de son portefeuille Un message s'affiche pour indiquer que le portefeuille a bien été sauvegardé
Scénario alternatif	L'utilisateur est sur la page de définition de son portefeuille. Il commence à définir son portefeuille A tout moment il annule sa saisie Un message est affiché pour le prévenir qu'il va perdre sa saisie S'il valide, les modifications de son portefeuille ne sont pas sauvegardés et les anciennes valeurs si elles existent sont affichées, si elles n'existent pas les champs sont vidés. S'il annule il peut reprendre sa saisie en cours là où il l'avait arrêtée.

Le tableau 11 détaille le cas d'utilisation « Définir son équipe »

Tableau 11 description de cas d'utilisation "Définir son équipe"

	Définir son équipe
Objectif	Permettre au manager de définir la composition de son équipe pour qu'il puisse avoir la vision globale des portefeuilles de son équipe.
Acteurs	Manager IDR (Responsable de domaine)
Précondition(s)	Le manager doit avoir défini son UPR
Déclencheur(s)	Accès à la définition de l'équipe dans la gestion du portefeuille

Post condition(s)	La composition de l'équipe est sauvegardée et prise en compte.
Scénario nominal	Dans la gestion du portefeuille, l'utilisateur accède à la section pour définir son équipe. Il saisit le/les identifiants (CUID) des pilotes qu'il souhaite suivre. Le portail ajoute le CUID saisi et recherche le nom correspondant et l'affiche s'il le connait. L'utilisateur valide les informations saisies. Le portail enregistre la composition de l'équipe.

2. CONCEPTION DE MICROSERVICE PORTEFEUILLE

2.1. **Architecture logicielle de microservice portefeuille**

Le microservice portefeuille est une application web en N tiers. La figure 27 montre l'architecture logicielle de cette application.

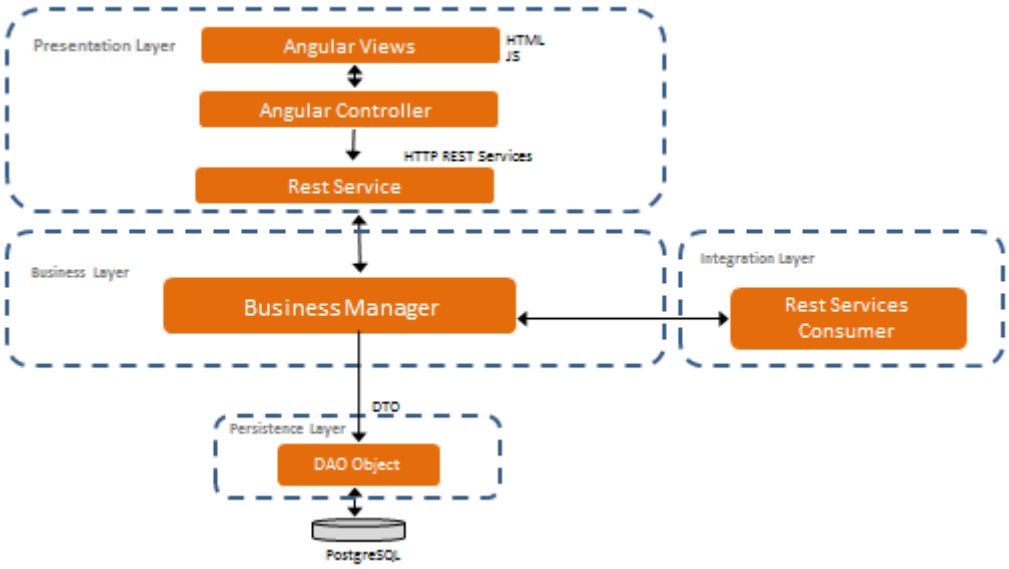


Figure 27 Architecture logicielle de microservice portefeuille

2.2. **Modèle relationnel de microservice portefeuille**

Le microservice portefeuille possède une base de données relationnelle « PostgreSQL » dont le modèle est présenté dans la figure 28.

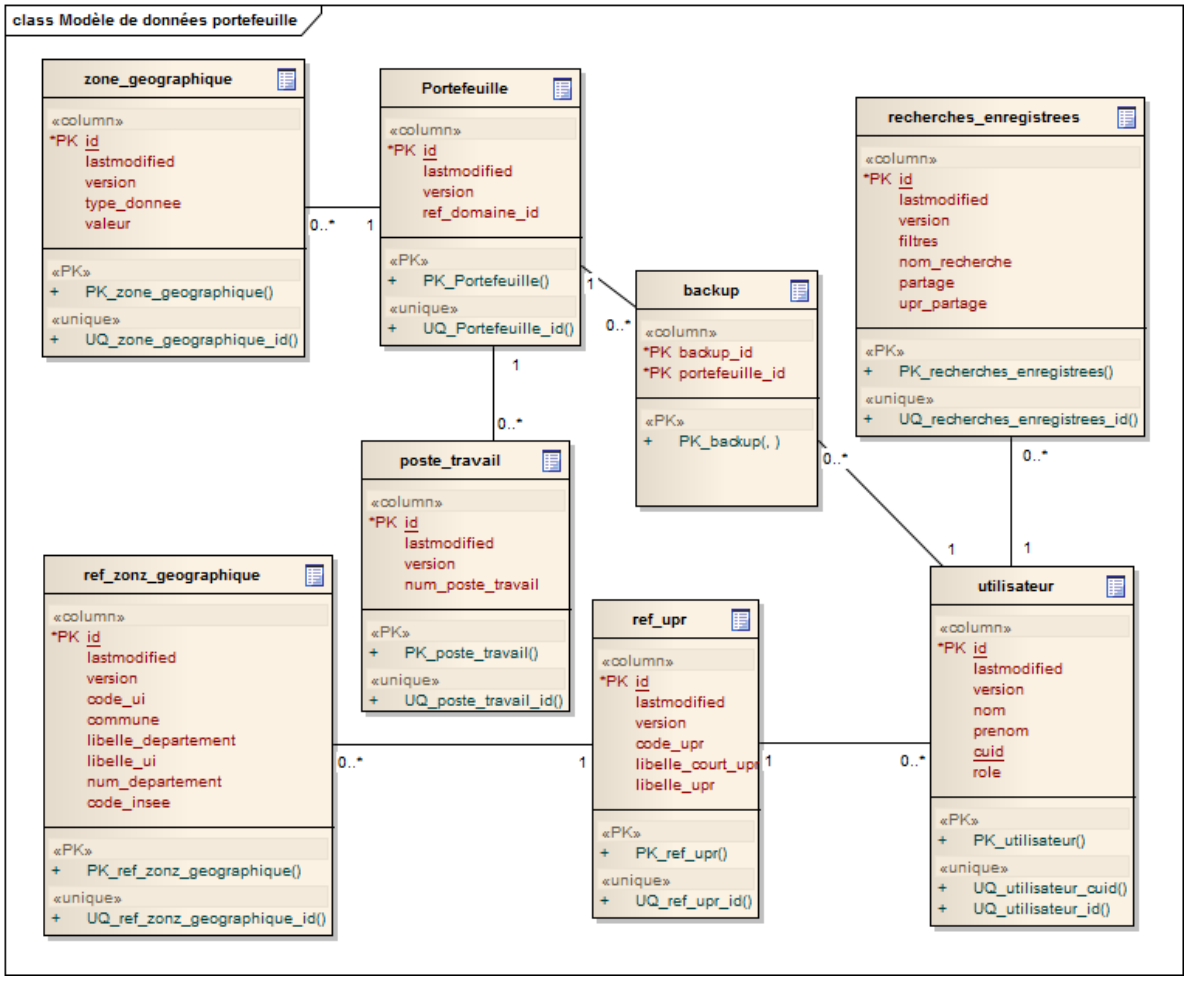


Figure 28 Modèle de domaine de gestion de portefeuille

2.3. **Structure globale de microservice portefeuille**

Le diagramme de la figure 29 présente les classes de microservice de gestion de portefeuille.

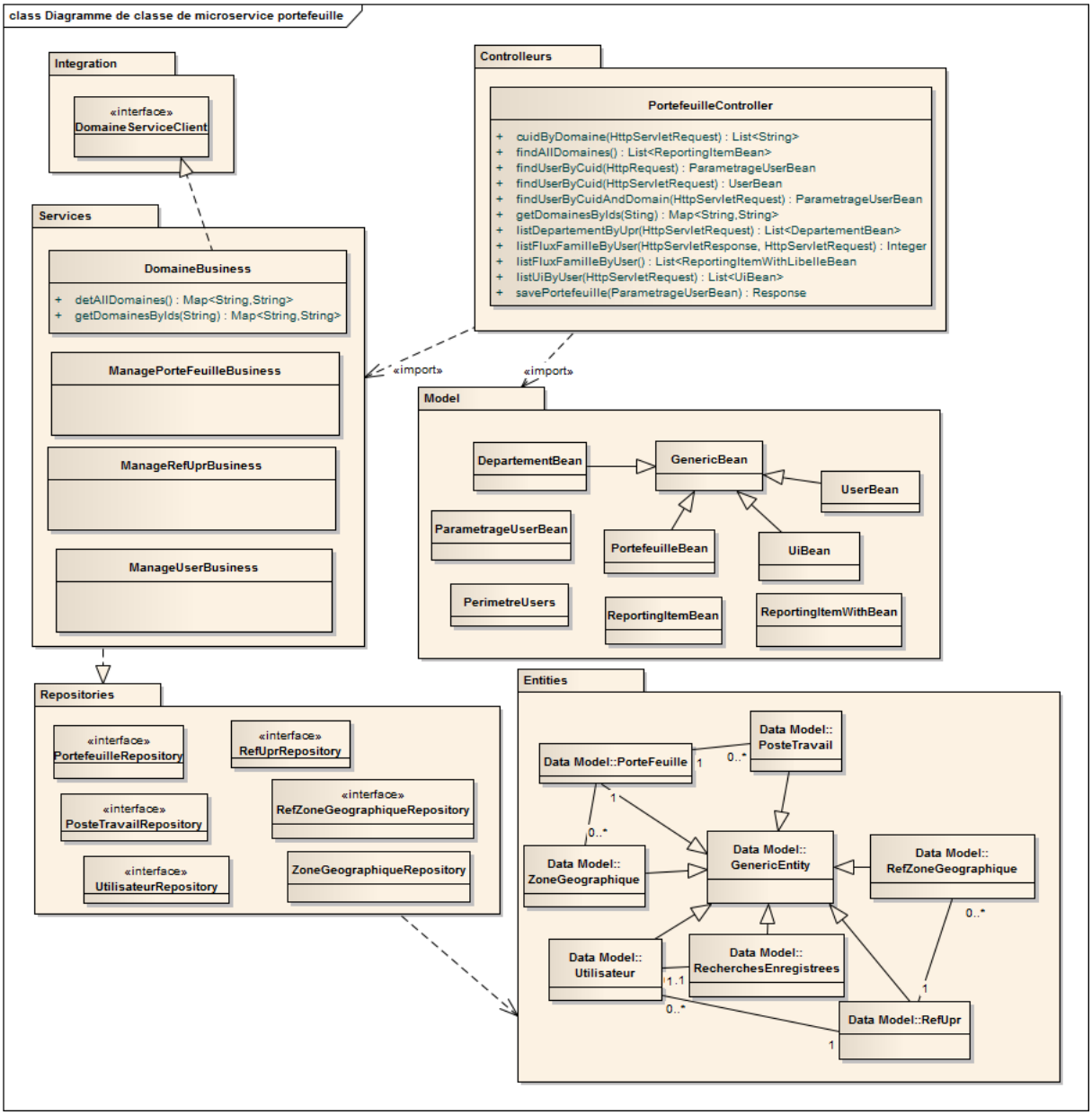


Figure 29 Structure globale de microservice portefeuille

2.4. **Modélisation dynamique de cas d'utilisation modifier portefeuille**

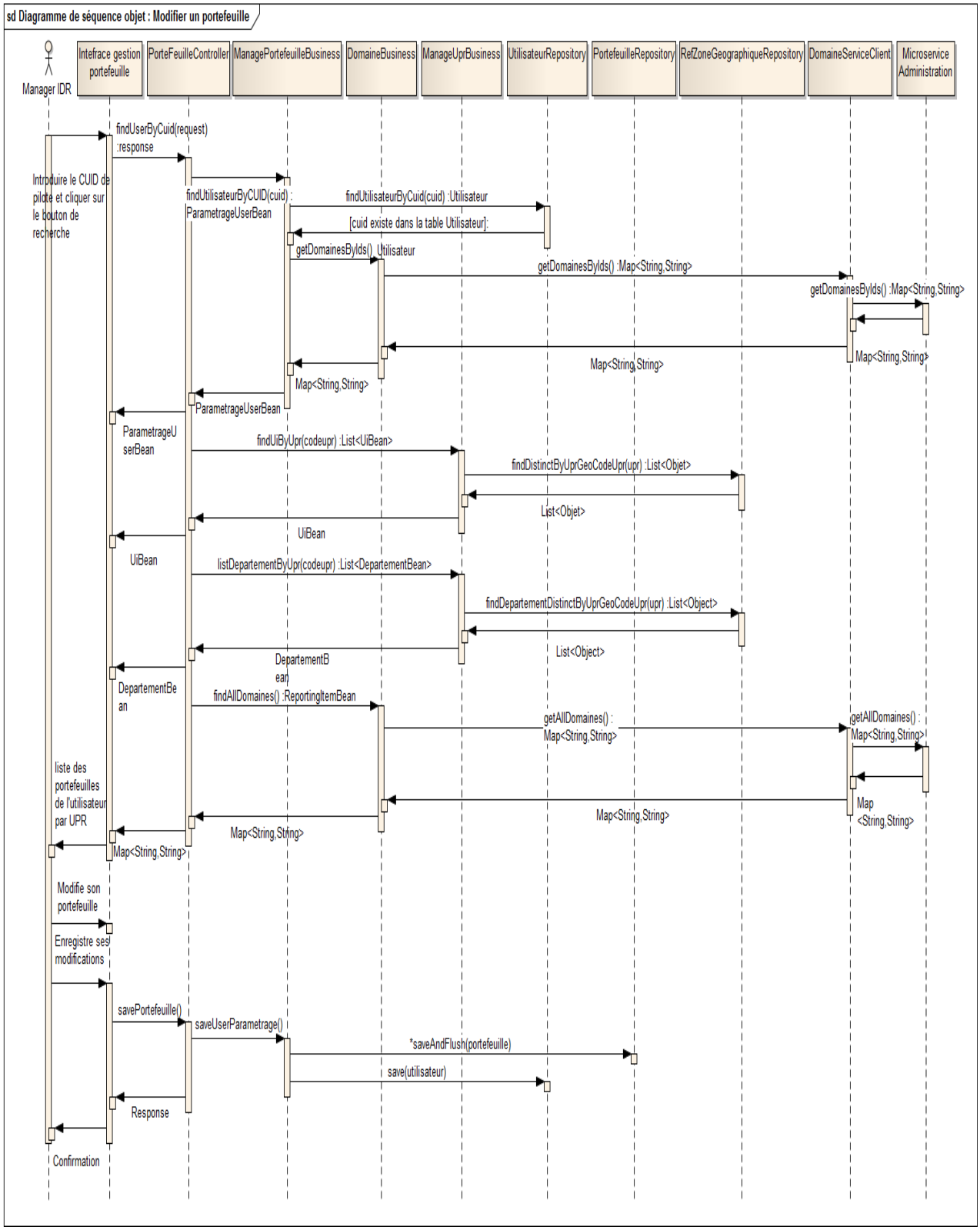


Figure 30 Diagramme de séquence : Modifier un portefeuille

2.5. **Diagramme de déploiement de microservice portefeuille**

Le diagramme de déploiement présenté dans la figure 31 permet de comprendre la communication entre les différents composants de microservice.

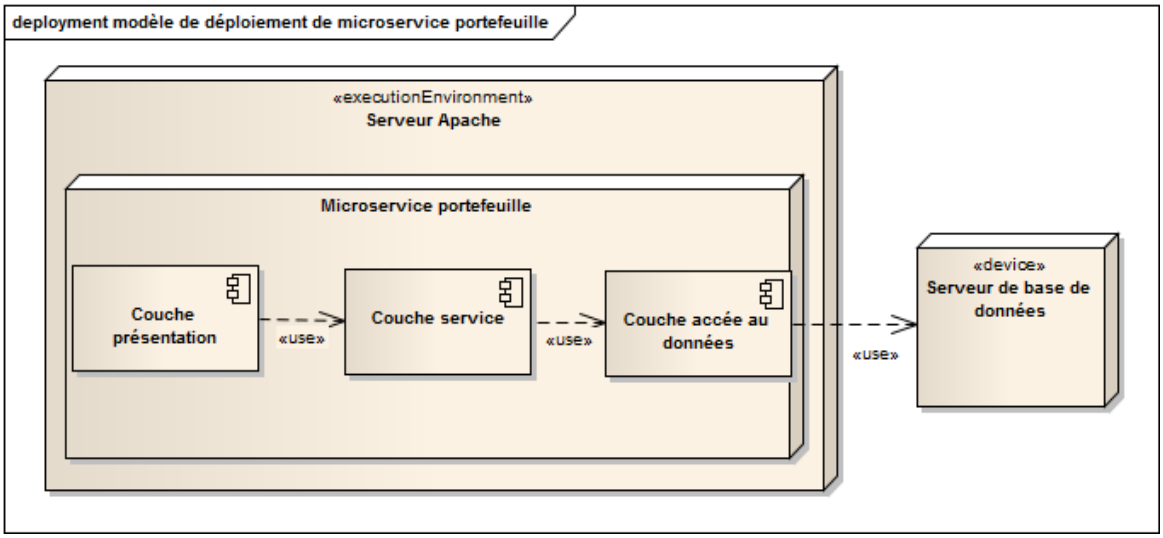


Figure 31 Diagramme de déploiement de microservice portefeuille

3. CONFIGURATION LOGICIELLE DE MICROSERVICE PORTEFEUILLE

Malgré l'orientation générale prise pour l'application chaque microservice peut bénéficier de ses propres configurations logicielles et choix techniques. Pour réaliser le microservice portefeuille JEE nous avons utilisé les outils suivants :

- **JDK 1.8**

Java Development Kit (JDK) désigne un ensemble de bibliothèques logicielles de base du langage de programmation Java. Dans ce microservice nous avons utilisé java 8.

- **Spring Tool Suite 3.7.2**

Spring Tool Suite (STS) est un environnement de développement basé sur Eclipse et recommandé pour tout développement Spring.

- **Spring cloud Brixton.RC2**

Spring Cloud est un framework qui facilite le développement des systèmes distribués. Au démarrage de ce projet de refonte nous avons utilisé la version « Brixton.M3 » de spring cloud. Le problème de corrélation ID connu dans la supervision des architectures microservice, n'a

pas été résolu dans cette version. C'est pourquoi nous avons eu recours à la version « Brixton.RC2 » sortie en 19 avril 2016 qui a résolu ce problème.

- **Spring Boot 1.3.3.RELEASE**

Spring Boot est un projet Spring qui simplifie le démarrage et le développement des applications. Nous avons commencé le développement de microservice avec la version « 1.3.0.RELEASE ». En raison de non compatibilité avec la nouvelle version de spring cloud, nous avons basculé vers la version « 1.3.3.RELEASE ».

- **Dozer 5.5.1**

DOZER est un convertisseur de formats. Il procède en suivant le patron de conception Mapper. Etant donné que nous cherchons à éviter la dépendance directe entre les objets de la couche DAO et les modèles à exposer sous formats JSON à la couche présentation. Ce patron de conception se chargera des communications entre ces différents objets.

- **PostgreSQL 9.5**

PostgreSQL est un système de gestion de base de données relationnelle et objet (SGBDRO) qui peut gérer des gros volumes par rapports à des SGBD relationnelles.

- **Spring data jpa et spring data rest**

Spring data est un projet de Spring qui répond aux besoins d'écrire plus simplement l'accès aux données et d'avoir une couche d'abstraction commune à de multiples sources de données. En fait, Spring data jpa offre une implémentation par défaut des repositories JPA. Spring data rest est au-dessus des repositories. Il analyse le modèle de domaine et expose les ressources http.

- **Groovy**

Groovy est le nom d'un langage de programmation orienté objet destiné à la plate-forme Java. Nous utilisons ce langage pour implémenter le fichier logback qui est responsable à la gestion des logs.

4. REALISATION DE MICROSERVICE PORTEFEUILLE

Le manager peut introduire une clé unique d'un membre de son équipe pour gérer son portefeuille. L'interface de gestion de portefeuille est présente par la figure 34. Cette interface

permet de définir le périmètre des opérations que l'utilisateur doit suivre par les critères suivants:

- Unité de production Réseau (UPR)
- Domaine
- Pour chaque domaine : poste(s) de travail, Unité(s) d'intervention(UI), département(s), backup

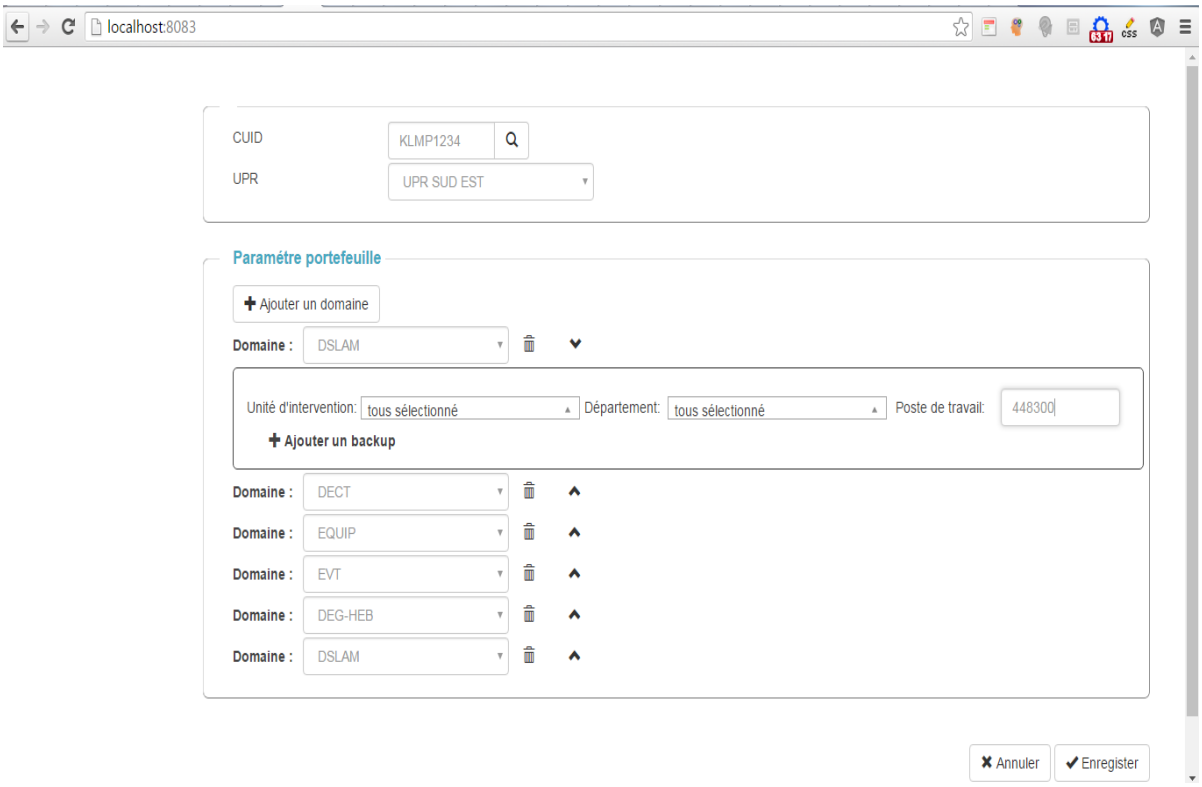


Figure 32 Interface de gestion de portefeuille

5. TEST DE MICROSERVICE PORTEFEUILLE

Pour vérifier le bon fonctionnement des services et repositories de microservice portefeuille nous avons réalisé les tests unitaires présentés par la figure 33. A travers ces tests nous vérifions l'ajout et la modification des portefeuilles. Nous utilisons ainsi des Mocks pour suivre les états des objets et simuler le comportement de mappage entre les objets avec dozer. Le taux de couverture des tests unitaires pour ce microservice est proche de 50% et les tests réalisés sont validés.

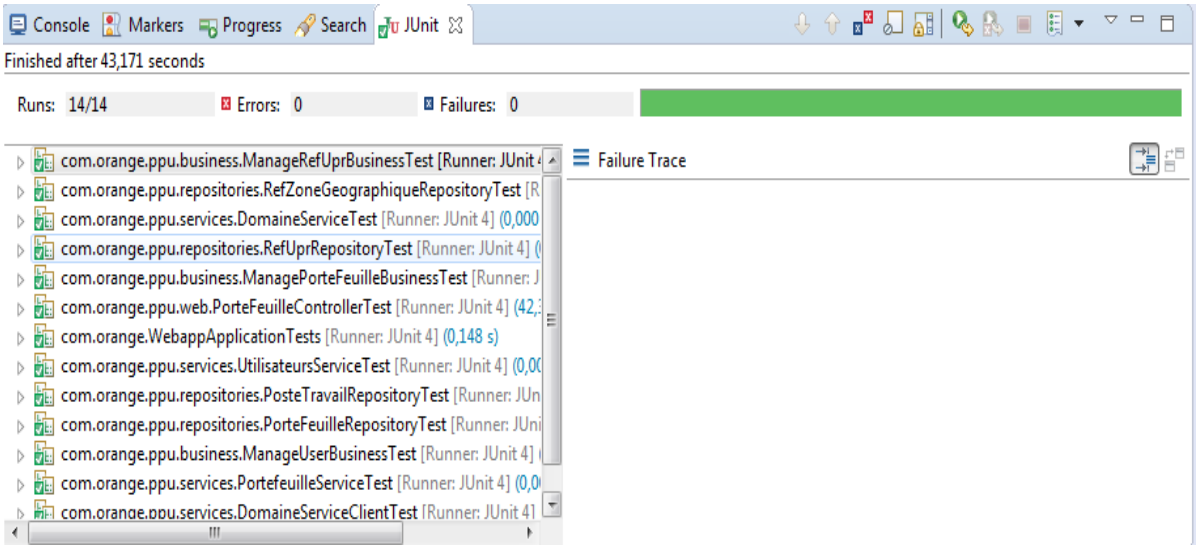


Figure 33 Test unitaire de microservice portefeuille

CONCLUSION

Durant ce chapitre nous avons présenté la construction de microservice portefeuille. Nous avons mis l'accent sur sa conception puis le livrable sur lequel nous avons effectué les tests unitaires puis il était validé par le product owner. Le chapitre VI détaille le cycle de vie de microservice reporting.

CHAPITRE VI

SPRINT 3 : MICROSERVICE REPORTING

Sommaire

Introduction 58
1. Étude fonctionnelle de microservice reporting 58
2. Solution pour l'amélioration de temps de réponse 60
3. Conception de microservice reporting 62
   3.1. Architecture logicielle de microservice reporting 62
   3.2. Modèle de données de la base NOSQL orienté document 62
   3.3. Structure globale de microservice reporting 65
4. Choix technologiques de reporting 65
   4.1. Base de données : MongoDB 65
   4.2. Framework d'accès aux données 66
5. Réalisation de microservice reporting 67
6. Test de microservice reporting 68
Conclusion 68

INTRODUCTION

Ce microservice est le noyau de portail de pilotage. Par contrainte de temps nous avons choisi de développer quelques fonctionnalités de ce service afin de favoriser les aspects non fonctionnels tels que le temps de réponse, le monitoring, la tolérance aux pannes et la mise à l'échelle.

1.

2. ÉTUDE FONCTIONNELLE DE MICROSERVICE REPORTING

Cette phase consiste à cadrer les fonctionnalités réalisées du microservice reporting à partir de la présentation des cas d'utilisation. La figure 34 présente le diagramme de cas d'utilisation pour les fonctions réalisés du microservice reporting.

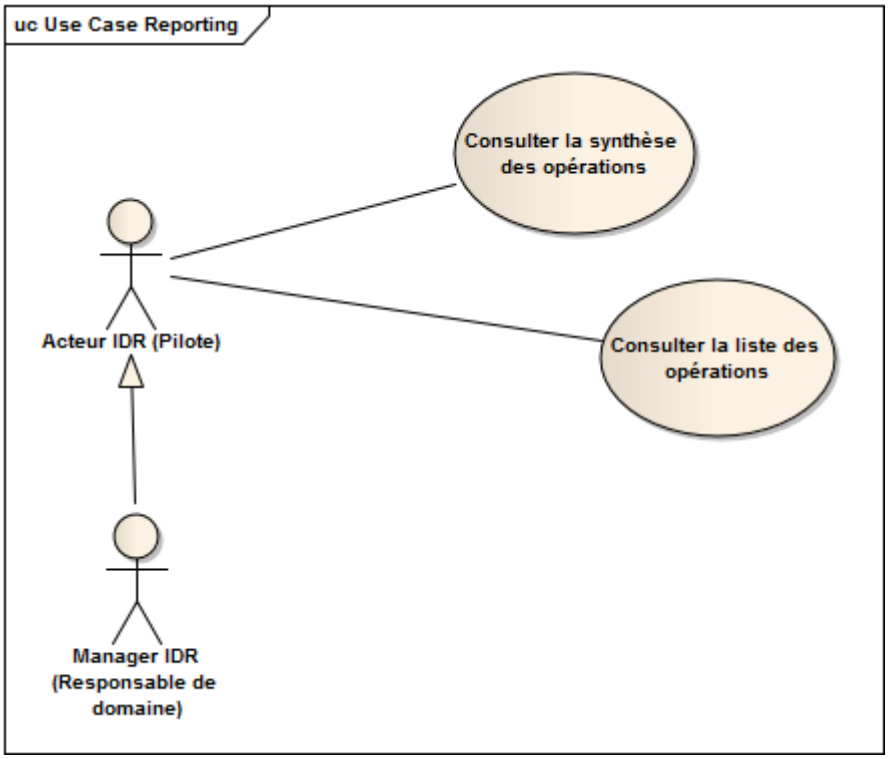


Figure 34 Diagramme de cas d'utilisation de reporting

La description de cas d'utilisation « Consulter la synthèse des opérations d'une famille est présenté par le tableau 12.

Tableau 12 Cas d'utilisation : Consulter la synthèse des opérations d'une famille

	Consulter la synthèse des opérations d'une famille
Objectif	Permettre à l'utilisateur visualiser la ventilation de ses opérations en cours par article en fonction des seuils d'alertes (règles métier et/ou des règles locales)
Acteurs	Acteur IDR (Pilote) Manager IDR (Responsable de domaine)
Précondition(s)	L'utilisateur a un périmètre de portefeuille défini.
Déclencheur(s)	Connexion au portail ou Navigation sur la synthèse des opérations
Post condition(s)	Rapport sur les opérations en cours
Scénario nominal	• L'utilisateur se connecte • Le portail récupère le périmètre du portefeuille de l'utilisateur UPR, domaines, UI, … • Pour chaque famille (issues du périmètre), le portail construit la structure du tableau de synthèse : -Pour les lignes : liste des articles et des flux paramétrés par le métier -Pour les colonnes (articles de la famille): le total des mouvements le total des opérations = somme des opérations de la ligne le nombre d'opérations inactives les jalons majeurs et les activités suivies : défini par l'administrateur local • Avec le périmètre le portail récupère les opérations correspondantes et les ventile selon leur état actuel et le seuil calculé • La page d'accueil est affichée : il s'agit du tableau de synthèse des opérations de la première famille de son portefeuille.

La description de cas d'utilisation « consulter la liste des opérations » est présentée dans le tableau 13.

Tableau 13 Cas d'utilisation : Consulter la liste des opérations

	Consulter la liste des opérations
Objectif	Permet à l'utilisateur de visualiser la liste des opérations correspondant à la famille, à l'article, à l'activité et au seuil qu'il a sélectionné dans le tableau de synthèse
Acteurs	Acteur IDR (Pilote). Manager IDR (Responsable de domaine).
Précondition(s)	L'utilisateur consulte le tableau de synthèse des opérations
Déclencheur(s)	Clic sur une cellule du tableau de synthèse
Post condition(s)	Visualisation de la liste des opérations sélectionnés sans le tableau de synthèse
Scénario nominal	• L'utilisateur clique sur une cellule du tableau de synthèse en cours de consultation. • Le portail récupère les critères de la cellule, la famille, l'article et le périmètre de l'utilisateur ainsi que : Pour la colonne des mouvements : toutes les opérations de l'article ayant un mouvement non consulté. Pour la colonne des inactives : toutes les opérations qui n'ont pas eu de mouvement depuis X jours (nombre de jours à définir) Pour les jalons : l'activité, le seuil. • Le portail affiche la liste des opérations correspondant à ces critères (il doit y avoir dans la liste le même nombre de ligne que le nombre d'opérations indiqué dans la cellule du tableau de synthèse). • L'utilisateur consulte la liste.

3. SOLUTION POUR L'AMELIORATION DE TEMPS DE REPONSE

Les bases de données relationnelles sont conçues en respectant les formes normales. Ces formes permettent de limiter les redondances de données et d'éviter les processus de réécritures. Cette normalisation provoque une lenteur de lecture avec les requêtes complexes. Les besoins de reporting engendrent une complexité des requêtes et des volumes de données importants.

Par conséquente une lenteur de réponse. Dans le contexte où le temps de réponse est exigé par l'unité de production nous avons décidé de s'éloigner de la normalisation pour favoriser le temps de lecture. Les bases SQL ne sont pas adéquates aux grandes tailles de données, la mise en œuvre des clusters est complexe et elle ne se marie pas avec la dénormalisation. Donc nous avons eu recours aux bases NOSQL [B8] grâce à leurs requêtes sans jointure, leur réplication simple, leur cohérence à terme, leur énorme volume de données et leur scalabilité. La figure 35 présente les propriétés des bases NOSQL par rapport à celle des bases SQL. Selon le théorème CAP de Brewer il est impossible sur un système informatique de calcul distribué de garantir en même temps les trois contraintes suivantes :

- Cohérence : tous les nœuds du système voient exactement les mêmes données au même moment.
- Disponibilité : garantie que toutes les requêtes reçoivent une réponse.
- Tolérance au partitionnement : aucune panne moins importante qu'une coupure totale du réseau ne doit empêcher le système de répondre correctement.

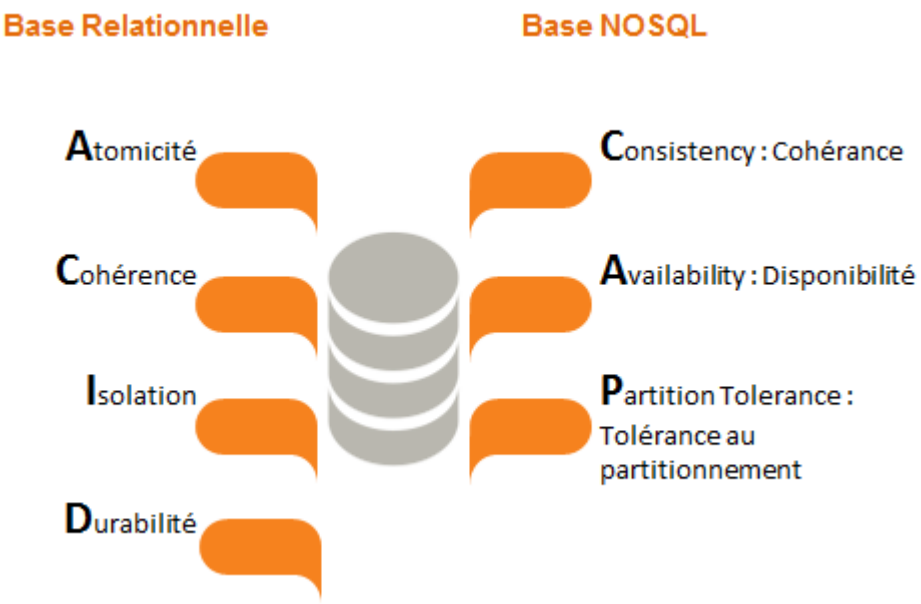


Figure 35 Propriétés des bases relationnelles et NOSQL

Tous les systèmes NOSQL devraient vérifier la propriété P (Partition : Tolérance). A (Availability : Disponibilité) ou C (Consistency : Cohérance) dépend des compromis effectués entre rapidité de réponse et disponibilité des données.

Nous avons choisi d'utiliser une base orienté document pour pouvoir récupérer sans jointure les résultats de reporting.

L'annexe A présente les familles des bases NOSQL.

4. CONCEPTION DE MICROSERVICE REPORTING

3.1. **Architecture logicielle de microservice reporting**

Le microservice reporting est une application web en N-tiers. Nous présentons son architecture logicielle dans la figure 36.

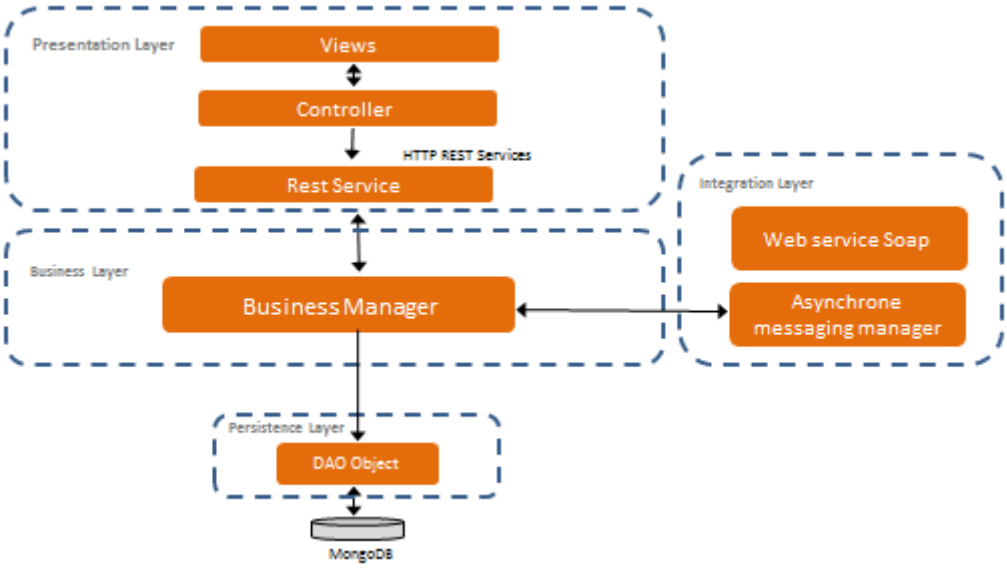


Figure 36 Architecture logicielle de microservice reporting

3.2. **Modèle de données de la base NOSQL orienté document**

Les sources de données de microservice reporting sont externes. Les données métiers avec le préfixe « Adm » sont reçues de microservice administration et les données avec le préfixe « Opus » sont intégrées à partir du workflow OPUS à l'aide d'un web service SOAP. Ses données sont représentées sous forme de données avant la phase de dénormalisation par la figure 37.

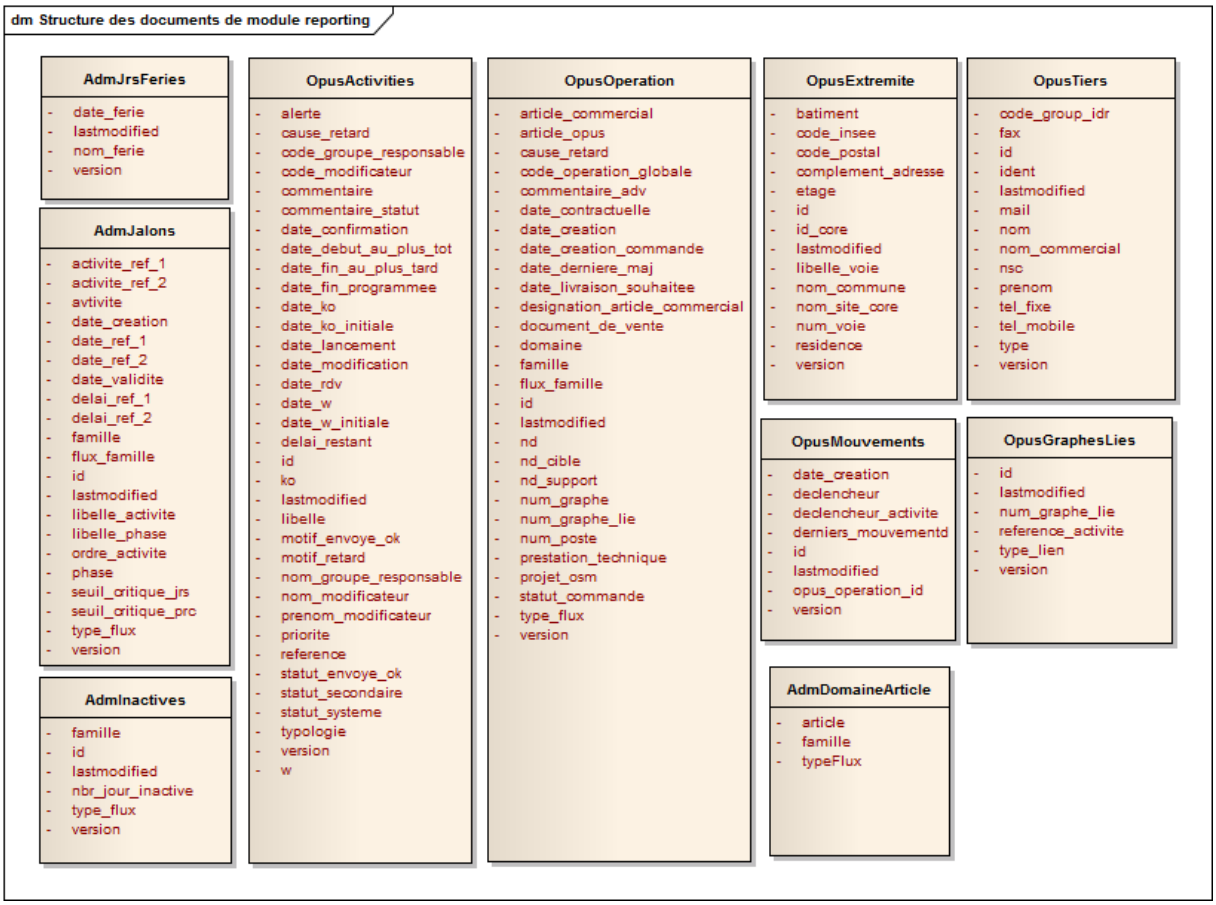


Figure 37 Structure des documents de reporting sans dénormalisation

Pour construire notre modèle de données nous avons utilisé quatre stratégies de dénormalisation présenté dans le tableau 14.

Tableau 14 Règles de dénormalisation

Règle de dénormalisation	**Explication**	**Utilisation**
Imbrication de document	Le document fils est imbriqué dans le document parent	Si on crée le document parent avant le document enfant
Duplication d'un sous ensemble de champs	L'idée est de dupliquer le minimum de données d'un document à un autre. De préférence celles qui ne changent pas souvent	Si le temps de lecture et prioritaire par rapport au temps d'écriture

Ajout d'un champ calculé	Consiste à sauvegarder des champs calculés	
Duplication de la clé primaire	Consiste à dupliquer la clé primaire du document B dans A	Si on veut faire un compromis entre le temps d'écriture et de lecture

La figure 38 illustre les actions de dénormatisation appliqués sur la base de reporting.

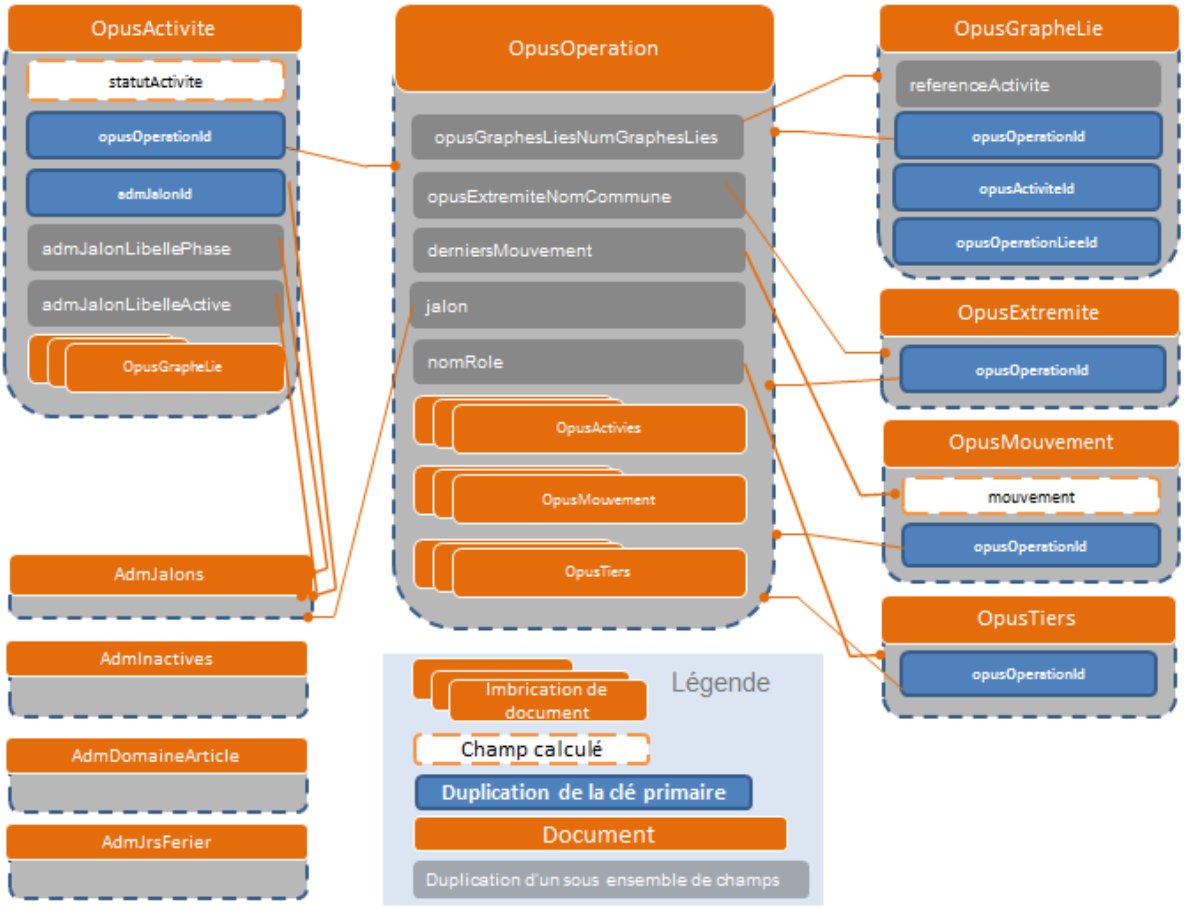


Figure 38 Modèle qui illustre la dénormalisation de la base de reporting

3.3. **Structure globale de microservice reporting**

La figure 39 présente la structure globale de microservice reporting.

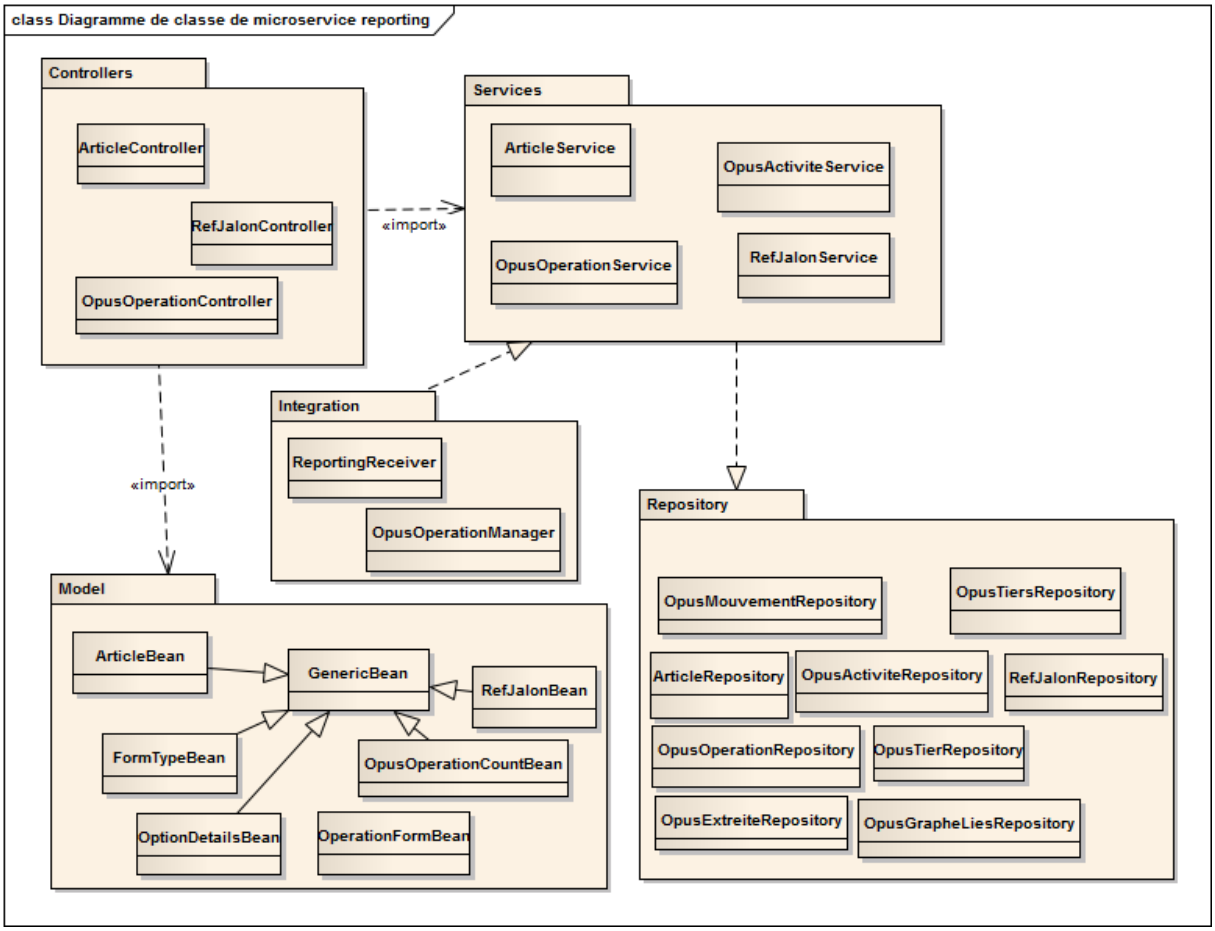


Figure 39 Structure globale de microservice reporting

5. CHOIX TECHNOLOGIQUES DE REPORTING

4.1. **Base de données : MongoDB**

MongoDB est la base NoSQL orienté document la plus populaire. Elle est développée par une société nommée 10 Gen, qui vend des services autour de MongoDB. Le quadrant magique de « Gartner » publié en 12 Octobre 2015 [W5] classifie la base de données MongoBD en tant que « leader » parmi les systèmes de base de données.

MongoDB est une base orientée document codé en « C++ » et utilise « Full-text search » pour la recherche des données. Les données sont stockées en BSON, c'est-à-dire du JSON encodé en binaire. Les documents sont stockés dans des collections, qui sont l'équivalent des tables dans une base relationnelle. MongoDB se base sur trois composants. Mongod « data store » est le responsable du stockage de données. Les serveurs de configuration détiennent les copies des métas donnés. Mongos

Présente l'ensemble des routeurs qui dirigent les clients de la base. Parmi les avantages de cette base nous citons :

- La latence très faible en lecture
- La latence très faible en écriture
- La haute disponibilité
- L'ajout facile des nœuds au cluster
- La réactivité de la communauté

Nous avons utilisé l'outil Robomongo pour l'administration des bases de données de MongoDB. La figure 40 met l'accent sur un document de la collection OpusOperation.

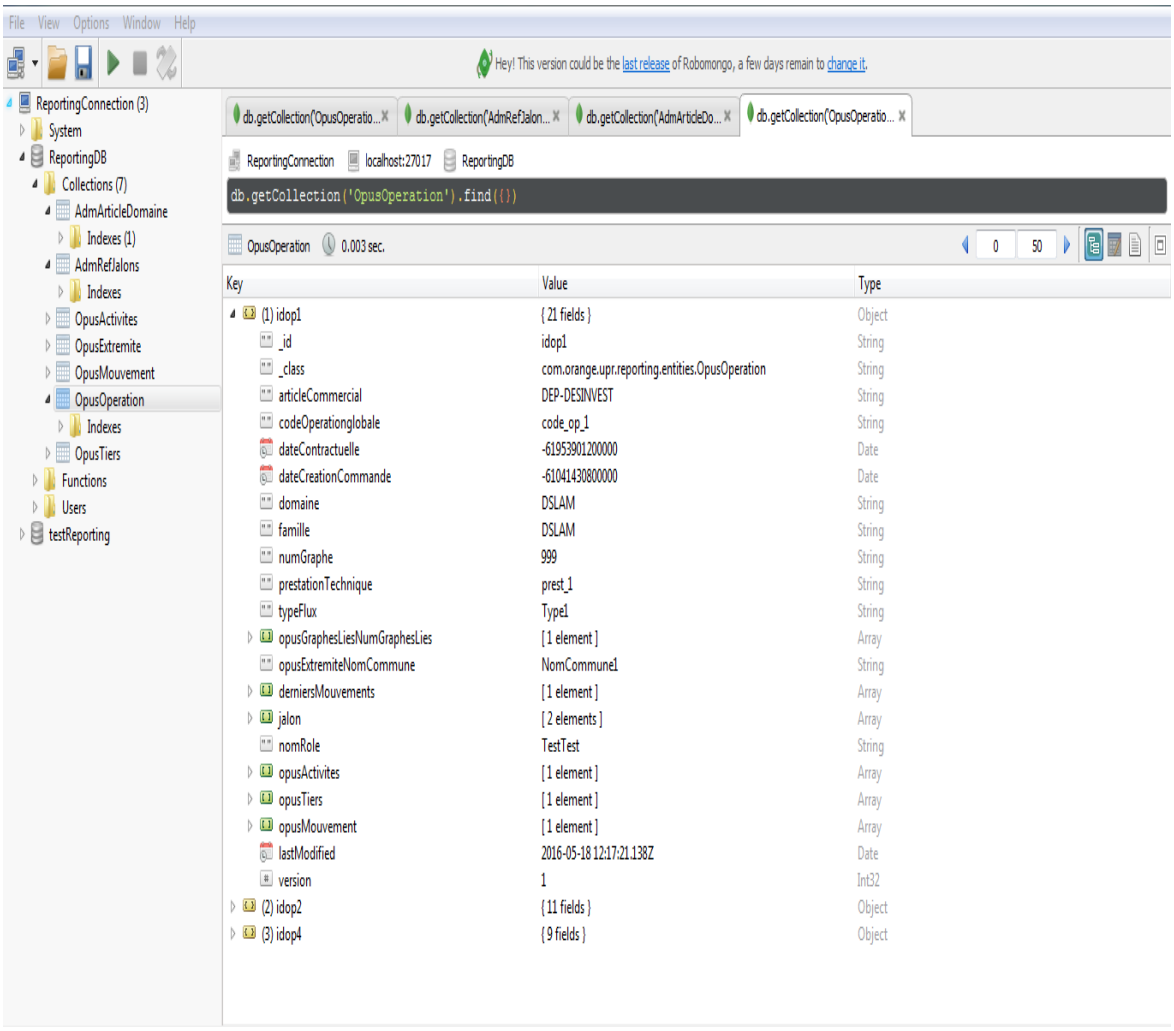


Figure 40 Capture avec Robomongo sur le document OpusOperation

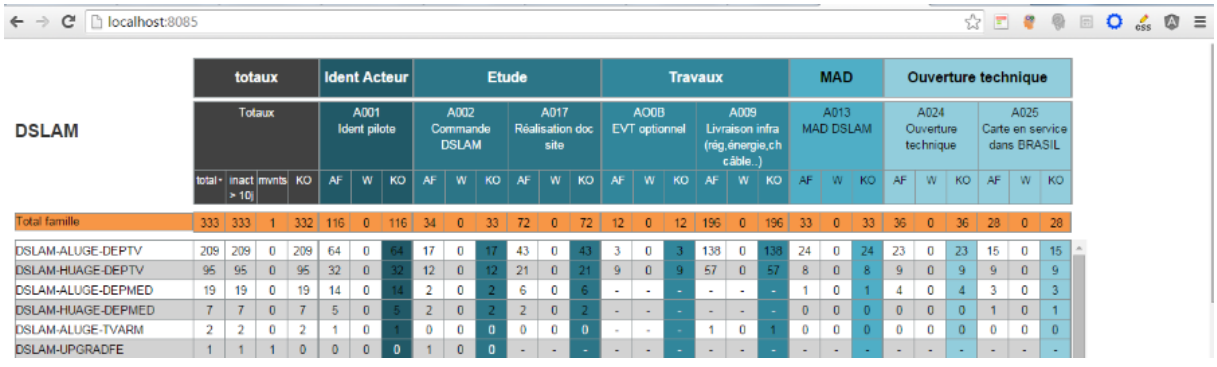
4.2. **Framework d'accès aux données**

Avec l'apparition des bases NOSQL, il est intéressant de suivre l'évolution de l'écosystème et notamment fournir des librairies d'intégration avec ces outils. Ces librairies apportant un

certain niveau d'abstraction émergent, avec l'espoir de voir apparaître des solutions de haut niveau comparables aux ORM que nous utilisons pour les bases relationnelles. Pour simplifier l'utilisation de MongoDB nous avons choisi le projet Spring Data MongoDB, qui propose une certaine unification pour les accès aux bases de données NOSQL. Spring Data MongoDB est une implémentation qui apporte une surcouche au client Java fourni par Mongo et en reproduit l'essentiel des fonctionnalités en s'intégrant au framework Spring. Cette librairie offre grâce à MongoTemplate des mécanismes d'abstraction pour interagir avec MongoDB et pour gérer la persistance des objets Java avec MongoDB de manière transparente. Ce projet permet la possibilité des recherches avec Criteria et un support des requêtes géographiques et une gestion d'index. Par ailleurs, la librairie apporte le support du pattern Repository de manière similaire à Spring Data JPA avec MongoRepository, qui permet de générer des fonctions de CRUD et des requêtes prédéfinies au format JSON.

6. REALISATION DE MICROSERVICE REPORTING

Le tableau de synthèse de la figure 41 présente les opérations OPUS en cours du portefeuille (lignes) réparties par jalons majeurs, le total du portefeuille, le nombre d'opérations inactives et le nombre d'opérations ayant eu un mouvement (colonnes). Pour chaque jalon majeur, les opérations en cours seront répartis selon des seuils d'alerte associés (AF = à faire, W = critique, KO = dépassé). Le comptage est en nombre d'instances de graphe OPUS du portefeuille correspondant aux critères affichés.



DSLAM	totaux				Ident Acteur			Etude						Travaux						MAD			Ouverture technique					
	Totaux				A001 Ident pilote			A002 Commande DSLAM			A017 Réalisation doc site			A00B EVT optionnel			A009 Livraison infra (rég.énergie,câble..)			A013 MAD DSLAM			A024 Ouverture technique			A025 Carte en service dans BRASIL		
	total	inact > 10j	mvnts	KO	AF	W	KO	AF	W	KO	AF	W	KO	AF	W	KO	AF	W	KO	AF	W	KO	AF	W	KO	AF	W	KO
Total famille	333	333	1	332	116	0	116	34	0	33	72	0	72	12	0	12	196	0	196	33	0	33	36	0	36	28	0	28
DSLAM-ALUGE-DEPTV	209	209	0	209	64	0	64	17	0	17	43	0	43	3	0	3	138	0	138	24	0	24	23	0	23	15	0	15
DSLAM-HUAGE-DEPTV	95	95	0	95	32	0	32	12	0	12	21	0	21	9	0	9	57	0	57	8	0	8	9	0	9	9	0	9
DSLAM-ALUGE-DEPMED	19	19	0	19	14	0	14	2	0	2	6	0	6	-	-	-	-	-	-	1	0	1	4	0	4	3	0	3
DSLAM-HUAGE-DEPMED	7	7	0	7	5	0	5	2	0	2	2	0	2	-	-	-	-	-	-	0	0	0	0	0	0	1	0	1
DSLAM-ALUGE-TVARM	2	2	0	2	1	0	1	0	0	0	0	0	0	-	-	-	1	0	1	0	0	0	0	0	0	0	0	0
DSLAM-UPGRADFE	1	1	1	0	0	0	0	1	0	0	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-

Figure 41 Table d'analyse des opérations de déploiement

Le pilote peut accéder à la liste des opérations par critère en cliquant sur les cellules non vides du tableau. Pour chaque instance les données des opérations et les jalons sont présentés sous forme de tableau dans une nouvelle fenêtre comme le montre la figure 42.

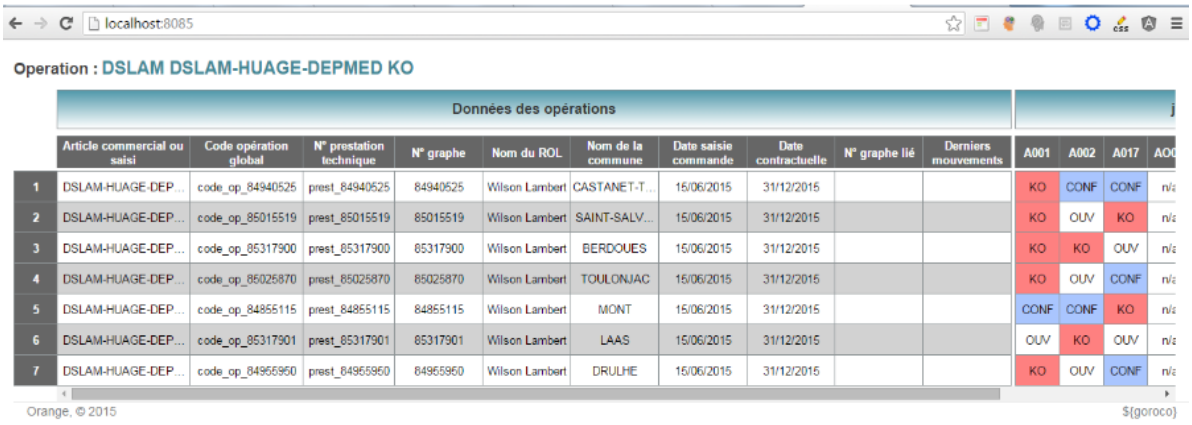


Figure 42 Liste des opérations de déploiement

7. TEST DE MICROSERVICE REPORTING

Pour réaliser les tests unitaires de microservice reporting nous avons utilisé JUnit. La figure 43 présente les tests réalisés. Pour ce microservice nous avons réalisés les cas de tests pour la lecture des données, le comptage des opérations par critères au niveau de la table d'analyse et le comportement et les états des objets de mappage. Cependant les scénarios de tests des opérations d'écriture sur le nouveau modèle de données dénormalisé ne sont pas encore réalisé.

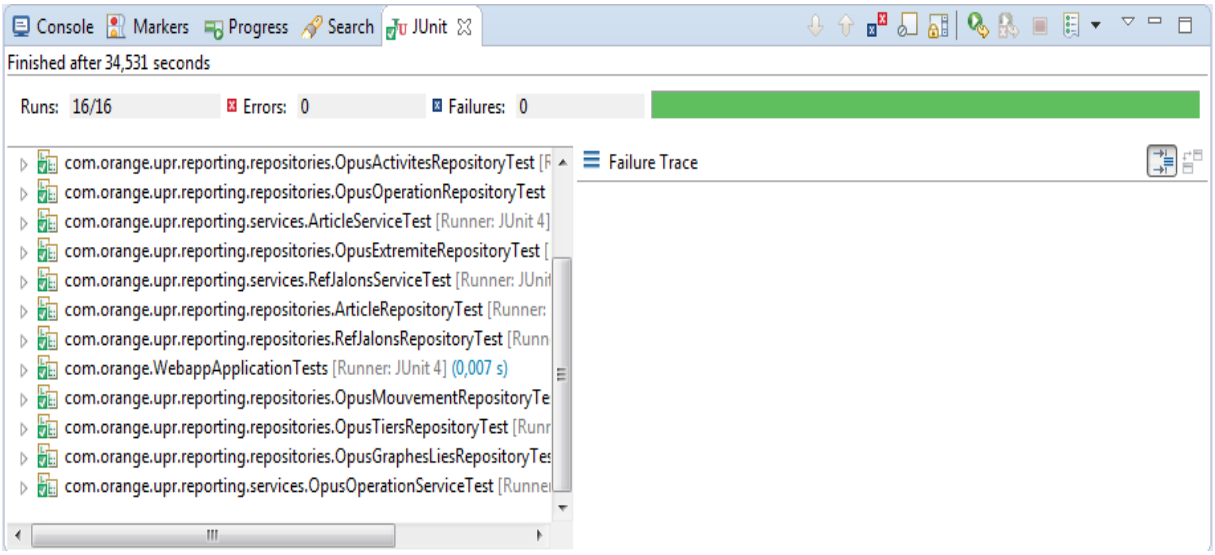


Figure 43 Test unitaire de microservice reporting

CONCLUSION

Ce chapitre a présenté le dernier microservice de portail UPR. Nous avons détaillé les cas d'utilisation réalisés. Puis, nous avons mis l'accent sur le choix de la base de données et la conception. Ensuite, nous avons présenté le livrable testé et validé. La vue globale de projet et l'intégration des microservices vont être détaillé dans le chapitre suivant.

CHAPITRE VII INTEGRATION DU PORTAIL UPR

Sommaire

Introduction 69
1. Intégration des microservices 69
1.1. Service de découverte 70
1.2. Navigation entre les microservices 72
1.2.1. API Gateway 72
1.2.2. Réalisation de l'API Gateway 73
1.2.3. Tolérance aux pannes au niveau de l'API Gateway 75
1.2.4. La mise à l'échelle de microservice reporting 76
1.3. Communication entre les microservices 77
1.3.1. Communication synchrone avec tolérance aux pannes 78
1.3.2. Communication asynchrone 81
2. Monitoring applicatif et des temps de réponses 83
2.1. Traçage distribué : implémentation avec Spring Cloud Sleuth 84
2.2. Supervision statique avec Twitter Zipkin 85
2.3. Supervision dynamique : Indexation des logs avec ELK 89
Conclusion 92

INTRODUCTION

Dans une architecture microservice chaque service est une unité autonome livrable. Cette unité peut être déployée pour répondre parfaitement à des fonctionnalités spécifiques. Par contre, nous avons besoins d'une plateforme cohérente qui répond aux différents besoins de pilotage. Premièrement, ce chapitre présente la phase d'intégration afin de formuler une application composée de microservices qui apparut à l'utilisateur comme un composant unique. Deuxièmement, il décrit la phase de monitoring applicatif et des temps de réponses du portail de pilotage.

1. INTEGRATION DES MICROSERVICES

Pour formuler un Portail cohérent à partir des microservices réalisés des actions d'intégration sont nécessaires. Premièrement, nous avons mis en place un service de découverte. Ce service permet d'enregistrer les microservices afin que nous puissions les appeler sans connaissance explicite de leurs adresses. Deuxièmement, nous avons fourni un point d'entré qui permet aux clients de portail une navigation simple tolérante à l'échec et scalable. Troisièmement, nous avons effectué des communications de nature synchrone et asynchrone entre les microservices afin d'échanger des données entre eux.

La figure 44 résume l'architecture technique du portail UPR en microservices dans sa totalité.

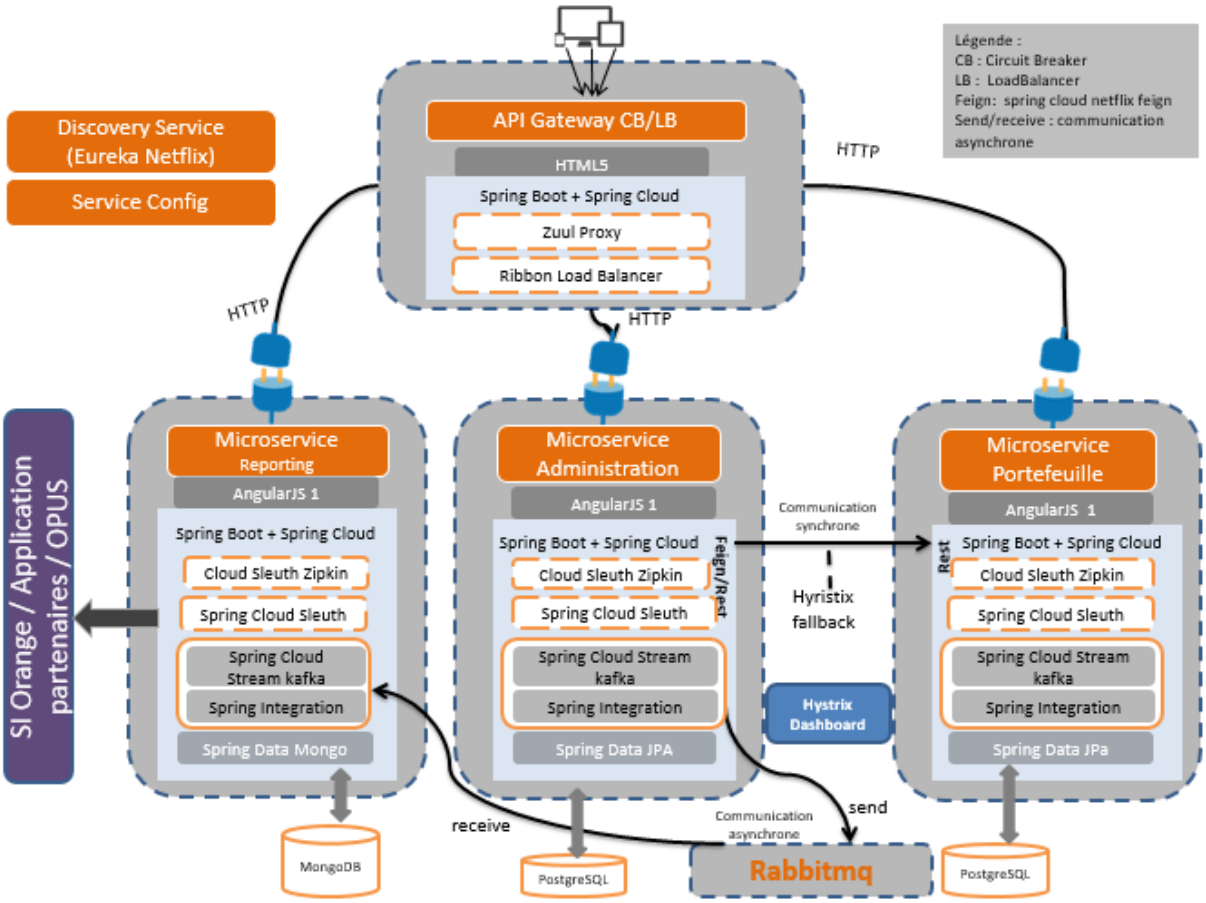


Figure 44 Architecture technique portail UPR microservices

1.1. **Service de découverte**

Dans une architecture microservices les services doivent se trouver et communiquer les uns avec les autres. C'est le service de découverte qui gère la façon d'enregistrement et de localisation des microservices déployés. Nous avons choisi le service de découverte Eureka 2.0 de Netflix qui est un framework de service de découverte conçu pour les déploiements Cloud. Eureka est un système très stable, Il est testé dans de grands déploiements sur le Cloud avec des dizaines de milliers de nœuds et il a gagné la bataille [W6].

La figure 45 montre les principaux composants dans un cadre de déploiement Eureka 2.0. [W7] Ce système se compose d'un cluster d'écriture et un cluster de lecture. Le cluster d'écriture est un sous-système statefull, qui gère les enregistrements des clients et maintient le registre de service interne. Le contenu du registre est répliqué entre tous les nœuds des serveurs d'écriture d'une manière cohérente. Le contenu du registre du pôle d'écriture est lu par le groupe de lecture, qui est finalement utilisé par les clients Eureka.

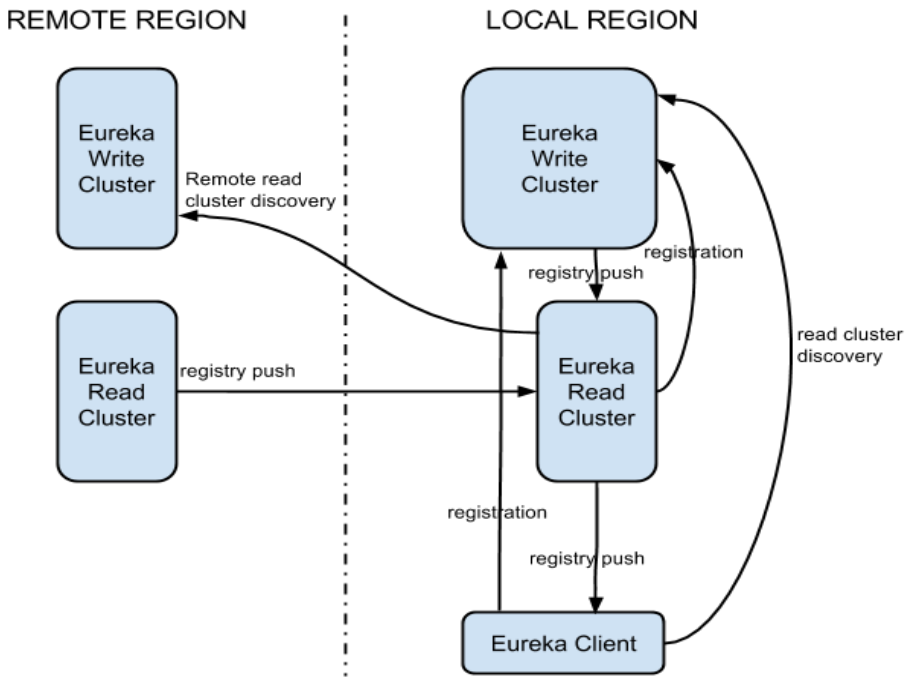


Figure 45 Architecture de service de découverte Eureka 2.0

La figure 46 présente l'enregistrement des microservices Portefeuille, Administration et Reporting dans le serveur de découverte Eureka.

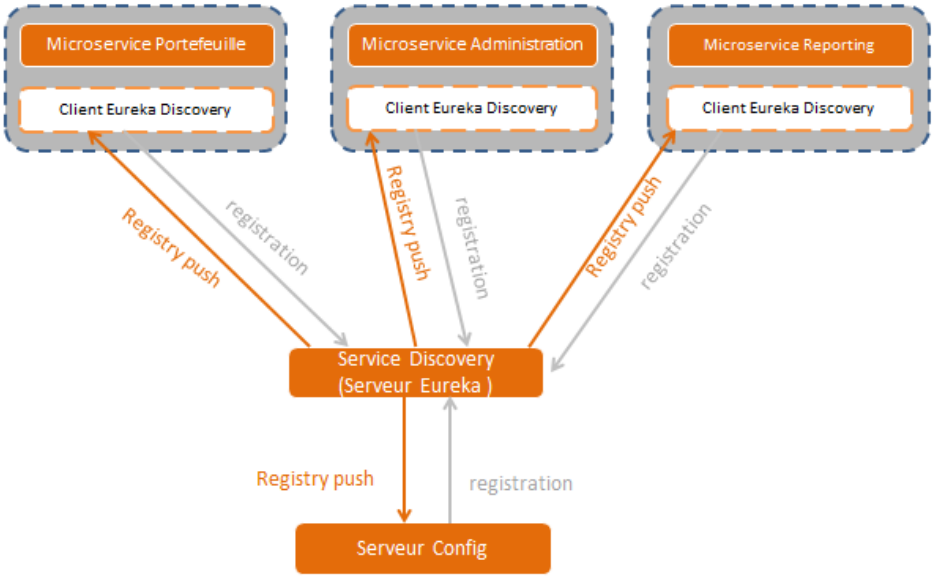


Figure 46 Enregistrement des microservices du Portail UPR dans l'annuaire

Les instances des microservices enregistrés chez le service de découverte Eureka sont présenté par la figure 47. A savoir dans ce cas une instance de microservice Administration, une instance de microservice Portefeuille et deux instances de microservice Reporting.

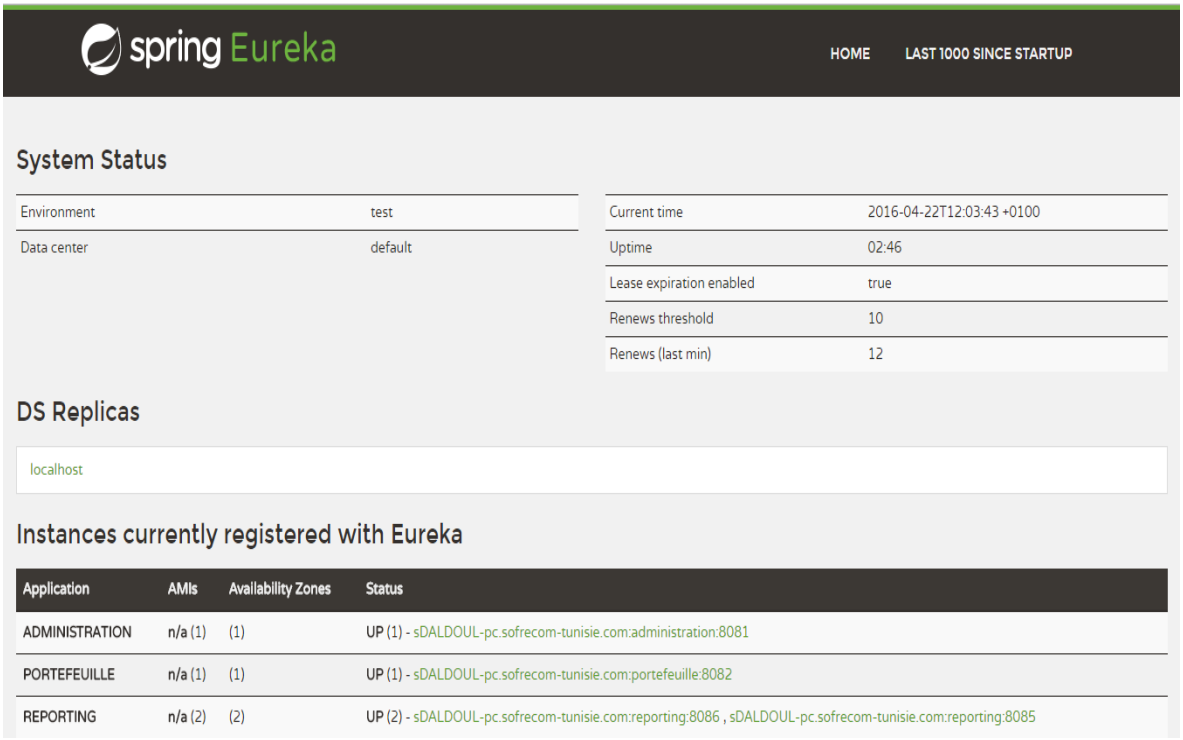


Figure 47 Interface de service de découverte Eureka

1.2. **Navigation entre les microservices**

Une architecture en microservices implique l'utilisation de plusieurs services qui peuvent changer ou modifier d'emplacement. Cette charge ne doit pas apparaitre aux clients. Pour assurer la transparence entre le partitionnement en microservice et la navigation de client nous avons utilisé le patron API Gateway.

1.2.1. **API Gateway**

Comme le montre la figure 48 l'API Gateway est le point d'entrée de l'application, le client ne doit savoir que son adresse et elle se charge de le router vers le microservice qui répond à son besoin en toute transparence. L'API Gateway est loin d'être un orchestrateur elle fournit des façades vers les microservices.

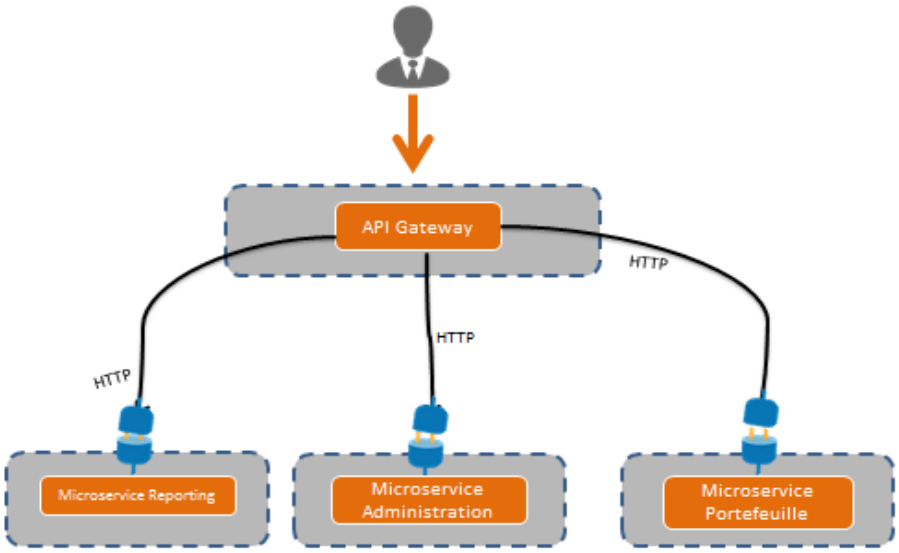


Figure 48 API Gateway pattern sur la portail UPR

Techniquement nous avons appliqué le patron API Gateway à l'aide de proxy Zuul. Nous utilisons Zuul pour présenter une seule url au client et pour lui inviter de naviguer sans arrêt entre différents sous domaines correspondant à notre plateforme. Comme le montre la figure 49 notre passerelle ne contient que le menu de notre application et elle charge dans un iframe html les rendus des microservices.

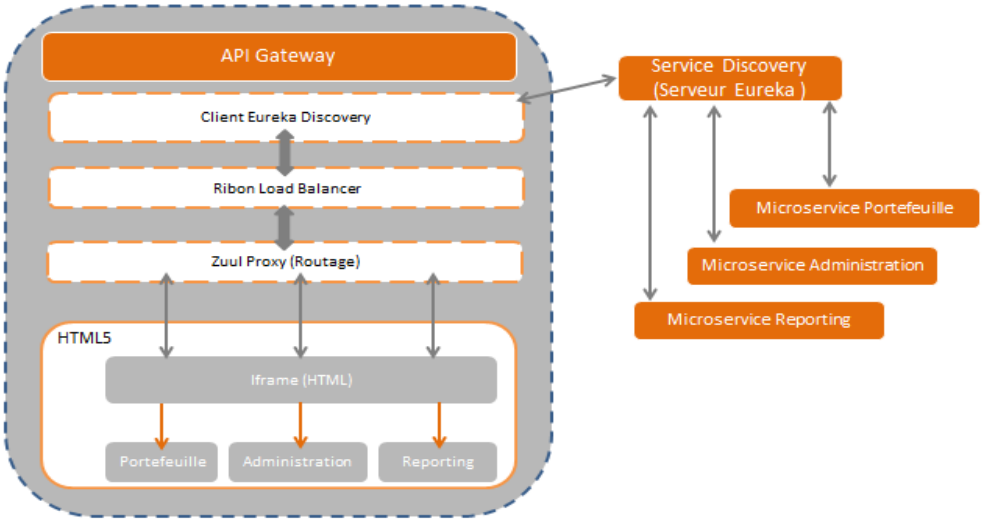


Figure 49 Fonctionnement de l'API Gateway

1.2.2. **Réalisation de l'API Gateway**

L'interface d'administration de microservice administration est accessible de l'API Gateway à travers le menu administration. La figure 50 présente l'intégration de cette interface dans l'API Gateway.

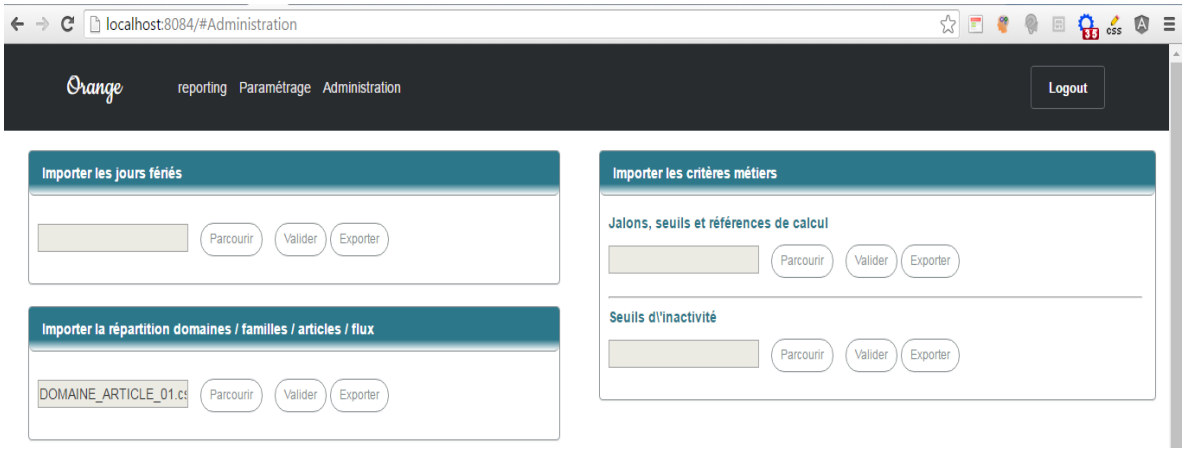


Figure 50 Interface de l'administration à partir de l'API Gateway

Le clic sur le menu Paramétrage charge l'interface de gestion de portefeuille à partir de microservice portefeuille comme le montre la figure 51.

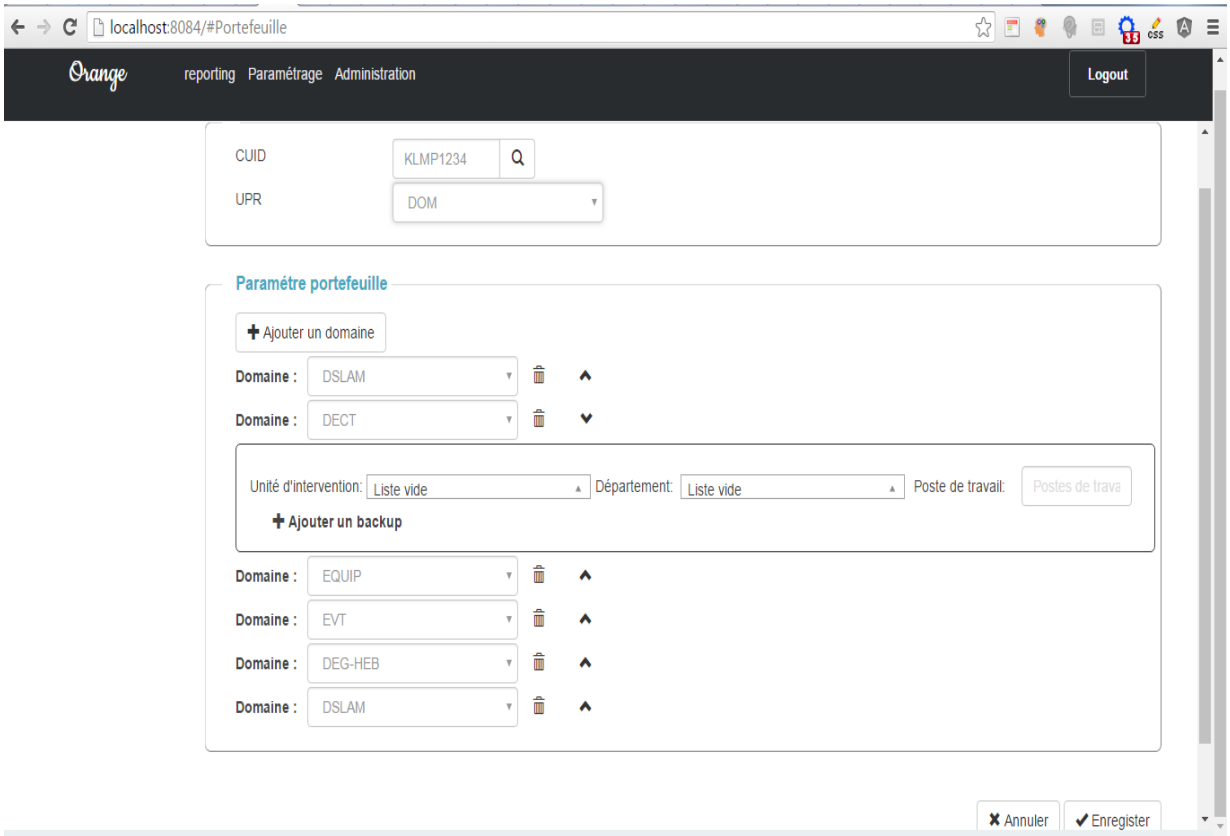


Figure 51 Interface de paramétrage portefeuille à partir de l'API Gateway

La page d'accueil de la plateforme charge la table d'analyse de microservice reporting présenté par la figure 52.

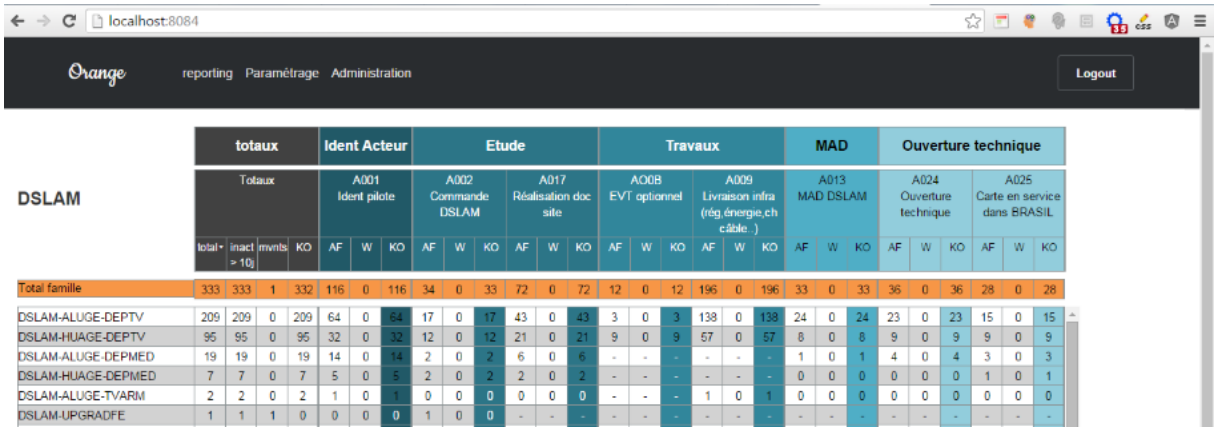


Figure 52 Table d'analyse à partir de l'API Gateway

En cliquant sur une cellule non vide de la table d'analyse la liste des opérations correspondant à la famille, l'article et au seuil qu'il a sélectionné dans le tableau de synthèse apparait comme le présente la figure 53.

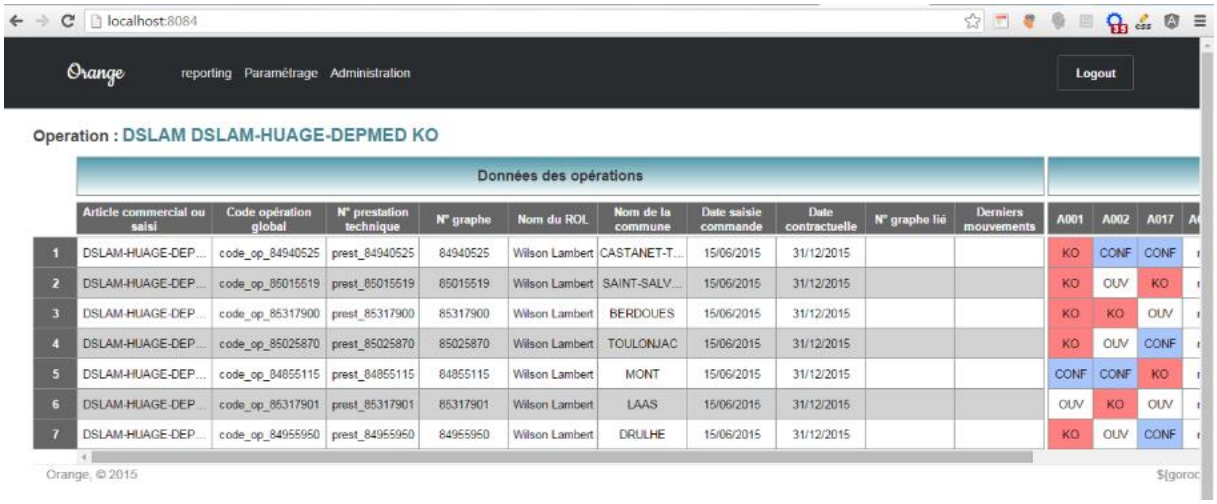


Figure 53 Données des opérations à partir de l'API Gateway

En plus du routage entre microservices via le service d'annuaire, l'API Gateway se charge de deux contraintes non fonctionnelles. Premièrement, elle assure la tolérance aux pannes. Deuxièmement notre API Gateway gère la mise à l'échelle.

1.2.3. **Tolérance aux pannes au niveau de l'API Gateway**

Chaque microservice est dupliqué et si jamais la première instance est en panne l'API Gateway fait appel à la deuxième instance inscrite chez le service de découverte comme le montre le diagramme de séquence de la figure 54 et la figure 55. Pour assurer cette tolérance nous avons appliqué le parton Circuit Breaker à l'aide de la technologie open source Hystrix de la société Netflix.

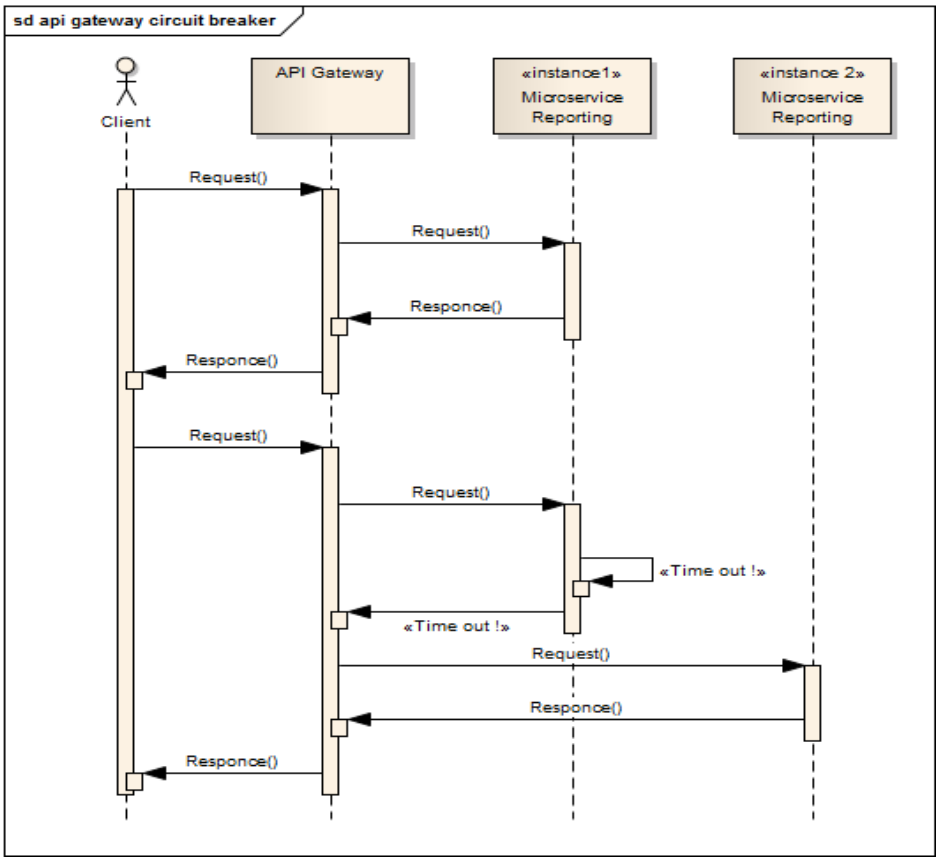


Figure 54 Anticipation de Circuit Breaker au niveau de l'API Gateway

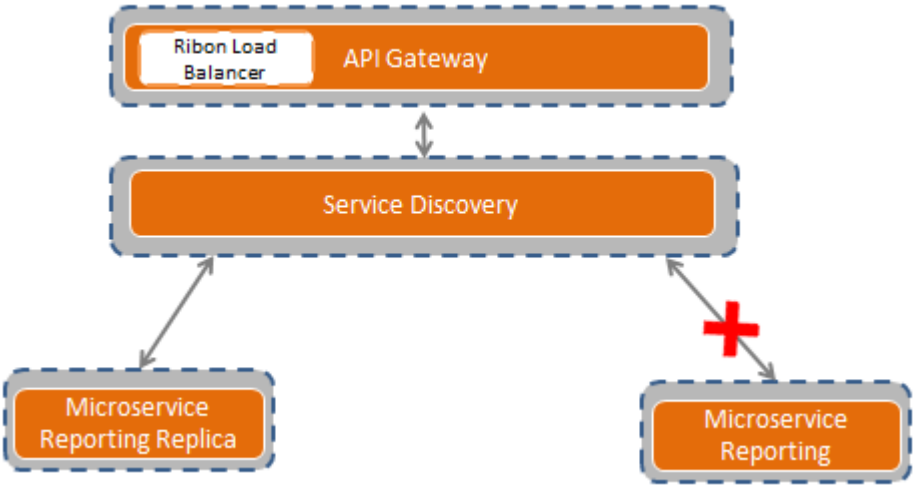


Figure 55 Tolérance aux pannes du microservice reporting

1.2.4. **La mise à l'échelle de microservice reporting**

Le microservice reporting est le noyau du portail UPR. Il est le responsable de l'analyse des données d'opérations de déploiement. Nous avons choisi ce microservice pour être mis à

l'échelle. Nous avons utilisé « *Ribbon Load Balancer* » un service de Netflix pour répartir la charge entre plusieurs instances de microservice reporting comme le montre la figure 56.

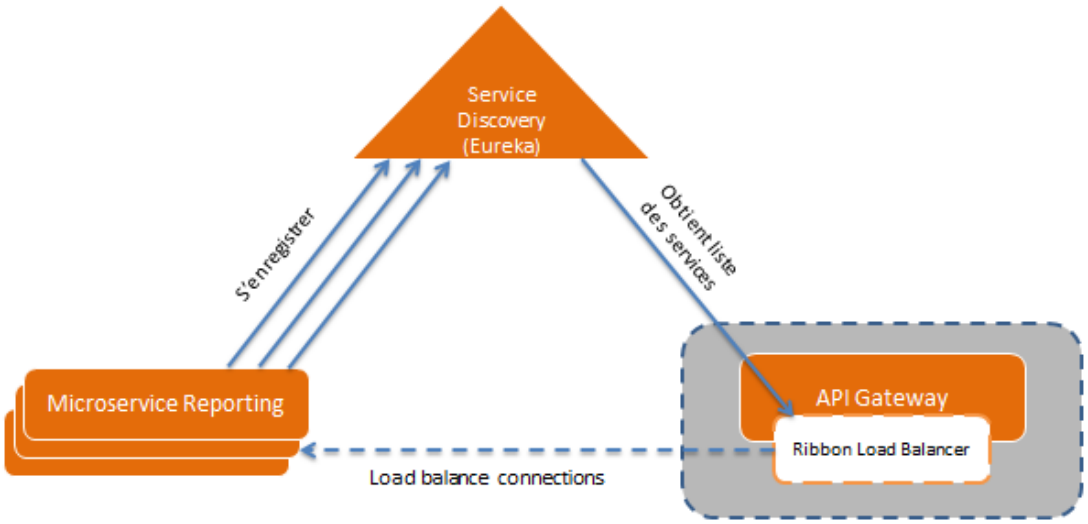


Figure 56 Load balancing avec Netflix Ribbon

Ribbon fournit des équilibreurs qui répartissent la charge entre les différents services inscrits chez Eureka selon une certaine logique. Nous pouvons choisir la règle à appliquer pour équilibrer la charge (selon Le temps d'arrivée d'un Microservice dans l'annuaire « *Round Robin* », selon le temps de réponse le plus court d'un Microservice « *Weighted Response Time* », selon le nombre de connexion « *Availibility Filtering* »…) et dispatcher les requêtes pour améliorer le temps de réponse d'un MS en cas de montée en charges.

Certains équilibreurs de charge peuvent également fournir des fonctionnalités avancées. Nous citons par exemple :

- La division en zones (pour favoriser les serveurs de la même zone)
- Les statistiques des serveurs (pour éviter les serveurs avec une latence élevée ou de pannes fréquentes)
- Les statistiques des zones (pour éviter les zones qui pourraient être en panne)

1.3. **Communication entre les microservices**

Les microservices sont autonomes. Mais, ils ont besoins d'échanger des données pour réaliser quelques fonctionnalités. Dans une architecture en microservices chaque microservice gère ses propres données et ne les partage pas directement avec d'autres services. C'est pourquoi nous avons réalisé des intercommunications synchrones via des protocoles ouverts et asynchrones via un service de file d'attente de messages.

1.3.1. **Communication synchrone avec tolérance aux pannes**

Pour la gestion des portefeuilles utilisateurs le microservice portefeuille a besoin des données concernant les domaines à partir du microservice administration. Ce besoin est traduit en une communication synchrone à travers des appels clients Rest.

Spring Cloud offre trois façons d'appels clients Rest :

- **Une façon traditionnelle en utilisant Rest Template.**

Le problème avec cette méthode est que le microservice client de l'API doit connaitre l'adresse complète de microservice producteur.

- **Rest Template avec Ribbon et Eureka.**

Netflix ruban fournit une bibliothèque pour faire des appels REST qui résout le problème de connaissance d'hôte en se basant sur le service d'annuaire Eureka. Le problème avec cette solution et que la configuration reste complexe.

- **Une façon simplifiée avec Netflix Feign et Eureka**

Cette solution offre un rendu de performance pareil qu'avec ribbon avec une interface simple et des annotations paramétrées.

Nous avons choisi de consommer l'API REST exposé par le microservice administration par un Client Netflix Feign au niveau de microservice portefeuille comme le montre la figure 57.

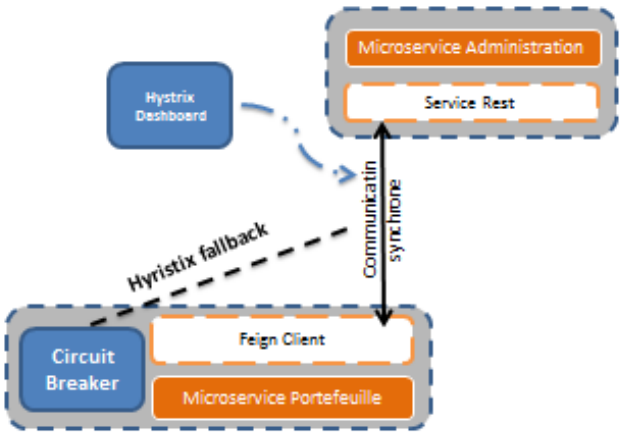


Figure 57 Communication synchrone entre les microservices administration et portefeuille

En admettant cette solution le micoservice portefeuille ne doit connaitre que le nom d'inscription de service administration chez l'annuaire Eureka pour communiquer avec lui et l'API qu'il veut consommer. Dans ce cas le microservice administration change d'adresse sans aucun impact sur le microservice portefeuille.

Une des grandes différences entre les appels en mémoire et les appels à distance est que les appels distants peuvent échouer, ou attendre sans réponse jusqu'à ce qu'un délai d'expiration soit atteigner. Pour assurer une tolérance à cet échec nous avons appliqué le parton circuit breaker à l'aide de la technologie open source hystrix de la société Netflix.

L'idée de base derrière l'utilisation de circuit breaker à ce niveau est très simple. Premièrement, nous avons plusieurs instances de microservices inscrits dans le service de découverte Eureka. Si la première instance de microservice administration est en panne le circuit breaker dirige le microservice portefeuille vers une autre instance. Deuxièmement, dans le cas où toutes les instances de microservices administration sont en pannes, nous avons préparé une méthode locale qui offre des résultats par défaut. L'appel de cette méthode est enveloppé dans le circuit breaker réalisé avec Hystrix, qui surveille les défaillances. Une fois que les échecs de l'appel de la méthode de microservice administration atteignent un certain seuil nous retournons les résultats de la méthode locale.

La figure 58 présente les états du patron disjoncteur. Si le microservice administrateur répond parfaitement à l'appel du microservice portefeuille notre disjoncteur est dans un état « Open ». Lorsque le seuil du temps est dépassé sont l'obtention d'une réponse l'état du circuit breaker est « Half Open ». Maintenant si le microservice administration est échoué l'état est « Closed »

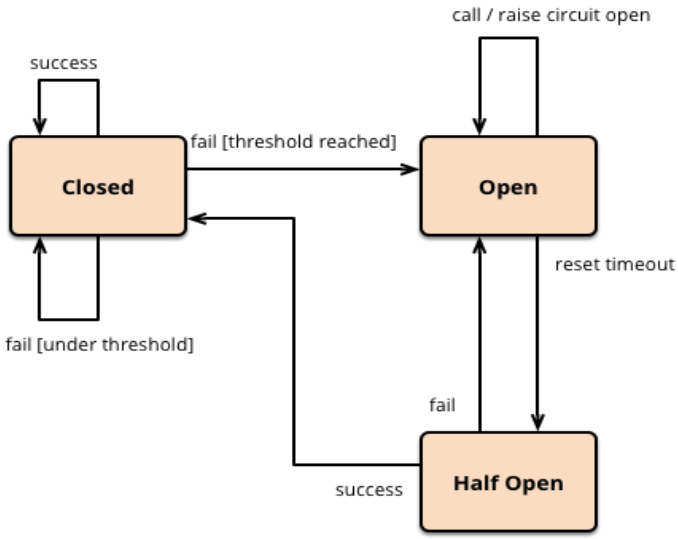


Figure 58 Patron circuit breaker

La figure 59 est capturée du dashbord hystrix lorsque le microservice administration est en exécution. Elle présente des appels distants dans un état « Open » de circuit breaker. Dans ce cas les appels distants sont réussis.

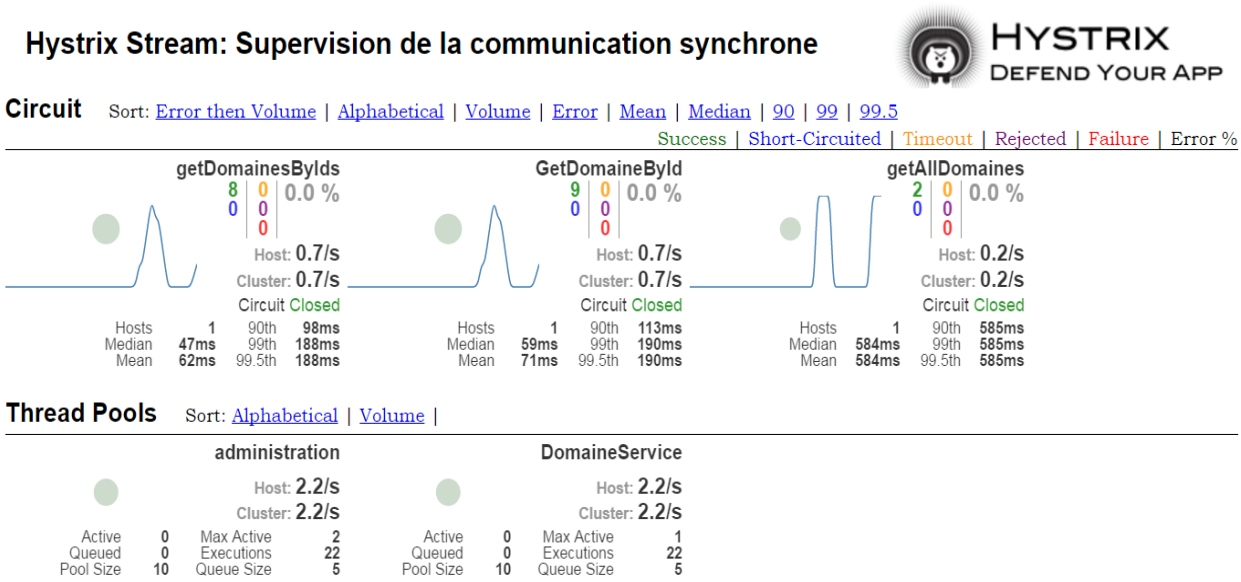


Figure 59 Hystrix Dashboard quand le microservice administrateur est lancé

La figure 60 est prise lorsque le microservice administration n'est pas en exécution. C'est-à-dire dans un état « Closed » du circuit breaker. Le microservice portefeuille se limite sur les résultats des méthodes locales de secours.

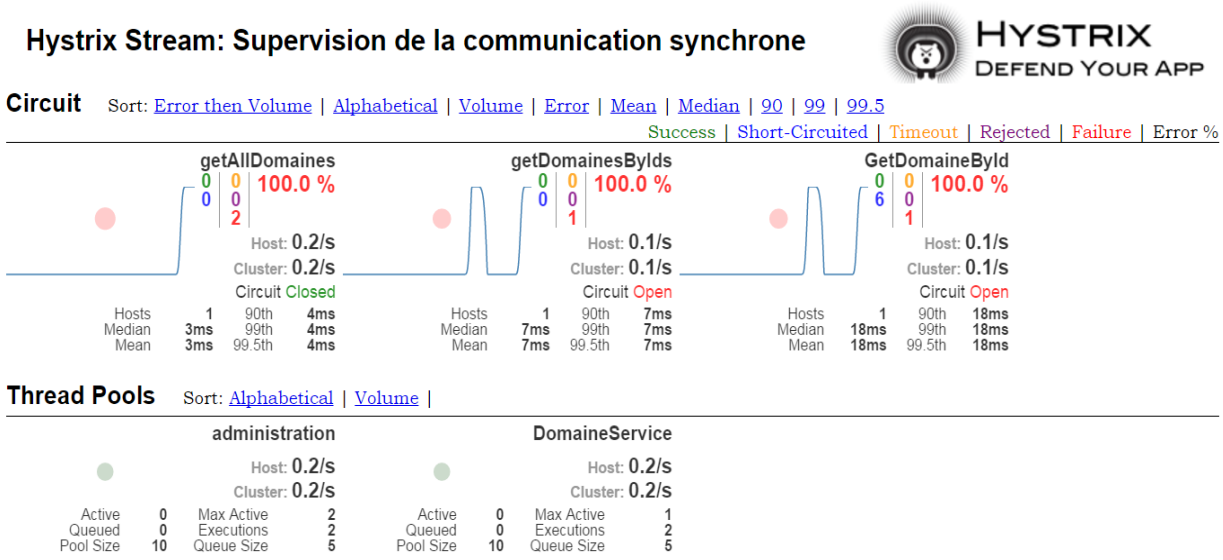


Figure 60 Hystrix Dashboard quand le microservice administrateur a échoué

Ces dashboard en temps réel offrent à notre application un monitoring d'intégration.

1.3.2. **Communication asynchrone**

Le microservice Reporting nécessite des données métiers à partir du microservice administration pour l'affichage des tables d'analyse et des résultats de reporting. En raison des exigences de temps de réponse pour ce microservice nous avons favorisé les communications asynchrones.

Afin de choisir le service de file d'attente de message le plus adéquat nous avons réalisé un ensemble de simulations pour tester la performance des services « rabbiMq », « activeMq » et « Apache kafka ». Les résultats des simulations présenté dans l'annexe B sont synthétisés par le tableau 15.

Tableau 15 Synthèse RabbitMq Vs ActiveMq Vs Apache kafka

	RabbitMq	**ActiveMq**	**Apache kafka**
Envoi de messages (100 000)	8sec	45min59	3sec
Réception des messages (100 000)	5sec	48s	2sec
Taux d'envoi	25000/s	60/s	55000/s
Taux de consommation	30000/s	8000/s	65000/s
Mémoire utilisé pour l'envoi	40Mb/s	120Mb/s	20Mb/s
Mémoire utilisé pour l'édition	25Mb/s	120Mb/s	-
Latence	[0 -16ms]	0	-

Ces simulations montrent la performance de rabbitMq par rapport à activeMq et la performance d'Apache Kafka par rapport à rabbitMq. Cependant, l'utilisation de système de messagerie distribué Apache Kafka nécessite une utilisation de service de découverte d'Apache « Zookeeper ». Ce service permet aux instances kafka de communiquer entre eux. Ce choix ajoute une complexité non justifié au projet surtout que nous utilisons déjà le service de découverte Eureka. Par conséquent, nous avons utilisé le service de file d'attente RabbitMq qui répond suffisamment aux besoins.

Par contre nos choix technologiques nous permettent de modifier le système de messagerie à tout moment avec des coûts presque nuls. A savoir la préparation des messages se fait à travers

Spring Cloud Stream et l'envoi et la réception des messages à travers le framework Spring Integration. Par conséquent, le portail UPR est extensible, un passage de service de file d'attente « RabbitMq » au système de messagerie distribué « kafka » en cas de besoin est pris en considération, il suffit d'installer les services zookeeper et kafka puis modifier la dépendance Maven dans le fichier « pom.xml » de « spring-cloud-starter-stream-rabbit» à « spring-cloud-starter-stream-kafka».

La figure 61 illustre l'envoi des données par le micrsoservice administration au service de file d'attente et la lecture de ses données par le microservice reporting à partir de la queue partagée.

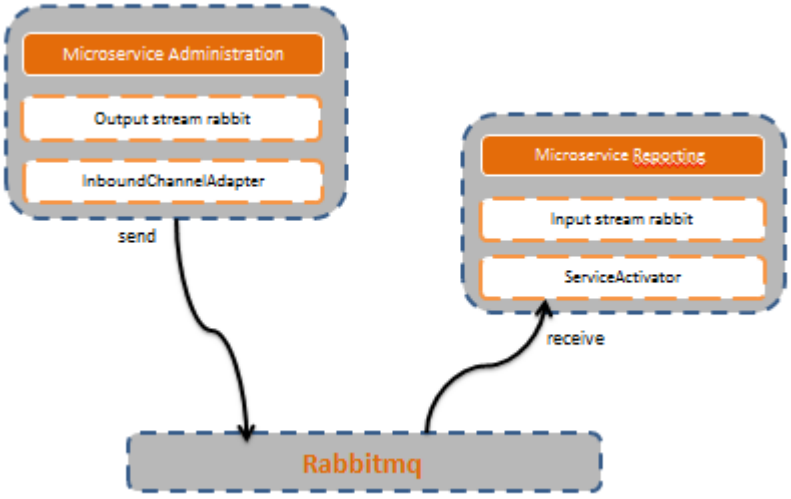


Figure 61 Communication synchrone avec rabbitMq

La figure 62 présente la liste de queues utilisées par le Portail UPR dans le service de file d'attente rabbitmq.

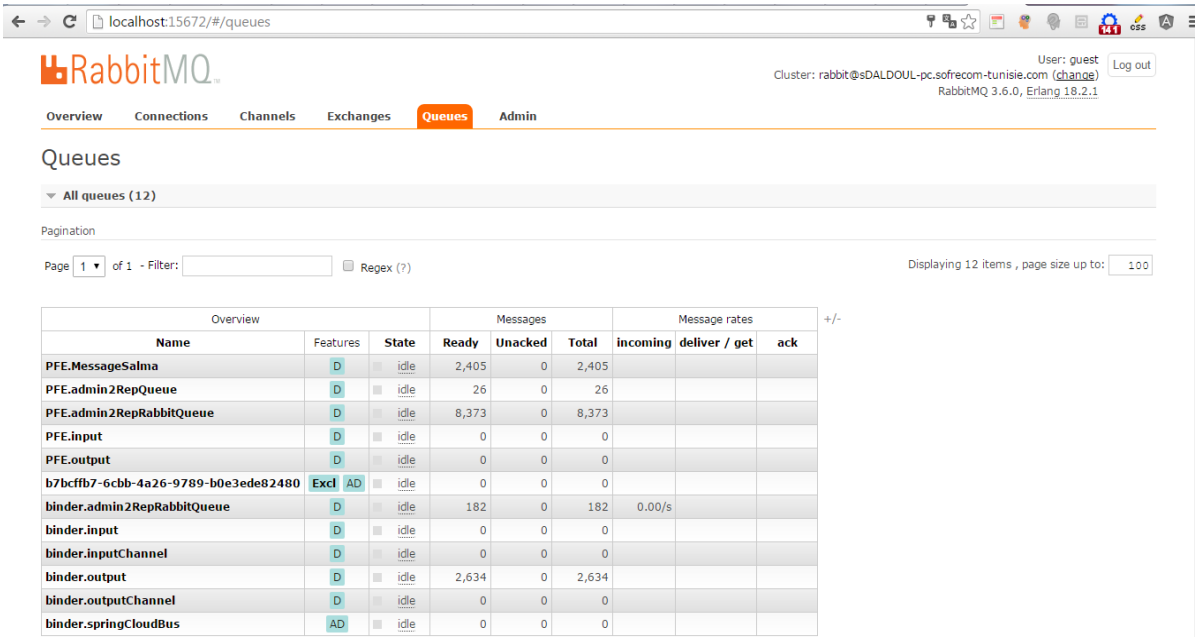


Figure 62 Liste des queues

2. MONITORING APPLICATIF ET DES TEMPS DE REPONSES

La multiplicité des services entraine une complexification de la supervision du système. En fait, une transaction peut passer par plusieurs microservices à travers des communications. Que la communication entre services soit synchrone ou asynchrone, la supervision doit être automatisée au maximum. En effet, le nombre de services peut croitre régulièrement au fur et à mesure de la vie du système. Il devient vite impossible pour un humain de suivre tous les services manuellement. La supervision devient un élément central pour une application en microservices.

L'architecture microservice met en avant deux problèmes critiques pour la supervision et l'exploitation des logs.

- La centralisation des logs

Un problème lié à la multitude de services est l'exploitation des fichiers de logs. Pour une application unique déployée sur plusieurs serveurs, ce problème peut s'avérer être fastidieux. Une bonne solution consiste à centraliser les logs des services au même endroit.

- La corrélation des Id

La centralisation des logs n'est pas suffisante dans le cadre d'une architecture microservices. En fait, une application en microservices est unité construite par différentes briques. Ces briques communiquent pour répondre aux besoins utilisateurs. Avec des logs agrégés sans corrélations le monitoring reste limité et la vue d'ensemble est vague.

La corrélation des id des logs consiste à hériter au microservice l'id de trace de son communicateur. Cet id passe à travers les appels distants de service. Cette corrélation nous permet de voir toutes les interactions entre les microservices pour répondre aux besoins.

Nous implémentons les corrélations des Id entre les traces des microservices avec Spring Cloud Sleuth. Ces traces sont les points d'entrés aux deux outils de supervision Twitter Zipkin et ELK. Entre autre avec Twitter Zipkin nous offrons des interfaces statiques pour la supervision des temps de réponse. Et avec ELK nous permettons au superviseur de l'application de créer dynamiquement des graphes et des dashboard à son choix partir des traces agrégés des microservices.

2.1. **Traçage distribué : implémentation avec Spring Cloud Sleuth**

Spring Cloud Sleuth est une bibliothèque qui implémente une solution de traçage distribué pour Spring Cloud compatible avec Zipkin, Htrace, ELK… En fait, il nous aide à mieux comprendre le comportement spécifié et réel d'un système.

Spring cloud sleuth empreinte la terminologie de Google Dapper :

Span: L'unité de base de traçage. Les spans sont identifiés par un ID unique de 64 bits et ont également d'autres données, telles que les descriptions, les événements temporisé, les tags, les prosses ID et l'ID de span en cause.

Trace: Un ensemble de spans formant une structure arborescente.

Annotations: sont utilisées pour enregistrer l'existence d'un événement dans le temps. Certaines des annotations de base utilisée pour définir le démarrage et l'arrêt d'une demande sont:

- Client Sent (CS) : Le client a fait une demande. Cette annotation représente le début de span.
- Server Received (SR) : SR reçoit la demande et commence son traitement. La différence entre les timestamps de SR et CS présente la latence du réseau.
- Server Sent (SS) : A la fin de traitement le SS envoi la réponse vers le client. La différence entre les timestamps de SS et SR présente le temps de traitement de la demande.
- Client Received (CR) : Signifie la fin de span. Le client a bien reçu la réponse du serveur. La différence entre les timestamps de CR et SS présente le temps nécessaire pour que le client reçoive la réponse.

Ces annotations sont illustrées dans la figure 69 par le popup de twitter zipkin.

La figure 63 présente le cycle de vie d'un span.

Lorsque qu'un span commence « start » son nom est attribué et le *Timestamp* est enregistré. Dans l'état «detach » le span n'est pas arrêté ou fermé. Il est seulement retiré du thread courant. Le span peut continuer avec une nouvelle instance « continue ». À la fermeture « close » le temps de la durée de span est enregistré et encore le span est retiré du thread courant. Si le traitement suivant dépend de celui-ci un nouveau span est créé avec un parent explicite qui est le span courant.

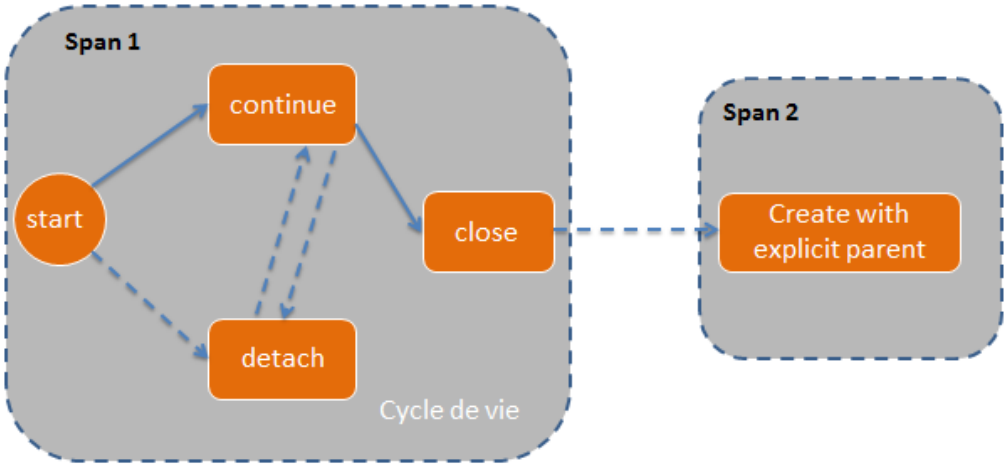


Figure 63 Cycle de vie de span

Pour chaque microservice les logs des appels distants implémentés avec Spring Cloud Sleuth sont envoyés aux collecteurs de zipkin pour une supervision statique et redirigés vers un fichier log en utilisant la bibliothèque « Groovy ». Ces fichiers sont utiles pour une indexation plus détaillés et dynamique avec ELK.

2.2. **Supervision statique avec Twitter Zipkin**

Zipkin est un système de visualisation de traçage distribué. Il aide à recueillir des données de synchronisation nécessaires pour résoudre les problèmes de latence dans les architectures de Microservice. Il gère à la fois la collecte et recherche de ces données. La conception de Zipkin est basée sur le papier Google Dapper. La figure 64 présente l'architecture de système de traçage zipkin.

Dès que les données de trace arrivent au niveau du collecteur Zipkin, il est validé, stocké, et indexé pour les recherches. Le stockage des données se fait par défaut dans une base de données scalable cassandra. Une fois que les données sont stockées et indexées, nous avons

besoin d'un moyen pour l'extraire. Le démon query fournit une API JSON simple pour trouver et récupérer des traces. Le principal consommateur de cette API est l'interface utilisateur Web qui fournit une méthode de visualisation des traces sur la base de services, de temps, et des annotations.

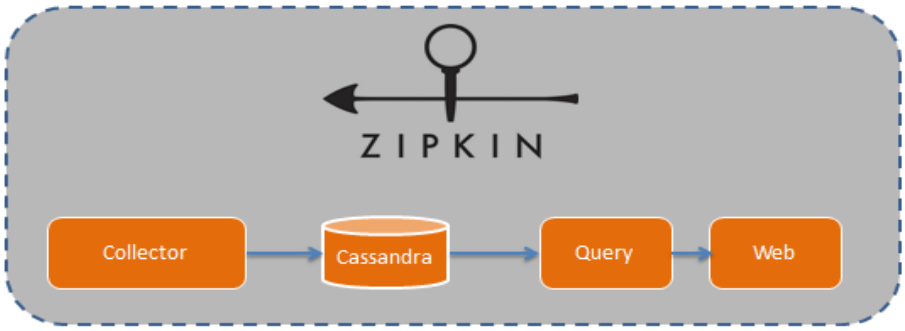


Figure 64 Architecture twitter zipkin

Plus que la centralisation des logs Zipkin résout le problème de corrélation id. En effet, les informations de suivi sont recueillies sur chaque microservice en utilisant la bibliothèque instrumenté spring cloud sleuth zipkin puis envoyé à Zipkin Collector comme le montre la figure 65. Lorsqu'un microservice fait une demande à une autre, il passe quelques identificateurs de traçage ainsi qu'une demande à Zipkin afin que ce système de traçage puisse plus tard attacher les données en corrélation.

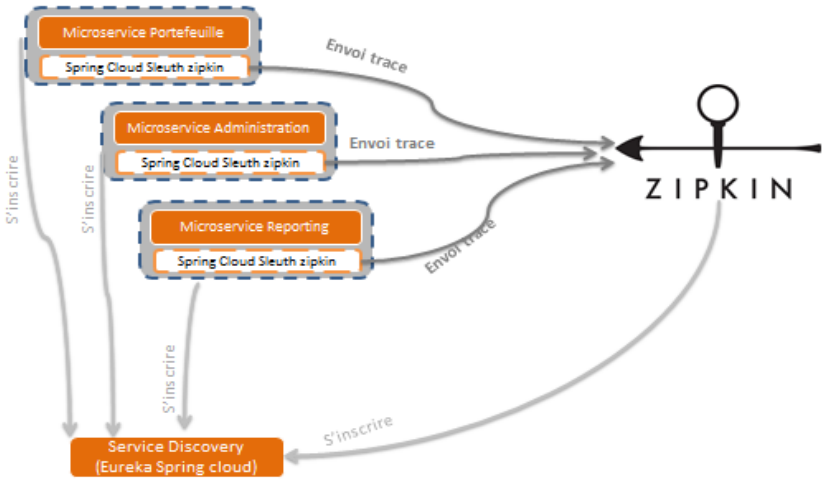


Figure 65 Supervision distribué avec twitter zipkin

La figure 66 présente la dépendance entre les deux microservices administration et portefeuille à travers Twitter Zipkin. Cette liaison est traduite à partir du correlation id.

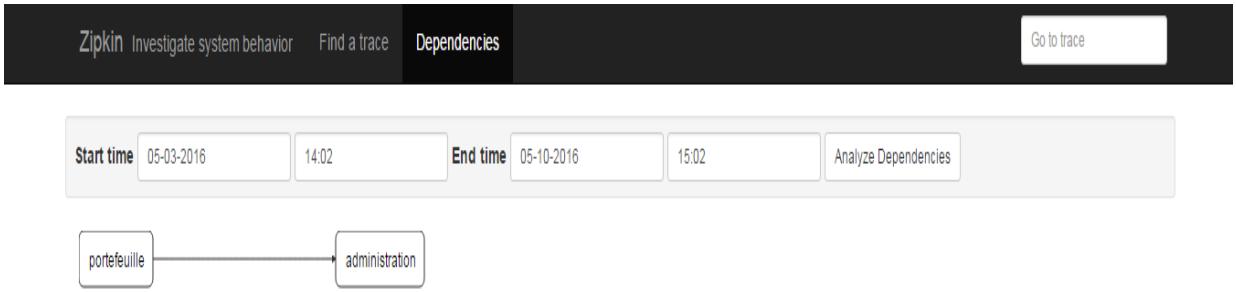


Figure 66 Dépendance entre les microservices portefeuille et administration

La figure 67 montre les traces du microservice portefeuille y compris les appels distants.

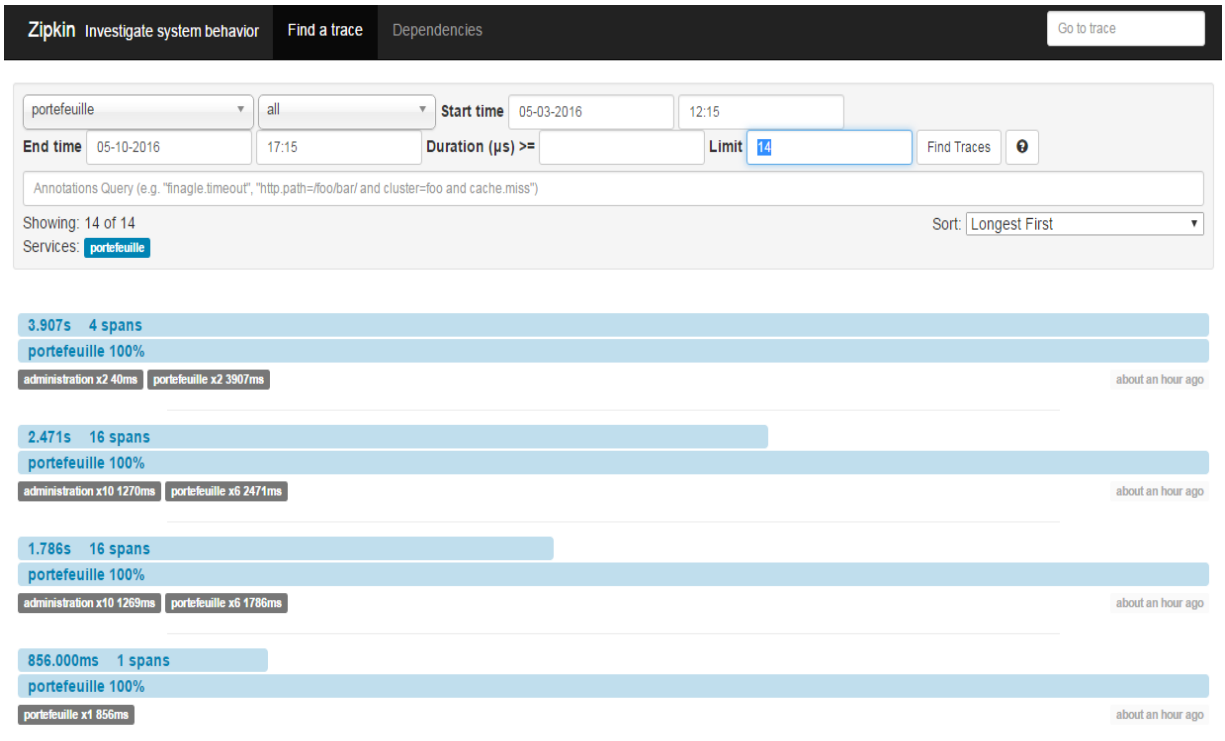


Figure 67 Supervision des traces dans zipkin

Lors de clic sur l'une des traces les détails apparaissent. Ce span présent dans la figure 68 correspond à un appel d'un API Rest du microservice administration par le microservice portefeuille.

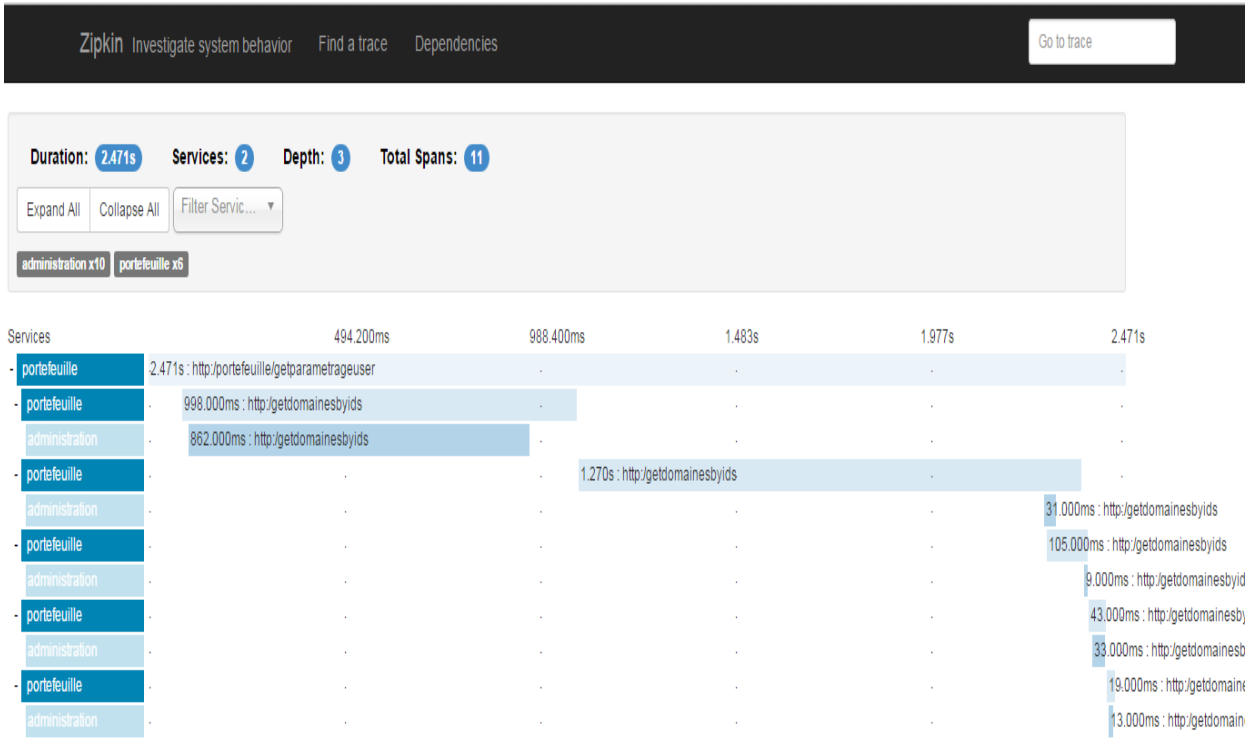


Figure 68 Trace d'une communication asynchrone

Le popup de la figure 69 détaille le temps relatifs pour chaque phase de la communication.

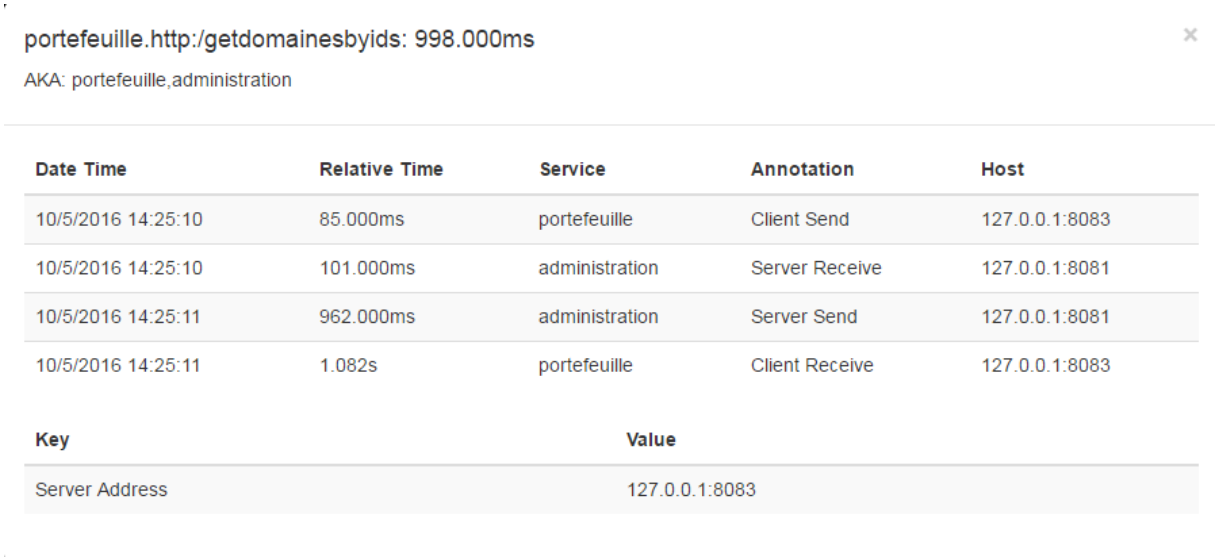


Figure 69 Appel distant de rest api

Les figures 70 et 71 montrent les traces avec Zipkin dans le cadre d'envoi de message par le microservice administration au microservice reporting à travers rabbitmq.

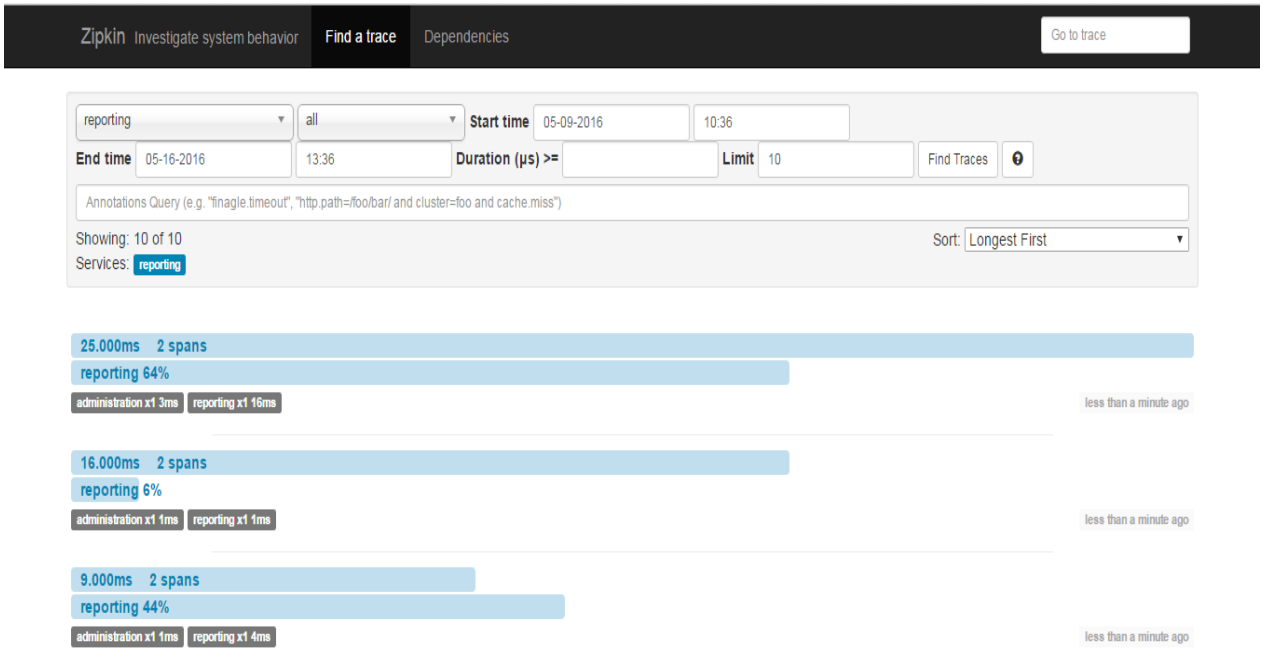


Figure 70 Trace des envois de message avec rabbitmq

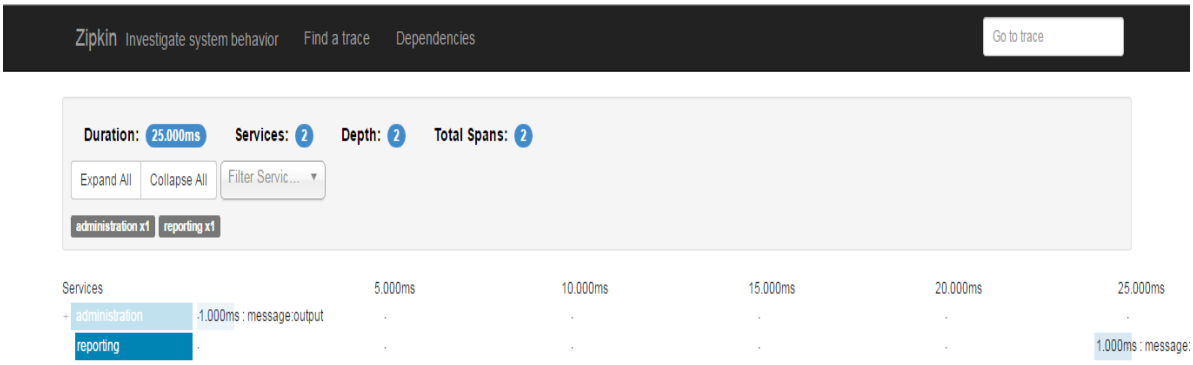


Figure 71 Détails sur une communication asynchrone entre administration et reporting

2.3. **Supervision dynamique : Indexation des logs avec ELK**

Pour un monitoring plus évolué et pour identifier rapidement tout comportement anormal du portail UPR en microservices nous utilisons la pile de logiciels ELK composée d'Elasticsearch, Logstash et Kibana.

Elasticsearch:

Un service de base de données qui est :

- Optimisé pour la recherche en temps réel (near-real time). Dès l'ajout des données, elles sont disponibles. Cela le rend efficace dans un contexte de supervision.
- Optimisé pour l'indexation des logs (utilise text Search engine pour faire l'indexation)

- Extensible
- Interopérable (accès via une API RESTful)

Logstash:

Un pipe permettant de collecter, parser et stocker des logs à l'aide d'entrées, de filtres et de sorties définis dans un fichier « logstash.conf ». La figure 72 présente la structure de ce fichier.

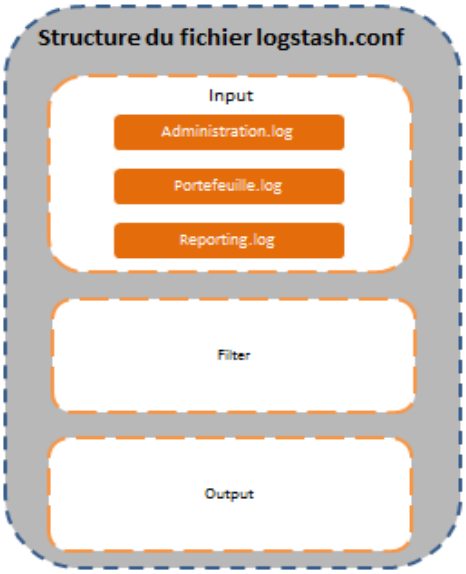


Figure 72 Structure du fichier logstash.conf

Kibana:

Une plate-forme d'analyse et de visualisation conçue pour fonctionner avec des données en temps réel. Elle fonctionne d'une façon transparente avec ElasticSearch. Elle permet de :

- Mettre en place des graphiques.
- Effectuer du requêtage spécifique.
- Mettre en œuvre des interfaces graphiques (la mise en place de tableaux de bord s'effectue par une interface web permettant de choisir l'affichage désiré)

La figure 73 illustre l'agrégation et l'indexation des logs des microservices avec le trio ELK.

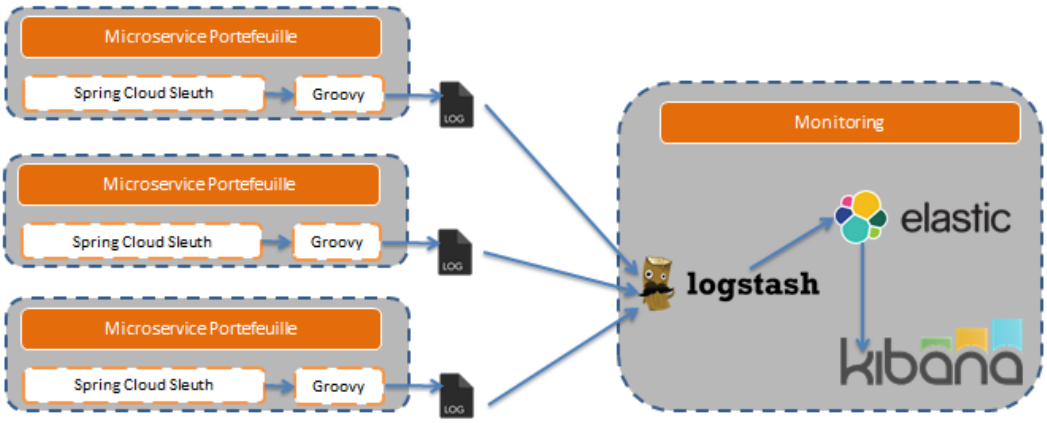


Figure 73 Indexation des log avec ELK

Le menu « Visualize » permet aux superviseurs d'ajouté des graphes en temps réel à partir des logs enregistrés dans elasticsearch. Chaque superviseur peut ajouter graphiquement des filtres à son choix, enregistrer ses graphes et les partager. La figure 74 donne un exemple de graphe réalisé avec kibana qui permet de visualiser le nombre d'appels distants des méthodes « getAllDomaines » et « getDomainesByIds » en fonction du temps. De sorte que nous suivons les communications synchrones entre les microservices administration et portefeuille.

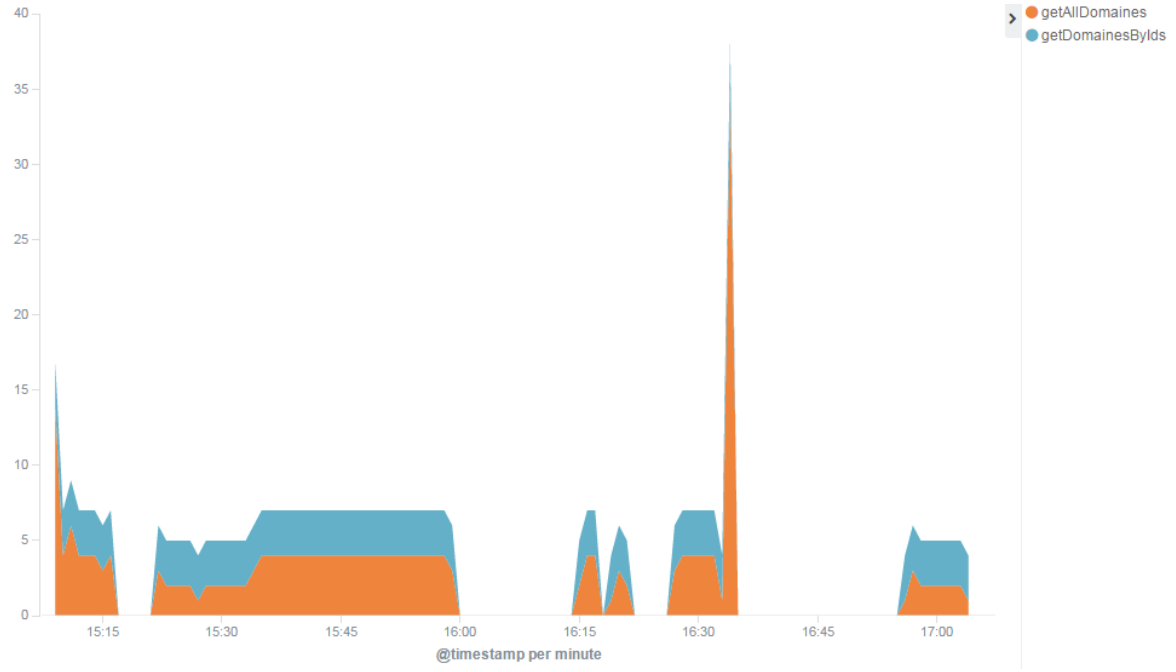


Figure 74 Exemple de graphe avec kibana

Le menu « Dashbord » permet aux superviseurs de formuler à partir des graphes réalisés des Dashboards pour simplifier la supervision quotidienne. La figure 75 montre un dashbord réalisé avec kibana.

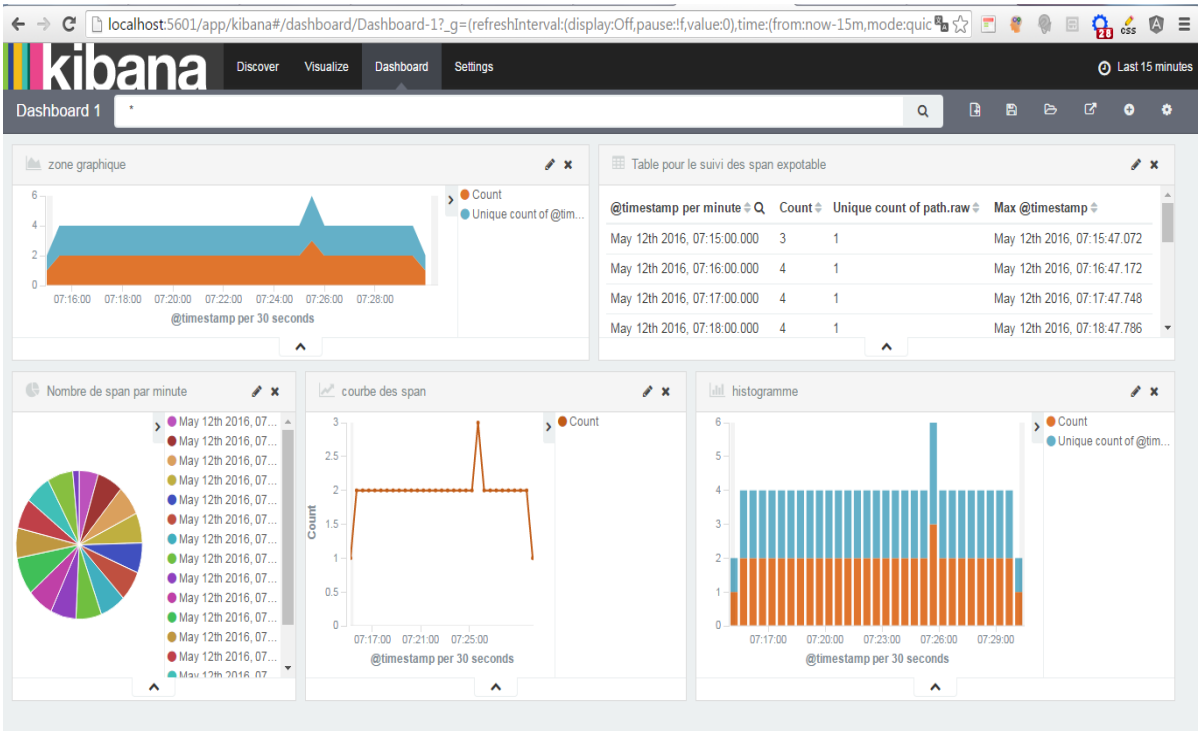


Figure 75 Exemple de dashbord avec kibana pour le portail UPR Microservices

CONCLUSION

Durant ce chapitre nous avons présenté l'unification des briques du portail de pilotage. Le déploiement d'application à base de microservices peut se révéler plus compliqués qu'une application monolithique. D'où le besoin à des solutions d'automatisation de déploiement. Le chapitre suivant décrit la phase d'exploitation du portail de pilotage.

CHAPITRE VIII DEPLOIEMENT DU PORATIL UPR

Sommaire

Introduction 94
1. Déploiement sur le Platform-As-A-Service (PaaS) Cloud Foundry 94
2. Préparation des conteneurs de déploiement (Docker) 96
3. Monitoring des conteneurs Docker 101
Conclusion 105

INTRODUCTION

Ce chapitre explique la phase de déploiement incrémental qui consiste à déployer un microservice par Sprint mis à part les services d'intégration et de supervision. C'est une phase primordiale que nous devons réaliser afin d'aboutir un déploiement continu.

1. DEPLOIEMENT SUR LE PLATFORM-AS-A-SERVICE (PAAS) CLOUD FOUNDRY

Platform as a Service (PaaS) est une catégorie de services de Cloud computing qui fournit la plateforme et l'environnement informatique nécessaire aux développeurs pour mettre en place leurs différents services et applications sur Internet.

Cloud Foundry (CF) est un PaaS développé en Ruby puis en Go qui permet de déployer, d'exécuter et de gérer la montée en charge des applications Java en mode cloud. Cloud foundry est acheté par Vmware de son créateur puis racheté par Pivotal Softeware. Nous avons utilisé Cloud Foundry Open Source Software (OSS) déployé sur l'infrastructure d'orange. La figure 76 présente l'architecture du Cloud Foundry.

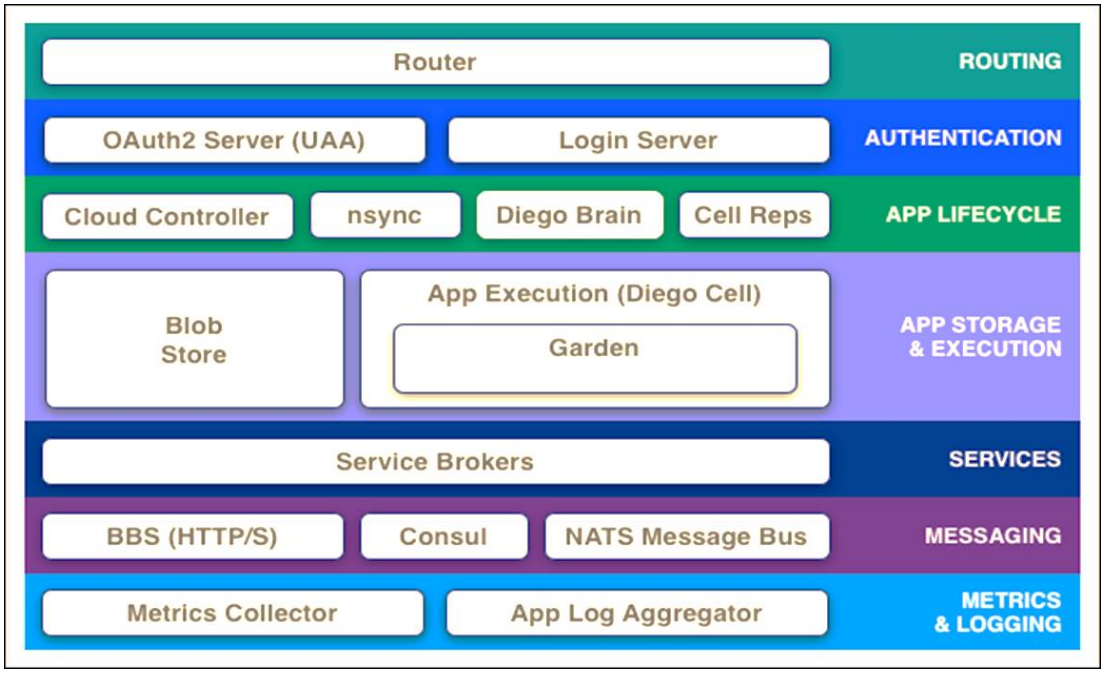


Figure 76 Architecture Cloud Foundry

- Le router intercepte les requêtes et les redirigent vers le composant approprié.
- User Account and Authentication (UAA) fonctionne pour fournir la gestion des identités.
- Le Cloud Controller (CC) dirige le déploiement des applications.

- Nsync (et autres) supervise l'état des applications déployées.
- BlopStore est le dépôt de donnée de CF, il contient le code source des applications et les Buildpacks.
- Diego Cell est responsable de l'exécution des applications et de la gestion des VM et leurs conteneurs.
- Les services brokers fournirent les services nécessaires aux applications.
- BBS et consul assurent la communication entre les composants du PaaS.
- Loggregator diffuse les logs des applications aux développeurs.

La figure 77 résume le déploiement du Portail UPR sur Cloud foundry.

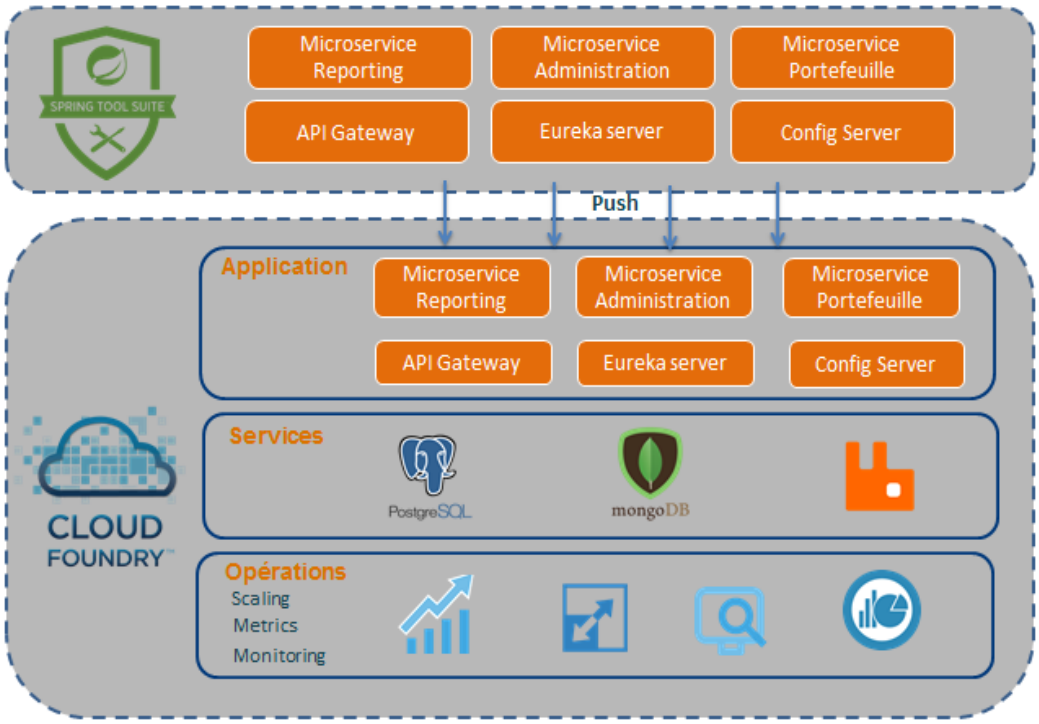


Figure 77 Déploiement au PaaS Cloud Foundry

A chaque push du code source :

1- Les bits seront interceptés par le Routeur de CF et après la procédure d'authentification seront envoyé au CC.

2- le CC envoie un message NATS qui contient des informations qui concernent l'application (App Guid, Target stack, buildpacks, url blopstore)

3- Diego stager reçoit le message, crée un RunOnce et le met dans ETCD

4- l'Executor va charger le RunOnce, crée un conteneur et exécute les actions du RunOnce.

5- Si l'opération s'achève avec succès, l'Executor marque le RunOnce comme terminé et le remet dans l'ETCD.

6- Finallement, Diego Stager retire le RunOnce et informe CC que le déploiement est terminé.

2. PREPARATION DES CONTENEURS DE DEPLOIEMENT (DOCKER)

Les conteneurs Docker encapsulent un logiciel dans un système de fichier complet qui contient tout ce dont ils ont besoin pour fonctionner (code, machine virtuelle, exécutable, outils système, bibliothèques...) Tout ce qu'on peut installer sur un serveur. Cela garantit que l'exécution sera toujours identique, quel que soit l'environnement dans lequel le conteneur sera déployé.

Docker est un outil pour les administrateurs systèmes et les développeurs. Il fournit une couche d'abstraction et d'automatisation au-dessus de fonctionnalités de virtualisation du noyau linux. Il s'appuie notamment sur :

- Les control groups (CGROUPs), qui permettent de limiter les ressources allouées à un processus (RAM, CPU, réseau, I/O...).
- Les namespaces, qui isolent les applications les unes des autres, comme si elles étaient seules dans l'OS.

Une image Docker est un template en lecture seule qui contient un logiciel configuré, un registre est un dépôt d'images et un conteneur est une instance de l'image. La figure 78 résume l'architecture de docker.

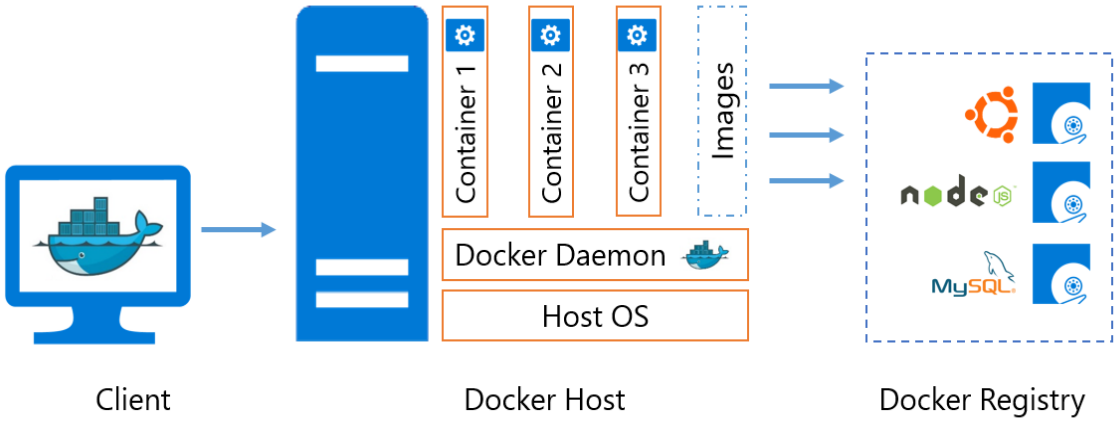


Figure 78 Architecture docker

Docker repose sur une architecture client-serveur. Le client Docker effectue des requêtes HTTP au démon. Le démon est en charge de construire, lancer et distribuer les conteneurs. Pour construire les conteneurs il s'appuie sur des images stockées sur des dépôts publics et/ou privés. Le client peut invoquer un démon local ou distant via, une API REST.

Nous avons choisi d'utiliser les conteneurs de déploiement Docker pour les intérêts et avantages qu'ils offrent. Le tableau 16 résume ces intérêts et avantages.

Tableau 16 les intérêts et les avantages de docker

Intérêts/avantages	Explication
Portabilité	L'atout principal de Docker c'est la portabilité. On peut créer un container et le faire fonctionner sur n'importe quelle machine ou service qui supporte Docker. Il y a deux limitations à cette portabilité : - il faut que l'architecture cible soit compatible (un container pour x86 ne fonctionnera pas sur ARM, et vice-versa) - le noyau linux étant celui de l'hôte, le container ne peut pas exécuter une image qui exigerait un noyau plus récent.
Démarrage rapide	un conteneur Docker démarre presque instantanément
Ressources limitées	Les conteneurs docker ne sont pas gourmands en ressource. Ils sont ultra légers.
Isolation	chaque conteneur a l'impression de disposer d'une machine entière pour lui seul.
Déploiement simple	Docker est une solution qui permet de standardiser la façon dont une application est packagée, puis s'exécute, dans n'importe quel environnement. Ce qui simplifie et accélère le déploiement.
Taille de la communauté	Docker bénéficie d'une communauté très dynamique. Il existe donc une bonne documentation fournie sur cette technologie open source.
Evolution rapide	Des changements intéressants sont apportés d'une version à une autre.

Tandis que Docker possède un ensemble d'inconvénients dont le tableau 17 met l'accent.

Tableau 17 Inconvénients de Docker

Inconvénient	Explication
Sécurité	Le démon Docker est lancé obligatoirement par l'utilisateur root, et le client doit appartenir au groupe Docker. Cela autorise ce dernier à faire une escalade de privilèges (et donc à obtenir les droits root) Par ailleurs parmi les milliers d'images Docker disponibles sur des dépôts publics, une proportion importante est basée sur des versions de logiciels possédant des failles de sécurité connues. Docker Inc. tente de régler ces deux problèmes. Nautilus (Docker inc.) et Clair (CoreOS) sont des scanners d'images qui détectent les failles de sécurité.
Evolution rapide	L'évolution très rapide a aussi un revers : ce qui est la norme à un instant t devient parfois obsolète quelques semaines plus tard.

La figure 79 montre les commandes dockers.

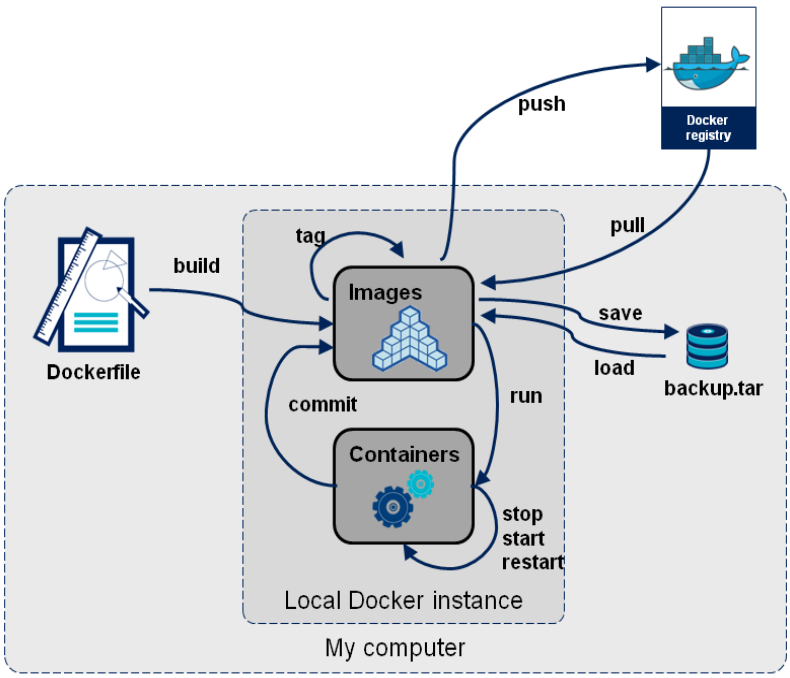


Figure 79 Les commandes docker

A chaque microservice développé et testé nous préparons un docker file qui permet la préparation de l'image et l'exécution de conteneur de ce service. Il est indispensable de pouvoir lier des conteneurs ; par exemple un serveur web et la base de données. Pour pouvoir lier deux conteneurs, il faut imposer leur nom, à l'aide de l'option « name », et lancer le second conteneur avec l'option « link » dont la valeur fait référence au nom du premier conteneur. Docker va alors fournir au conteneur du serveur web les infos lui permettant de communiquer avec la base de données.

Pour notre application en microservices nous avons utilisé Docker compose pour pouvoir démarrer et arrêter plusieurs conteneurs simultanément tout en définissant les liens entre eux.

Pour démarrer la totalité de l'application d'un seul coup avec docker compose nous suivons ces étapes :

● Décrire l'environnement de chacun des services de l'application à l'aide d'un dockerfile.
● Créer un fichier docker-compose.yml pour définir les services qui composent l'application, et compléter/surcharger certaines des propriétés définies dans leurs dockerfiles respectifs.
● Définir également les liens entre les services.
● Pour chaque service on donne un lien vers le dossier dans lequel se trouve son dockerfile, ou l'URL du dépôt Git dans lequel il est stocké.
● Lancer toute l'application avec docker-compose up

La figure 80 présente les conteneurs du portail UPR.

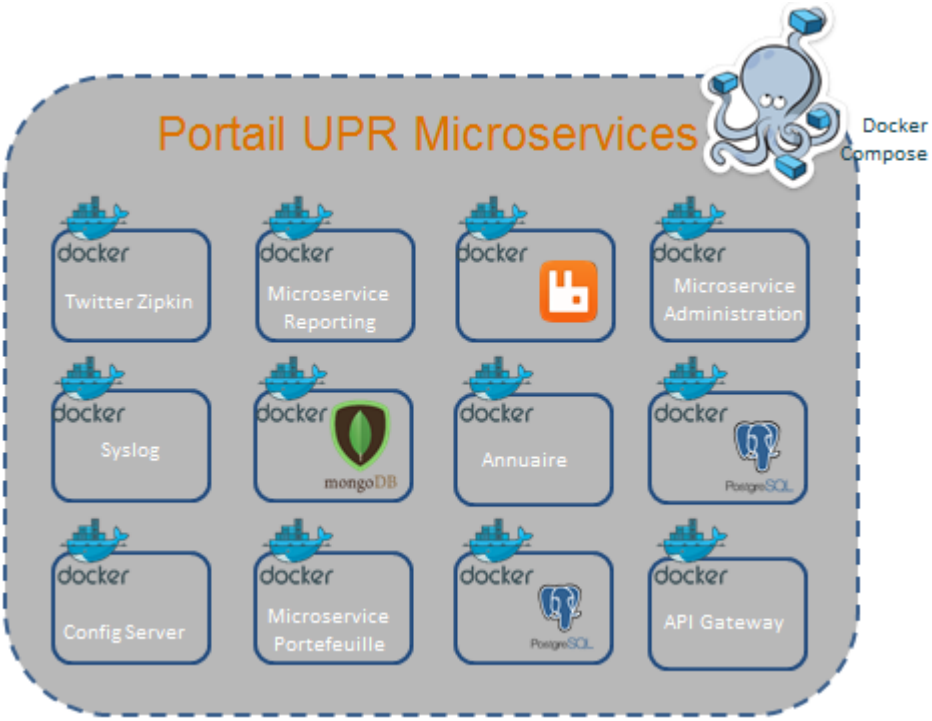


Figure 80 Conteneur du portail UPR

Nous utilisons Docker Swarm pour permettre la communication des conteneurs Docker placés sur plusieurs machines.

Le tableau 18 récapitule les bonnes pratiques avec docker.

Tableau 18 Les bonnes pratiques avec docker

Bonne pratiques	**Explication**
Une application par container	Techniquement rien n'empêche de mettre plusieurs applications au sein d'un container, mais c'est contraire à la philosophie de docker. Cela nuirait notamment à la réutilisation des containers, qui serait moins faciles à utiliser comme briques de base d'une architecture.
Limiter le nombre de couches	Même s'il est possible de se connecter en SSH sur un container, il est fortement recommandé de ne pas modifier quoi que ce soit. Il est préférable de modifier le dockerfile, puis de détruire le container existant et d'en créer un nouveau.

Immuabilité	Les containers sont éphémères. Un container doit pouvoir être stoppé, détruit, et remplacé par un autre avec un minimum de configuration et d'installation.

3. MONITORING DES CONTENEURS DOCKER

Dans cette partie nous présentons un monitoring des paramètres systèmes des conteneurs. Cette phase est nécessaire pour deux principales raisons. D'abord pour suivre le comportement de nos microservices et leurs consommations en processeur, en mémoire et en trafique réseau. Ensuite pour voir le gain de l'utilisation des conteneurs Docker en termes de consommation.

Notre Solution de monitoring consiste à :

- Installer Cdavisor (Outil de monitoring en temps réel des conteneurs Docker) sur chaque host qui porte des dockers.
- Envoyer les logs dans une instance InfulxDB (Appartient aux bases de données NOSQL pour séries temporelles qui permettent de gérer des données temporelles)
- Utiliser Grafana (Serveur de dashboard permet la visualisation des métriques) pour créer des Dashbords de supervision sur les données enregistrées dans influxdb

Pour un monitoring temps réel simple cAsvisor est largement suffisant. Mais nous avons fait recours à InfluxDB et Grafana pour premièrement sauvegarder les statistiques et avoir des graphes composés et plus adaptés aux besoins. La figure 81 décrit le concept de fonctionnement de monitoring adopté.

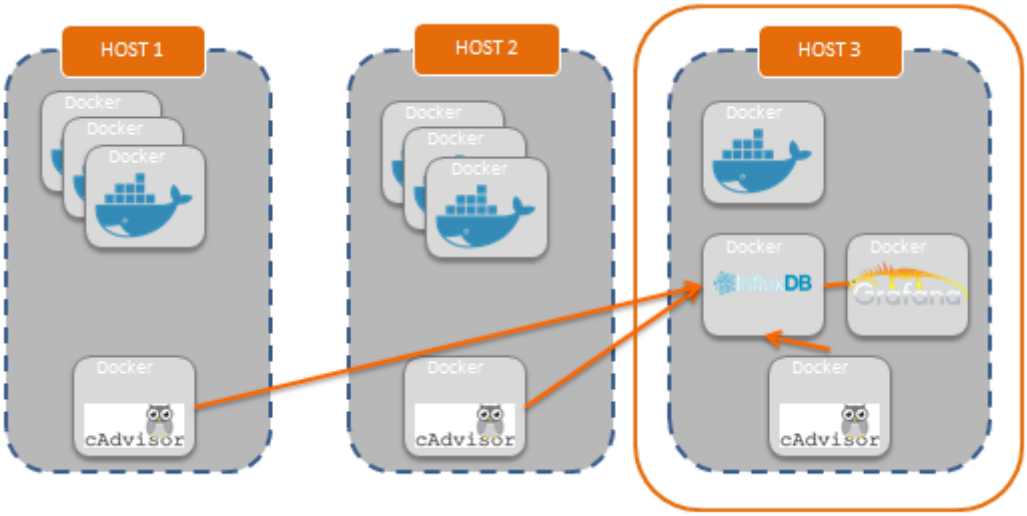


Figure 81 Monitoring des conteneurs docker

La mise en place de cette solution est réalisée par un fichier « docker-compose.yml » qui décrit les images docker des logiciels à télécharger, définit leurs environnements de fonctionnement et accorde les liens entre eux. Ce système permet un monitoring par conteneur et un monitoring sur l'ensemble. Nous commençons par une présentation de monitoring avec cAdvisor. La figure 82 présente la vue qui présente les ressources allouées à notre machine ubuntu sur laquelle nous avons installé docker daemon et docker compose.

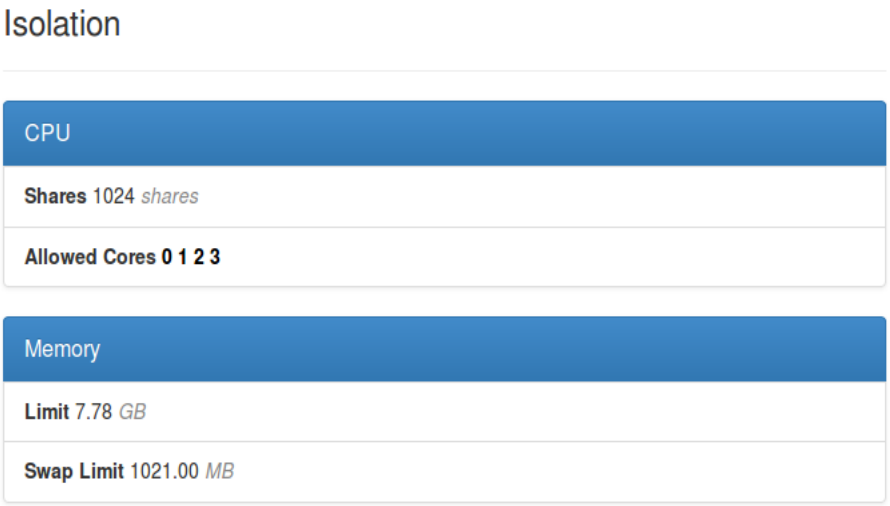


Figure 82 Ressources alloué à la machine ubuntu

La figure 83 donne un aperçu sur l'état de la machine avant le lancement de nos conteneurs. Elle présente un pourcentage de consommation de processeur, de mémoire et des fichiers système 1 et 2.

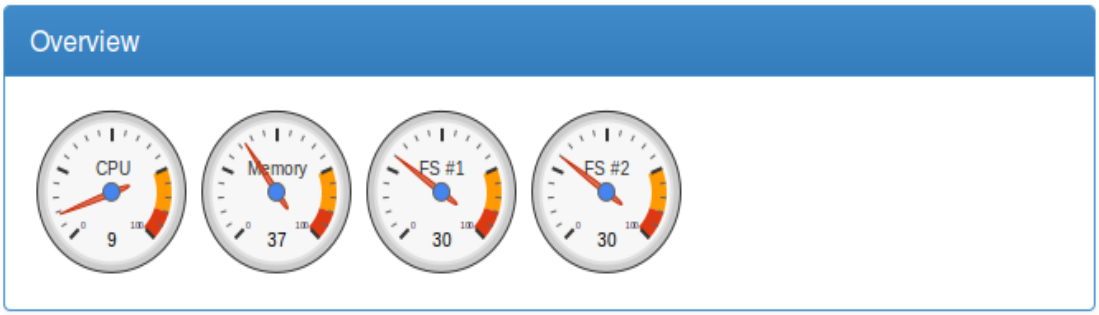


Figure 83 Aperçu général sur la consommation avant le lancement des conteneurs

Après avoir lancé l'ensemble des conteneurs avec docker compose l'aperçu sur la consommation devient comme le montre la figure 84.

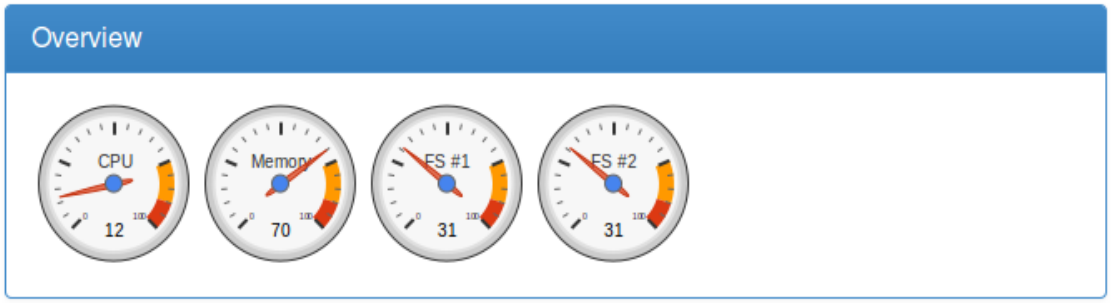


Figure 84 Aperçu général sur la consommation après le lancement des conteneurs

L'analyse des figures 83 et 84 montre une consommation modérée des multiples conteneurs des services qui formulent la plateforme de pilotage.

Le microservice reporting contient les traitements les plus gourmands en termes de consommation. Pour cela nous présentons des aperçus sur le comportement de son conteneur. La figure 85 présente un aperçu général sur l'usage des ressources en pourcentage de consommation pour le conteneur reporting sans la consommation des autres conteneurs ni la consommation système.

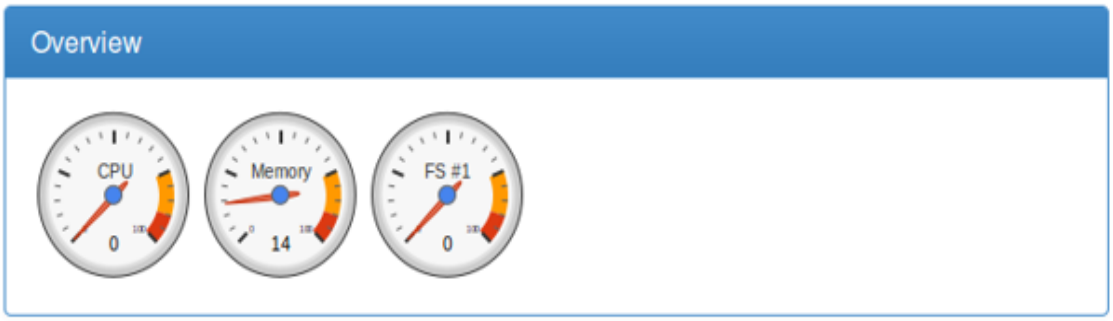


Figure 85 Aperçu général sur la consommation de conteneur de reporting seul

Cette figure montre une consommation nulle de processeurs (CPU). Réellement la consommation est autour de zéro pourcent mais pas nulle.

La figure 86 présente la courbe d'usage de CPU en temps réel par le conteneur reporting. D'après cette courbe nous constatons que les consommations sont proches de 0.015 % de CPU.

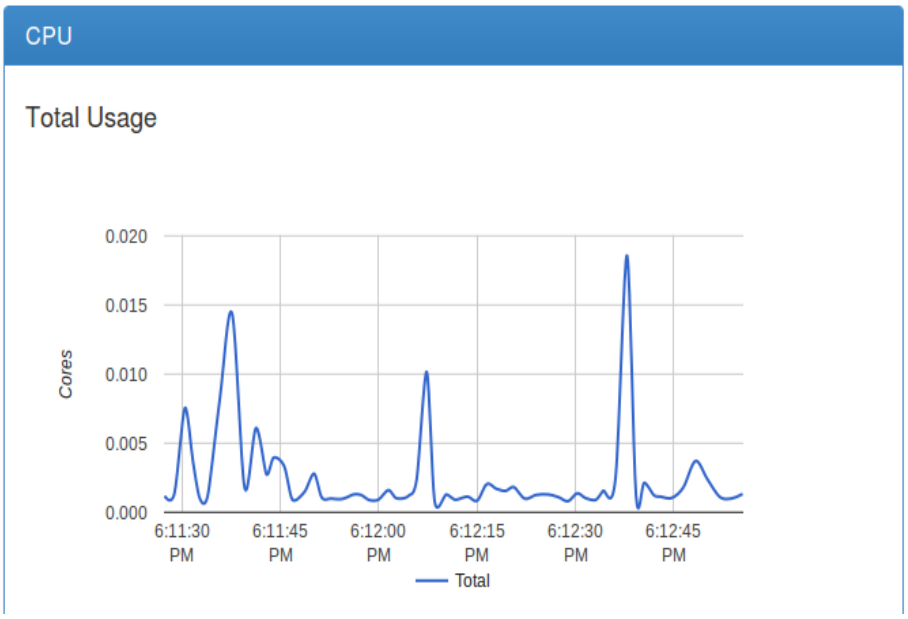


Figure 86 Usage de CPU par le conteneur reporting

La figure 87 présente les courbes de la consommation de mémoire par le conteneur reporting. A savoir la courbe rouge fait référence à l'usage de la mémoire vive « RAM » et la courbe en bleu à l'usage de la mémoire RAM et les échanges avec les autres dispositifs d'échange.

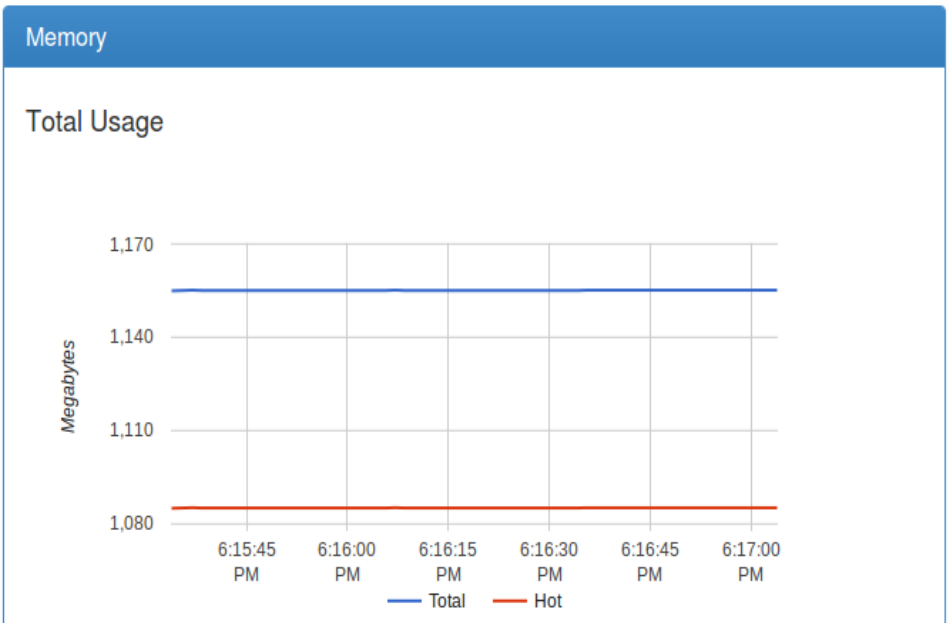


Figure 87 Usage de mémoire par le conteneur reporting

La figure 88 montre les courbes de trafique réseau de conteneur reporting. La courbe en bleu présente le trafic en transmission et la courbe en rouge présente le trafic de réception.

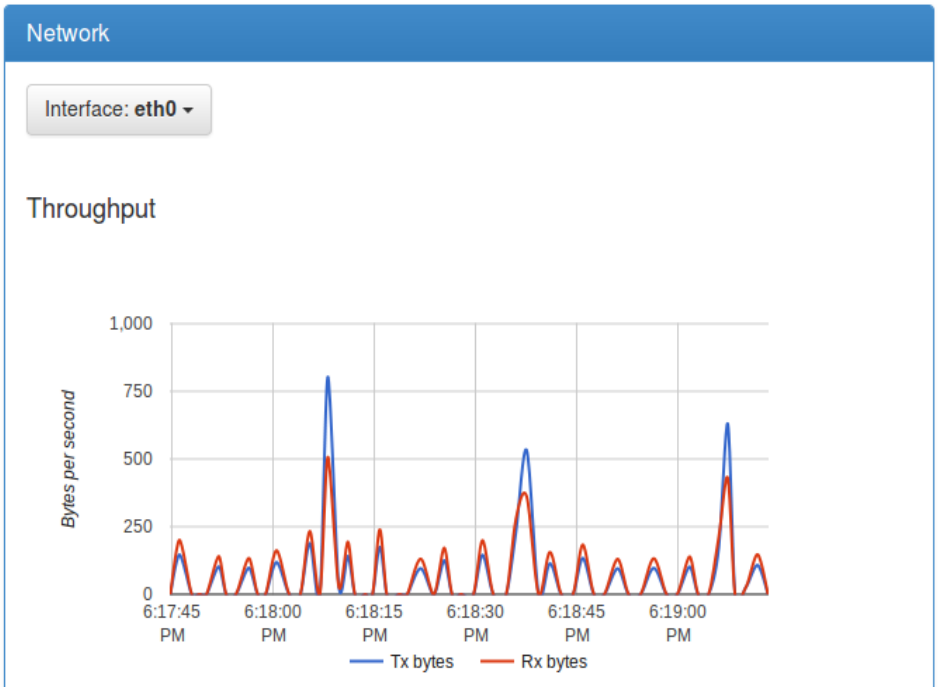


Figure 88 Usage de réseau par le conteneur reporting

Le comportement de conteneur de microservice reporting montre de plus un avantage de Docker. En effet l'élimination de la couche de virtualisation, consommatrice en ressources machines nous a permis un gain de consommation.

CONCLUSION

Ce chapitre fût l'occasion de détailler la phase de déploiement de l'application. Celle-ci nous a permis d'expliquer les différentes phases par lesquelles nous sommes passés afin de déployer la totalité de l'application.

CONCLUSION ET PRESPECTIVES

Le projet intitulé "Migration en microservices" lancé par Sofrecom Tunisie constitue un pas en avant vers une agilité de bout en bout. Son objectif issu d'une pure volonté stratégique, vise à faire évoluer son organisation, à revoir ses acquis, mais aussi à apporter des nouveaux projets à l'entreprise par le biais de son savoir-faire.

Pour ce projet pilote nous étions responsables de la mise en place de l'architecture de la solution ainsi que la conception, la réalisation, les tests et le déploiement.

Nous avons pu résoudre à travers cette migration les limites exigées par le style architectural monolithique sur les grands projets. A savoir :

- Pour chaque microservice nous choisissons les technologies qui conviennent mieux à sa réalisation selon le besoin.
- Nous pouvons mettre à l'échelle le microservice qui a besoin de plus de ressources indépendamment des autres.
- Chaque microservice est dédié à réaliser une tâche bien spécifique, donc la durée des tests est minimale.
- Chaque microservice est déployé séparément. En cas de modification impactant le périmètre d'un microservice on le redéploye seul.
- Un microservice adapte donc son fonctionnement en fonction de l'état du système.
- L'architecture microservices favorise l'agilité. Les microservices sont faiblement couplés. Chaque microservice a son cycle de vie et peut être géré par une équipe différente.

Notre projet a attiré l'attention de comité d'architecture chez Orange France. Les architectes de comité suivent l'avancement de projet chaque deux semaines avec intérêt.

Néanmoins, ces témoignages aussi rassurants qu'encourageants, ne font que nous inciter à améliorer davantage notre projet pilote et proposer de nouvelles perspectives à savoir :

- L'ajout des fonctionnalités non pas encore réalisées pour le microservice reporting
- L'adaptation avec des outils actuels de supervision de projet chez Orange (ex : Portail Unique Capacitaire)

- L’automatisation des tests d’intégration entre les microservices
- La gestion du changement et le contrôle de version (ex : Offrir des versions adaptées aux unités de production réseau hors Orange France)
- La migration vers le système de messagerie distribué Apache Kafka mais en utilisant le service de découverte Eureka

BIBLIOGRAPHIE

[B1] **K. Schwaber, M. Beedle**, Agile Software Development with SCRUM, Pearson Technology Group, 2002

[B2] **K. Schwaber**, Agile Project Management with Scrum, Microsoft Press, 2004

[B3] **R. Wilsenach**, DevOpsCulture, 9 July 2015 [en ligne], article disponible sur : http://martinfowler.com/bliki/DevOpsCulture.html (consulté le 15/04/2016)

[B4] **M. Fowler**, Martin Fowler Defines Continuous Delivery, 4 Jun 2013, article disponible sur : https://dzone.com/articles/martin-fowler-defines (consulté le 15/04/2016)

[B5] **E. Evans**, Domain-Driven Design: Tackling Complexity in the Heart of Software, 20 août 2003

[B6] **M. Fowler**, BoundedContext, 15 Jan 2014, article disponible sur : http://martinfowler.com/bliki/BoundedContext.html (consulté le 24/02/2016)

[B7] **M. Fowler, J. Lewis**, A definition of this new architectural term, 25 Mar 2014, article disponible sur : http://martinfowler.com/articles/microservices.html (consulté le 02/02/2016)

[B8] **P. Sadalage**, M. Fowler, NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence, August 2012 (consulté le 18/0/2016)

WEBOGRAPHIE

[W1] D. BOSE, Component Based Development Application In Software Engineering, Indian Statistical Institute: https://arxiv.org/ftp/arxiv/papers/1011/1011.2163.pdf (consulté le 01/04/2016)

[W2] MGILET, La persistance polyglotte avec Grails, 27 SEP 2012 : http://blog.zenika.com/2012/09/27/la-persistance-polyglotte-avec-grails/ (consulté le 01/04/2016)

[W3] Le container, http://www.containerinfo.net/le_container.htm (consulté le 03/05/2016)

[W4] S. Labourey, Conteneurs et déploiement de logiciels : la nouvelle culture du package, 23 Mar 2016, http://www.journaldunet.com/developpeur/expert/63632/conteneurs-et-deploiement-de-logiciels---la-nouvelle-culture-du-package.shtml (consulté le 04/05/2016)

[W5] Gartner, Inc., Magic Quadrant for Operational Database Management Systems by Donald Feinberg, Merv Adrian, Nick Heudecker, Adam M. Ronthal, and Terilyn Palanca, October 12, 2015 : https://www.mongodb.com/collateral/gartner-mq-2015(consulté le 07/03/2016)

[W6] Netflix, Eureka 2.0 Motivations, https://github.com/Netflix/eureka/wiki/Eureka-2.0-Motivations (consulté le 20/02/2016)

[W7] Netflix, Eureka 2.0 Architecture Overview, 31 Dec 2014, https://github.com/Netflix/eureka/wiki/Eureka-2.0-Architecture-Overview (consulté le 20/02/2016)

GLOSSAIRE

A :

- API Gateway : Application Programming Interface Gateway

Un point d'entrée unique pour les clients.

- API Rest : Application Programming Interface Gateway REpresentational State Transfer

Une interface de programmation d'application qui fait appel à des requêtes HTTP.

- ARM : Acorn RISC Machine

Architectures matérielles RISC.

B :

- BBS

Assure la communication entre les composants de Cloud Foundry.

C :

- CAMP : Culture, Automatisation, Mesure, Partage

Les piliers de DevOps.

- CAP : Consistency, Availability, Partition

Aussi connu sous le nom de théorème de Brewer dit qu'il est impossible sur un système informatique de calcul distribué de garantir en même temps les trois contraintes CAP

- CC : Cloud Controller

- CF : Cloud Foundry

- CGROUPS : Control Groups

- CPU : Contral Processing Unit

- CR : Client Received

- CRUD : Create, Read, Update and Delete

- CS : Client Sent

- CSV : Comma-Separated Values

Un format informatique ouvert représentant des données tabulaires sous forme de valeurs séparées par des virgules.

D :

- DAO : Data Access Object

Patron de conception pour les accès aux données.

- DDD : Domaine Driven Design
- DevOps :

Un mouvement visant à l'alignement de l'ensemble des équipes du système d'information, composé par la réunion des "dev" ou "Dev engineers" chargés de faire évoluer le système d'information et les "ops" ou "Ops engineers" chargés d'exploiter les applications existantes au sein d'une même équipe.

- DSI : Direction des Systèmes d'Inforamtion

E :

- ELK : Elasticsearch, Logstash, Kibana
- ESB : Entreprise Service Bus

L'ESB est une nouvelle génération d'EAI construite sur des standards comme XML, JMS ou encore les services web.

F :

- FS : File System

H :

- HTTP : HyperText Transfer Protocol

Le protocole de transfert hypertexte est un protocole de communication client-serveur développé pour le World Wide Web.

I :

- I /O : Input/Otput

J :

- JDK : Java Development Kit

- JPA : Java Persistence API

- JSON : JavaScript Object Notation

Un format de données textuelles générique.

- JUnit : Java Unit

Un framework open source pour le développement et l'exécution de tests unitaires automatisables.

- JVM : Java Virtual Machine

M :

- Mock

Des objets simulés qui reproduisent le comportement d'objets réels de manière contrôlée.

- MVC : Model View Controller

N :

- NOSQL : Not Only SQL

O :

- ORM : Object-Relational Mapping
- OS : Operating System
- OSS : Open Source Software

P :

- PaaS : Platform as a Service
- POC : Proof Of Concept

R :

- RAM : Random Access Memory
- REST : REpresentational State Transfer

S :

- SGBD : Système de Gestion de Base de Données

- SGBDRO : Système de Gestion de Base de Données Relationnel
- SI : Système d'Information
- SOA : Architectures Orientés Services
- SOAP : Simple Object Access Protocol
- SQL : Structured Query Language
- SR : Server Received
- SS : Server Sent
- STS : Spring Tool Suite

T :

- TDD : Test Driven Development

U :

- UAA : User Account and Authentication
- UML : Unified Modeling Language
- URL : Uniform Resource Locator

V :

- VM : Virtual Machine

X :

- XML : Extensible Markup Language

GLOSSAIRE MÉTIER

A :

- Activité

Composant d'une opération. Certaines activités seront définies comme des jalons à suivre sur les tableaux de pilotage.

- Article

Désigne pour chaque opération ce qui est à produire

C :

- COLORIS

Coloris fournira des données au portail qui viendront en complément des données envoyées par OPUS.

- Commande

Pour le portail un graphe = une opération = une commande = une production

D :

- Domaine

Un domaine est un regroupement de familles

- DSLAM : Digital Subscriber Line Access Multiplexer

Le DSLAM est un équipement généralement installé dans les centraux téléphoniques (répartiteurs) et utilisé pour transformer une ligne téléphonique classique en une ligne ADSL (Asymetric Digital Subscriber Line).
La fonction du DSLAM est de regrouper plusieurs lignes ADSL sur un seul support.

F :

- Famille

La famille est un regroupement d'articles et de type de flux. Chaque opération est liée à une famille en fonction de son article et de son type de flux.

- Flux/type flux

Le flux peut désigner plusieurs choses selon le contexte. Il peut s'agir :

- du flux lié à une opération qui détermine s'il s'agit d'une opération de suppression, modification, création ...
- du flux physique/réseau entre 2 machines.

G :

- GDP : Gestion De la Production.

GDP fournira des données au portail qui viendront en complément des données envoyées par OPUS. Les données GDP devront donc être rattachées aux données d'OPUS pour les présenter aux utilisateurs du portail.

- Graphe/N° Graphe

Pour le portail un graphe = une opération = une commande = une production

I :

- IDR : Ingénierie de Déploiement Réseau

J :

- Jalon

Les jalons sont les activités qui seront suivies dans le tableau de pilotage du portail.

O :

- Opérations

Il s'agit des productions qui seront envoyées par OPUS au portail.

- OPUS

C'est le principal fournisseur de données du portail. OPUS enverra une partie de ses productions au portail.

Assurer le suivi de production des produits fixes, des ressources de transmission, d'équipements et des faisceaux de circuits.

Il se situe entre la prise de commande et la facturation pour les produits permanents
Il assure la prise en compte des demandes clients internes et externes et le suivi de la production ROSI.
Il permet la maîtrise des processus de production et le respect des engagements contractuels sur les délais.

P :

- Poste de travail

Un poste de travail est un code sur 6 ou 8 chiffres qui permet d'identifier l'équipe ou la personne qui interviendra sur l'activité OPUS

S :

- Seuil

Le seuil permet de calculer la période pendant laquelle le jalon, s'il n'est pas réalisé, devient critique.

U :

- UI : Unité d'intervention
- UPR : Unité de Production

ANNEXES

Annexe A: Les familles des bases NOSQL

La découpe en famille des base NOSQL n'est pas stricte ni normalisée. Elle facilite juste l'abord du domaine. Certaines bases ne répondent pas exactement aux définitions qui vont suivre. Parmi les familles des bases NOSQL nous citons :

- **Bases orientées colonnes**

Ce modèle ressemble à une table dans un SGBDR sauf que dans cette famille le nombre de colonnes est dynamique. En effet avec ce modèle, le nombre de colonnes peut varier d'un enregistrement à un autre ce qui évite de retrouver des colonnes ayant des valeurs NULL.

- **Bases orientées clés/valeurs**

C'est une sorte de Hashmap distribuée conçue pour sauvegarder les données sans définir de schéma. Toutes les données sont sous forme de clef/valeur

- **Bases orientées documents**

Ce modèle étend le paradigme clef/valeur avec pour valeur un document de type JSON ou XML. Chaque document est un objet qui contient un ou plusieurs champs, et chaque champ contient une valeur typée : string, date, binary ou array. L'avantage est de pouvoir récupérer, via une seule clé, un ensemble d'informations structurées de manière hiérarchique. La même opération dans le monde relationnel impliquerait plusieurs jointures.

- **Bases orientées graphes**

Ce modèle est basé sur la théorie des graphes. Il s'appuie sur la notion de nœuds, de relations et de propriétés qui leur sont rattachées. Chaque élément contient directement un pointeur sur ceux avec lesquels il est en relation et donc il n'est pas nécessaire de refaire une requête pour parcourir une relation. Ce modèle est adapté au traitement des données des réseaux sociaux.

- **Bases pour séries temporelles**

Elles sont optimisées pour la gestion de données historisées et aux gros volumes immuables.

Annexe B: Simulation avec les services de file d'attente de messages

La configuration de la machine sur laquelle nous faisons les simulations est comme suit :

- RAM= 8.00 Go
- CPU=2.50GHz
- 4 Core

1. Simulations avec ActiveMq

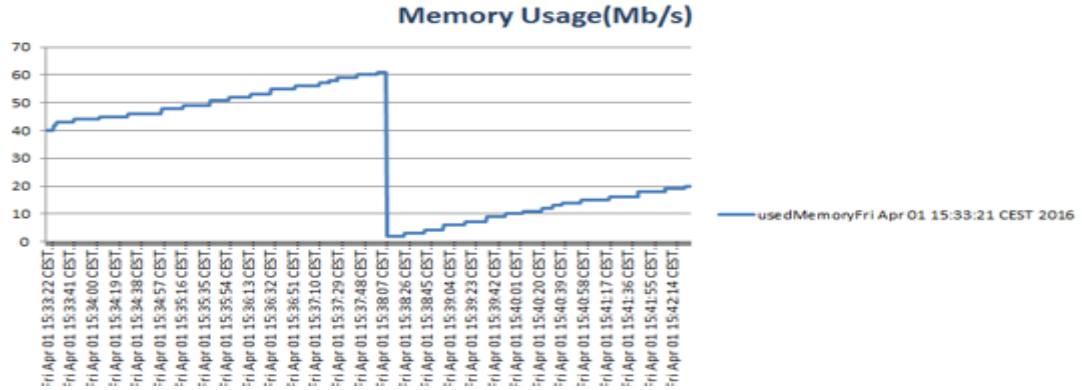
a) Mode send-receive

- Cas d'utilisation 1 : Envoi de 10 000 msg

La figure suivante montre le taux d'envoi des messages en fonction de temps.



La consommation de mémoire est illustrée par la figure suivante.



Ce tableau présente le temps d'envoi total des 10 000 messages selon le nombre de threads.

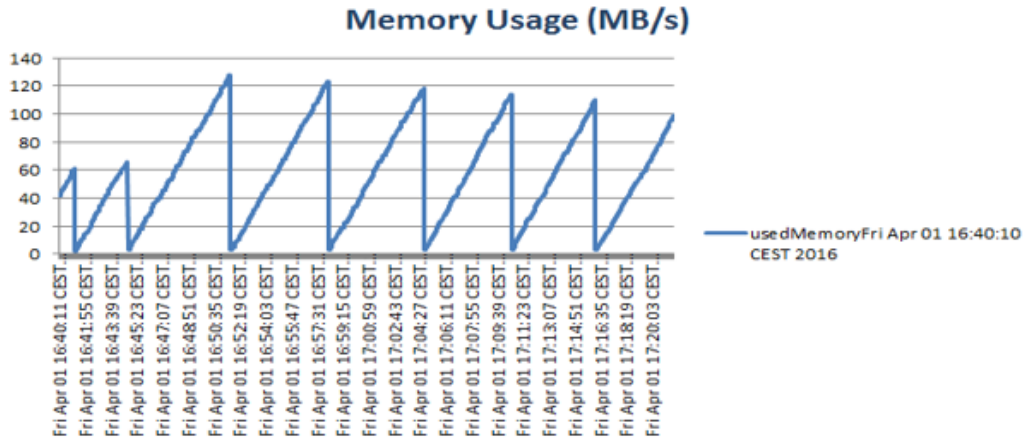
Nbr Threads	Time
1	06min04
10	04min54
100	05min25

- Cas d'utilisation 2 : Envoi de 100 000 msg

La figure suivante montre le taux d'envoi des messages en fonction de temps.



La consommation de mémoire est illustrée par la figure suivante.



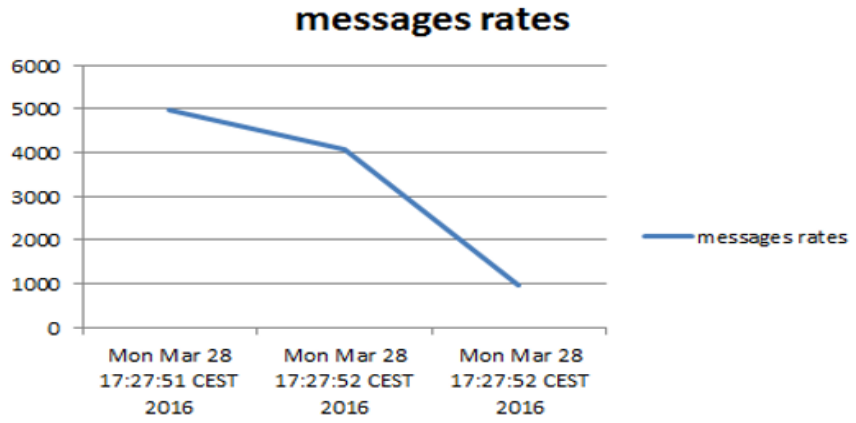
Ce tableau présente le temps d'envoi total des 100 000 messages selon le nombre de threads.

ActivMq envoi environ 40 message par seconde.

Threads	Time
1	45min59
10	45min13
100	49min42

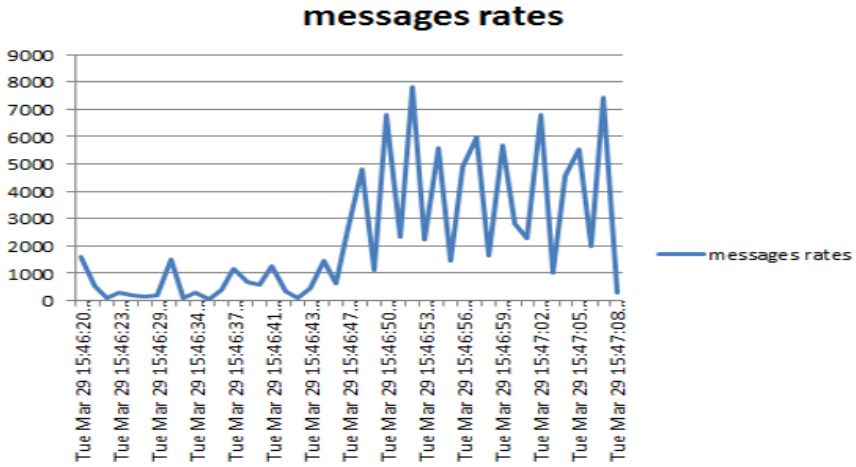
- Cas d'utilisation 3 : Réception de 10 000 msg

La figure suivante montre le taux de réception des messages en fonction de temps. La réception a duré 4 sec.



- Cas d'utilisation 4 : Réception de 100 000 messages

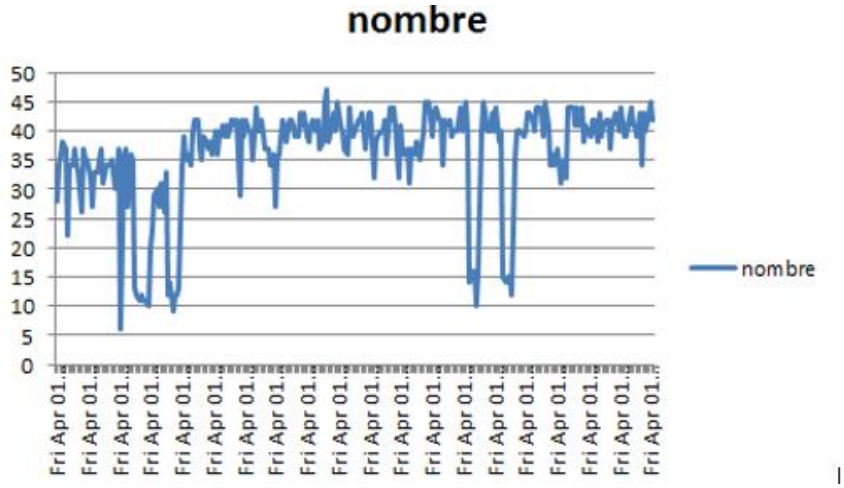
La figure suivante montre le taux de la réception des messages en fonction de temps. La réception a duré 48 sec



b) Mode pub-sub

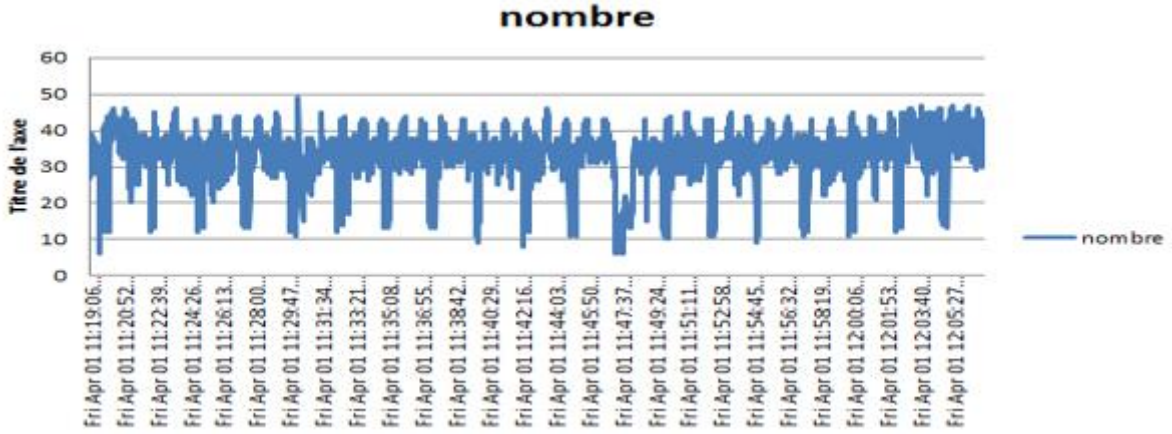
- Cas d'utilisation 1 : Publication de 10 000 messages

La figure suivante montre le nombre de messages publiés en fonction de temps. La publication a duré 4 min 29 sec.

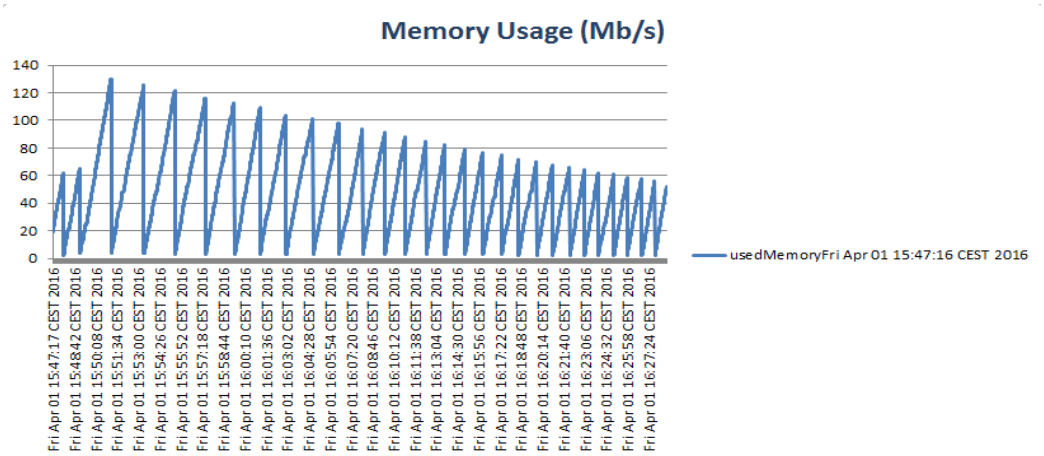


- Cas d'utilisation 2 : Publication de 100 000 messages

La figure suivante montre le nombre de messages publiés en fonction de temps. La publication a duré 48 min 02 sec.



La figure suivante montre l'utilisation de la mémoire au cours du temps.

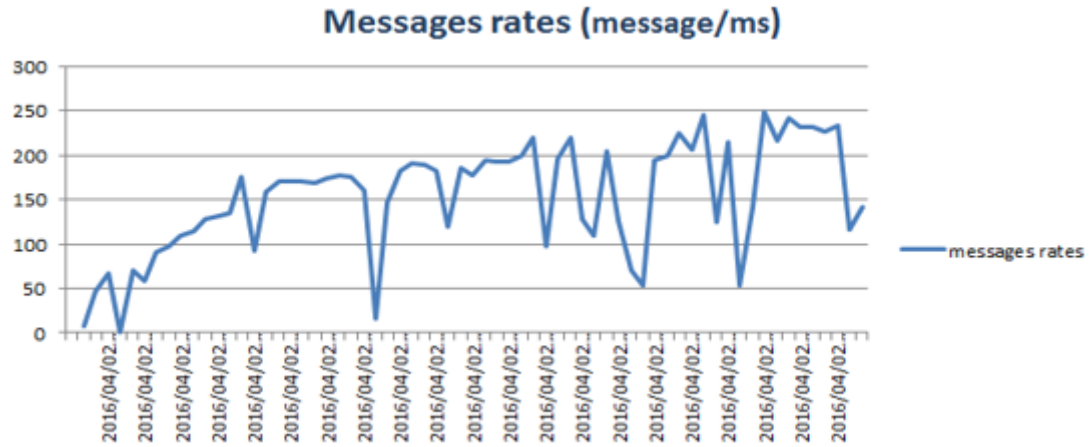


2. Simulations avec RabbitMq

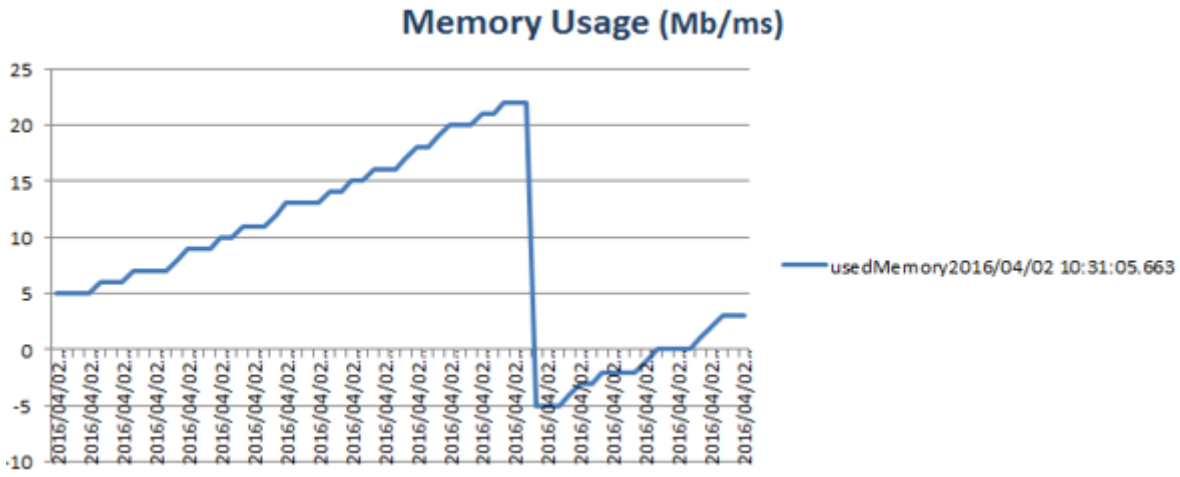
c) Mode send-receive

- Cas d'utilisation 1 : Envoi de 10 000 msg

La figure suivante montre le taux d'envoi des messages en fonction de temps.



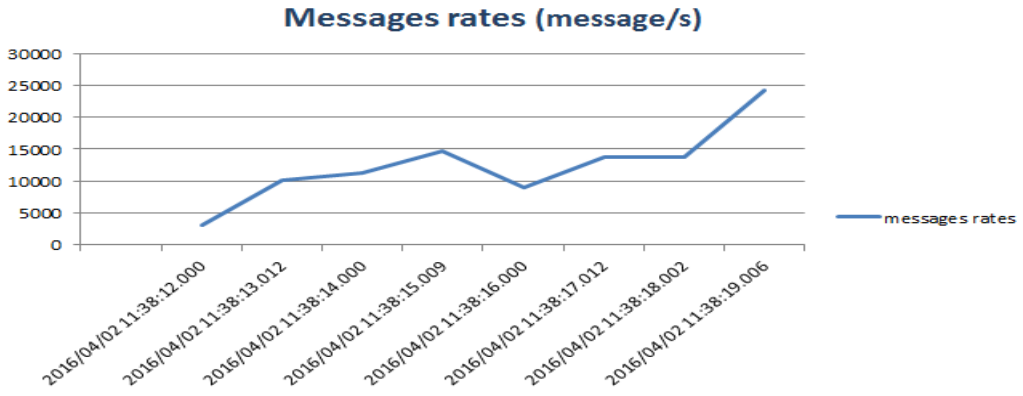
La consommation de mémoire est illustrée par la figure suivante.



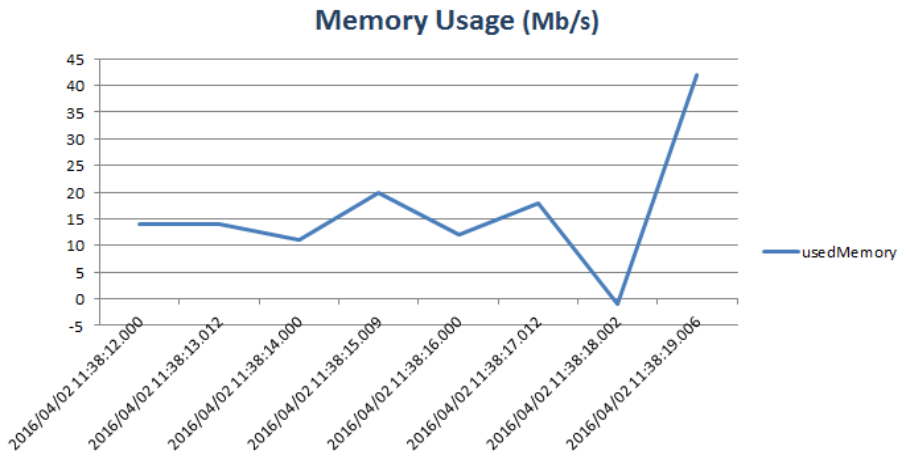
Le temps d'envoi est 1 sec 529 ms.

- Cas d'utilisation 2 : Envoi de 100 000 msg

La figure suivante montre taux d'envoi des messages en fonction de temps.



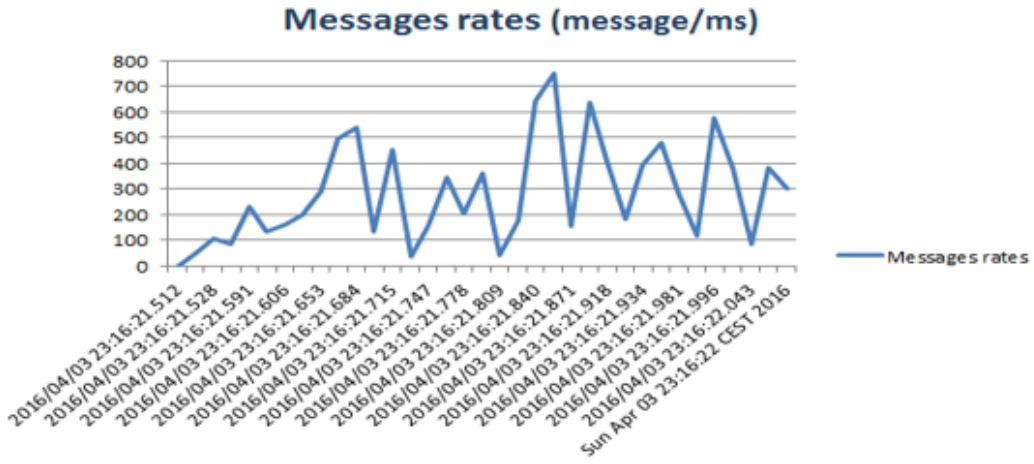
La consommation de mémoire est illustrée par la figure suivante.



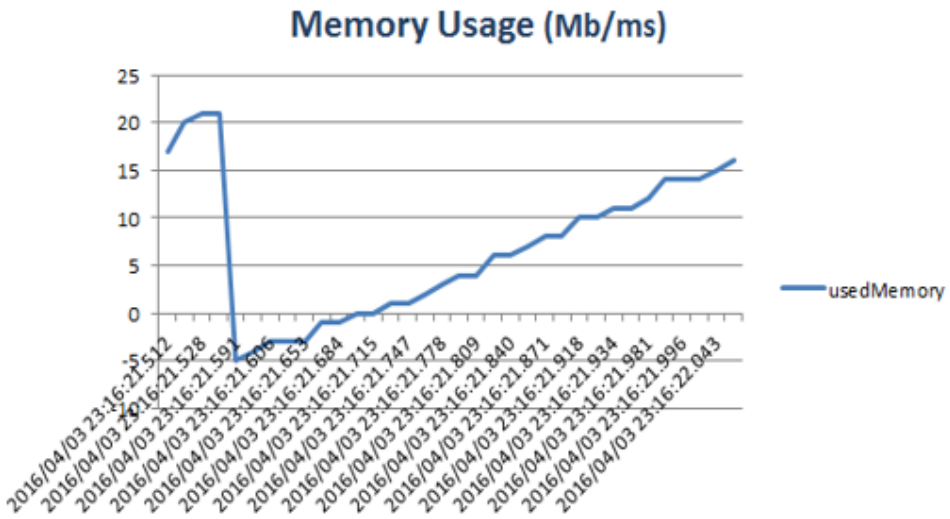
Le temps d'envoi égale à 8 secondes.

- Cas d'utilisation 3 : Réception de 10 000 msg

La figure suivante montre taux de réception des messages en fonction de temps. La réception a duré 1 sec 784 ms.

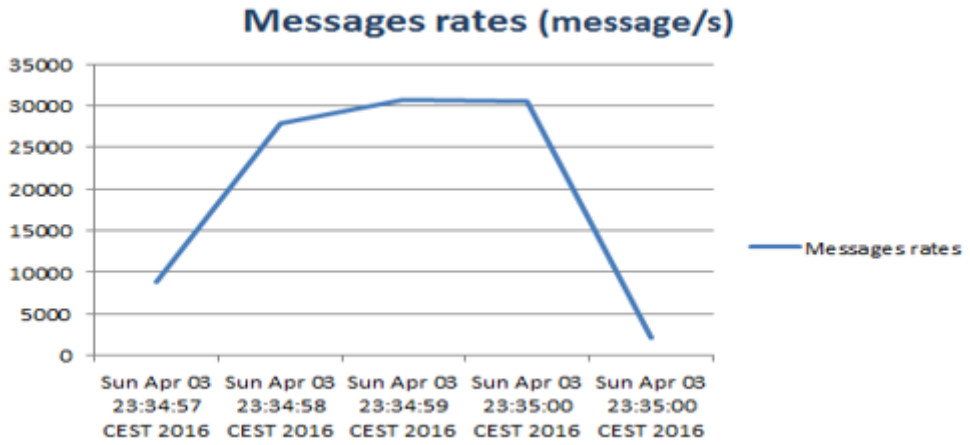


La figure suivante illustre la consommation de la mémoire au cours du temps.

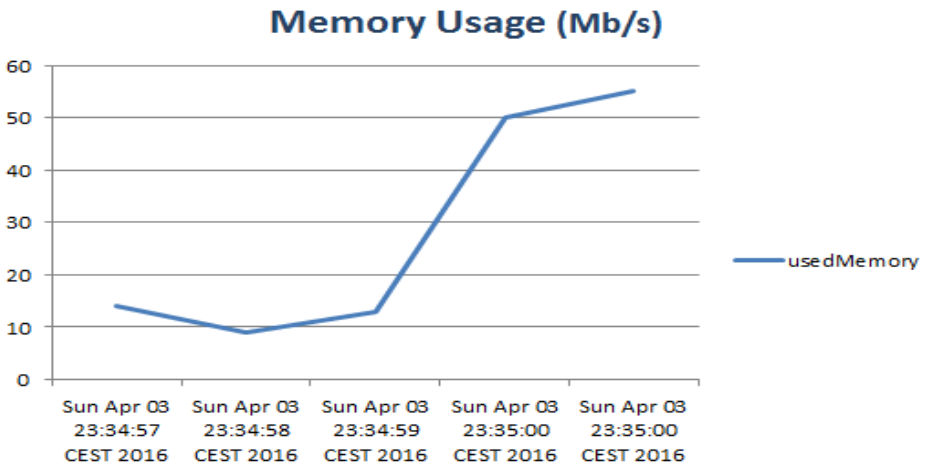


- Cas d'utilisation 4 : Réception de 100 000 messages

La figure suivante montre taux de réception des messages en fonction de temps. La réception a duré 5 sec



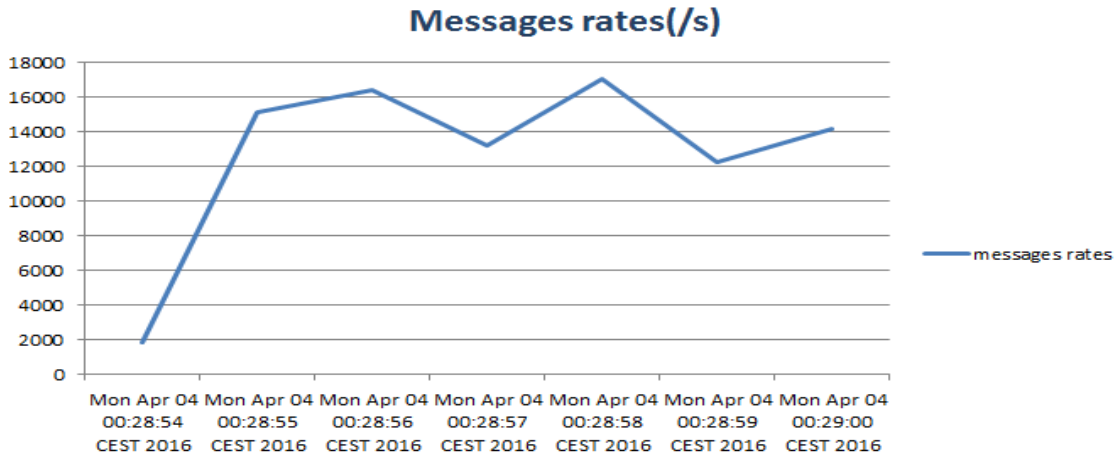
La figure suivante illustre la consommation de mémoire au cours du temps.



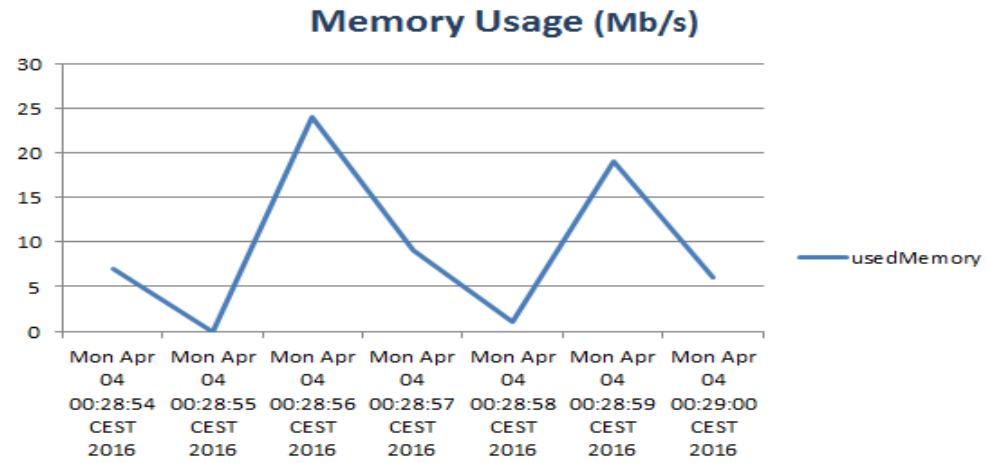
d) Mode pub-sub

- Cas d'utilisation 1 : Publication de 100 000 messages

La figure suivante montre le nombre de messages publiés en fonction de temps. La publication a duré 7 sec.



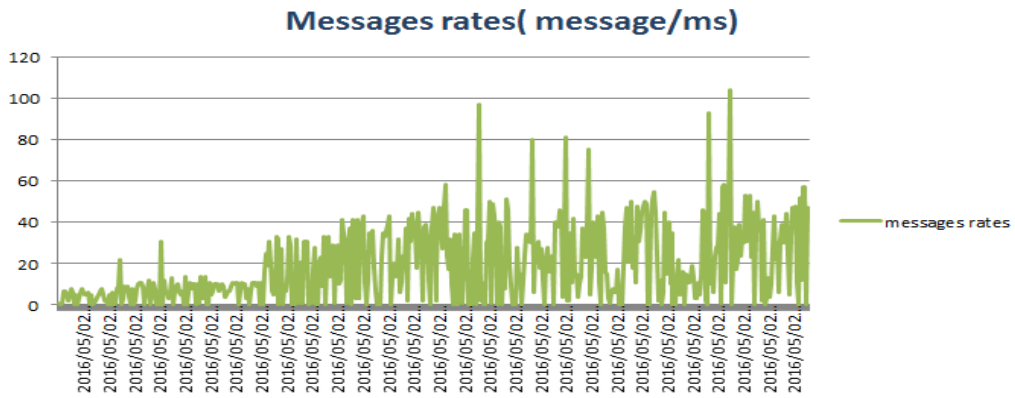
La figure suivante illustre la consommation de mémoire au cours du temps.



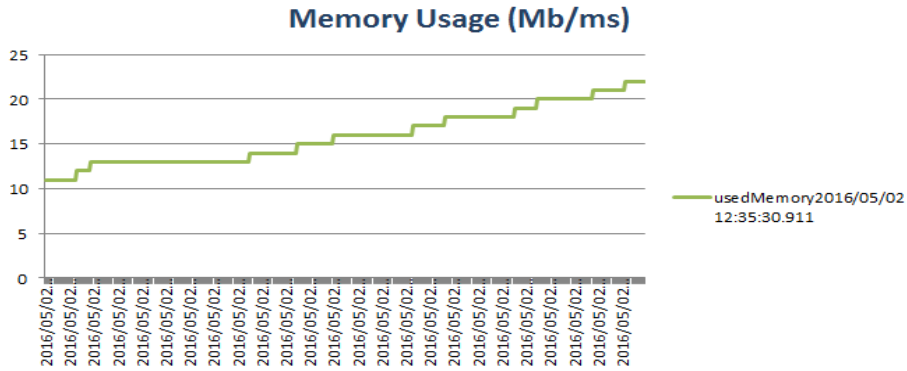
e) Simulations avec Kafka (Mode send-receive)

- Cas d'utilisation 1 : Envoi de 10 000 msg

La figure suivante montre le taux d'envoi des messages en fonction de temps. Il dure 1 sec 168.

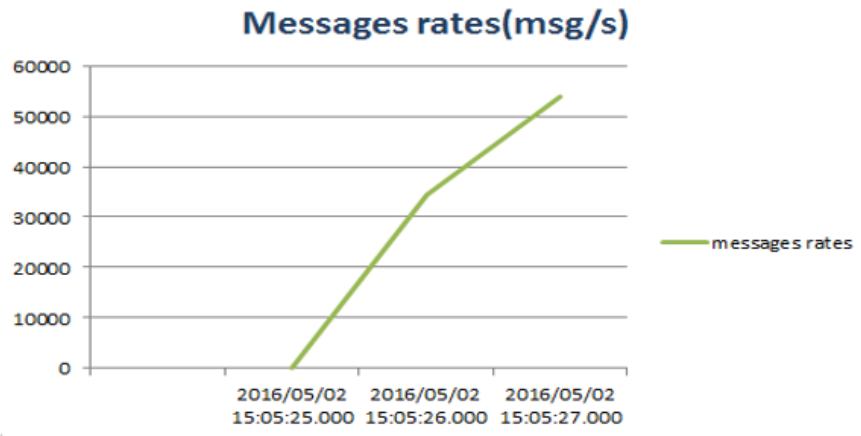


La figure suivante illustre la consommation de mémoire au cours du temps.

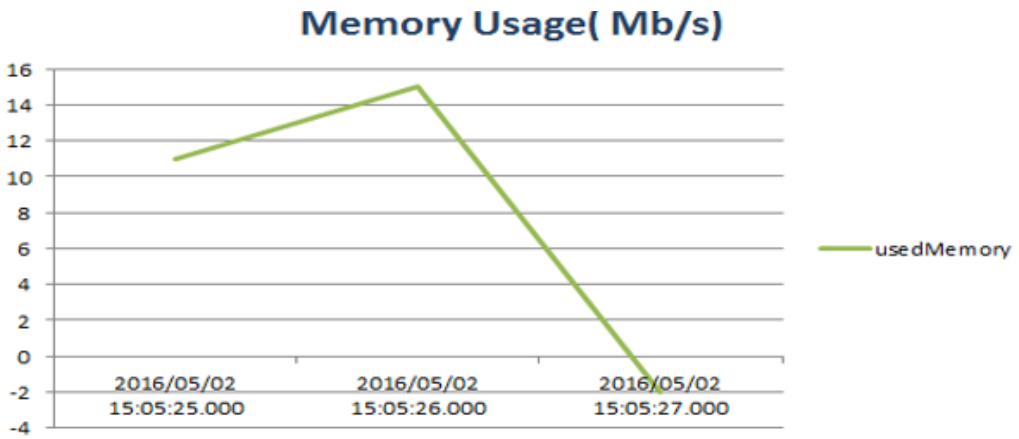


- Cas d'utilisation 2 : Envoi de 100 000 msg

La figure suivante montre le taux d'envoi des messages en fonction de temps qui a duré 3 sec.

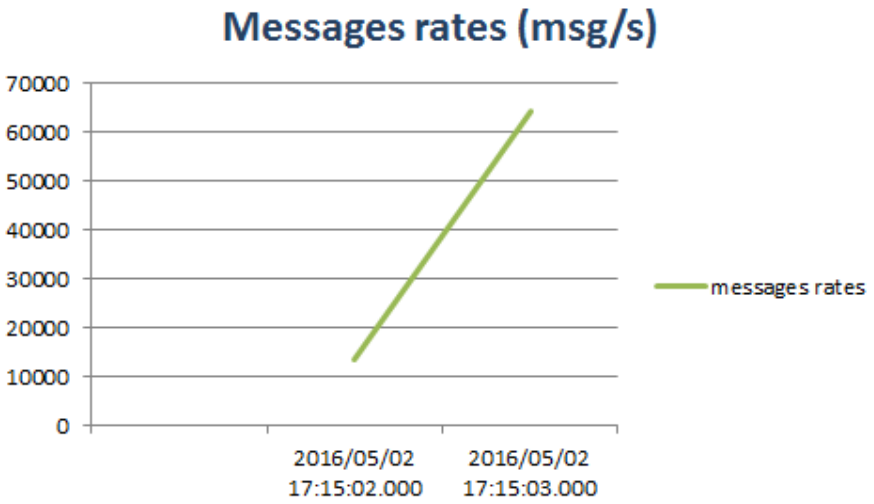


La figure suivante illustre la consommation de mémoire au cours du temps.

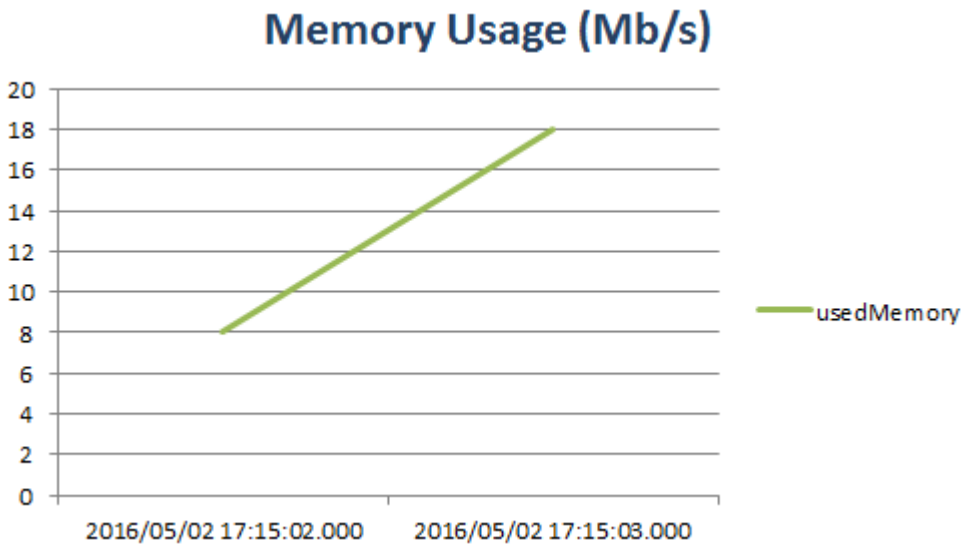


- <u>Cas d'utilisation 3 : Réception de 100 000 messages</u>

La figure suivante montre le taux d'envoi des messages en fonction de temps qui a duré 2 sec.



La figure suivante illustre la consommation de mémoire au cours du temps.



العنوان : إعادة تصميم منصة ذات صبغة متراصة لتوجيه النشر الشبكي باستخدام Microservices

ملخص: يعرض هذا التقرير الأعمال المنجزة في إطار تربص نهاية الدراسة للحصول على دبلوم مهندس وطني في هندسة البرمجيات داخل المؤسسة سوفركوم.كانت المهمة إعادة بناء بنية نشر شبكة لمنصة متراصة على أساس ميكروسرفيس. لتنفيذ هذا المشروع، بدأنا بدراسة الموجود للتحقق من الإخفاقات الحالية. في الخطوة الثانية، قمنا بدراسة نظرية أدت إلى تطوير دورات حياة ميكروسرفيس. ومع كل ميكروسرفيس وصفنا خصائصه،التصميم، التنفيذ و النشر في حاوية مستقلة. في الخطوة الثالثة، قمنا بإدماج المنصة و اعداد وسائل الاشراف عليها.

المفاتيح : ميكروسرفيس، عاجل، التطوير و التشغيل، التصميم حسب المجال، النشر المستمر

Refonte architecturale d'une plateforme monolithique de pilotage de déploiement réseau en utilisant des Microservices

Résumé : Le présent rapport synthétise le travail effectué dans le cadre du projet de fin d'études pour l'obtention du diplôme national d'ingénieur en génie logiciel au sein de l'entreprise Sofrecom. Le travail consiste à refaire l'architecture d'une plateforme monolithique de pilotage de déploiement réseau en se basant sur des Microservices. Pour mener à bien ce projet, nous avons commencé par une étude de l'existant pour dégager les défaillances. Dans un deuxième temps, nous avons mené une étude théorique qui a abouti à l'élaboration des cycles de vie de constitution des Microservices. Pour chaque Microservice nous avons décrit ses fonctionnalités, sa conception et son implémentation et son déploiement isolé. Dans un troisième temps, nous avons intégré la plateforme et nous avons préparé des solutions pour sa supervision.

Mots-clés : Microservices, Agile, DevOps, Domain Driven Design, Déploiement continu

Architectural redesign of a monolithic platform for network deployment controlling using Microservices

Abstract: This report synthesizes the work done in the project graduation for obtaining the national software engineer degree in Sofrecom company. We were tasked to rebuild the architecture of a monolithic platform for network deployment controlling based on Microservices. To carry out this project, we started with a study of the existing system. In time, we conducted a theoretical study that resulted in the development of a life cycle for Microservices construction. Then, for each Microservice we described its features, its design, implementation and containerization. Finally, we integrated the platform and we prepared a solution for its monitoring.

Keywords: Microservices, Agile, DevOps, Domain Driven Design, Continuous deployment

Intitule et adresse complète de l'entreprise :

Entreprise : Sofrecom Tunisie
Adresse : Immeuble Matrix – Berges du lac 01 – 1053
Tél. : +216 31 300 778
Email : http://www.sofrecom.com